University of Tunis El Manar

National Engineering School of Tunis

Information and Communication Technologies Department

End-of-Studies Project

Submitted by:

Insaf FARES

In partial fulfillment of the requirements for the:

National Engineering Degree in Telecommunications

Design and Development of a Supervision Tool for the Port Control Protocol (PCP) in an IPv6 Environment

---

Carried out at:
TELNET

Presented on: **04/09/2026**

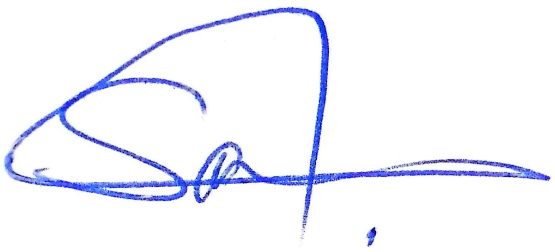
Jury Members:

President:	**Mrs. Meriem KASSAR**
Reviewer:	**Mr. Anis BEN AISSA**
Company Supervisor:	**Mr. Khalil KARBOUL**
ENIT Supervisor:	**Mrs. Aida BEN CHEHIDA**

Academic Year: 2025-2026

Signatures

Host Company Supervisor: Mr Khalil KARBOUL



ENIT Supervisor: Mrs. Aida BEN CHEHIDA

Dedication

*To **Allah**, for His guidance, blessings, and strength throughout this journey, and for giving me the patience and determination to reach this important milestone. To my beloved father, **Mohammed**, whose sacrifices, strength, constant support, and unconditional belief in me have always been a source of courage and motivation. Your presence beside me throughout every stage of my life has given me the confidence to continue moving forward and to pursue my goals. This achievement would not have been possible without you. To my beloved mother, **Awatef**, whose endless love, patience, prayers, and encouragement have accompanied me through every difficult and rewarding moment. Your kindness, care, and unwavering support have been one of the greatest sources of strength throughout my journey. To my grandfather, for his love, wisdom, prayers, and encouragement. To my sisters, for their love, presence, and continuous encouragement. To my aunt **Zohra** and my uncle **Mohammed**, for their kindness, encouragement, and constant support. To my friends, whose companionship and encouragement made this journey more meaningful. To everyone who supported me, directly or indirectly, along the way, I dedicate this work with sincere gratitude and appreciation.*
Thank you.

Insaf

Acknowledgments

First and foremost, I would like to express my sincere gratitude to **TELNET** for giving me the opportunity to carry out my final-year project within a professional, innovative, and enriching environment. This experience allowed me to strengthen my technical skills, discover real telecommunication validation challenges, and apply my academic knowledge to the development of a practical engineering solution.

I would like to express my deepest gratitude and sincere appreciation to my company supervisor, **Mr. Khalil Karboul**, for his availability, valuable advice, continuous guidance, and support throughout the completion of this work. His experience and recommendations greatly contributed to the progress and quality of this project.

I would also like to extend my sincere thanks and respect to my academic supervisor, **Mrs. Aida BEN CHEHIDA**, for her valuable guidance, constructive remarks, and support during the preparation and writing of this report. Her feedback greatly helped me improve its quality, structure, and presentation.

I would like to express my deep respect and appreciation to the members of the jury for the honor of accepting to examine and evaluate this work. I am grateful for the time and attention devoted to the assessment of this project.

To everyone who contributed, directly or indirectly, to the completion of this work, I express my sincere gratitude.

Abstract

This end-of-study project was carried out at TELNET in the context of home gateway (HGW) firmware validation. During HGW firmware development, validation engineers verify newly implemented network features and ensure that previously validated functionalities remain unaffected. For PCP and UPnP, this process involves protocol requests, gateway inspection, packet analysis, log verification, and repeated regression scenarios. Since these operations relied on several independent tools, the existing workflow was time-consuming, repetitive, and difficult to analyze globally.

The objective of this project was to design and develop NetSupervision, a centralized platform dedicated to PCP and UPnP validation in IPv6 and DS-Lite environments. The platform combines a web application with distributed LAN Agents that preserve the actual validation network context and a modified PCP server capable of reproducing controlled error scenarios. It automates the main validation operations and supports firmware regression campaigns by allowing engineers to rerun predefined test scenarios after firmware changes and verify whether the expected behavior is preserved. The platform also centralizes protocol exchanges, HGW states, mappings, errors, debug information, and regression results.

The developed solution was evaluated through representative validation and regression scenarios on different HGW models. The results demonstrated improved test reproducibility, traceability, and visibility while significantly reducing manual effort and execution time. In particular, thirteen PCP error scenarios that previously required approximately six and a half hours can now be executed in about seven minutes using the automated workflow.

Keywords : Home Gateway, PCP, UPnP, DS-Lite, Network Validation, Test Automation, Regression Testing.

Résumé

Ce projet de fin d'études a été réalisé au sein de **TELNET** dans le contexte de la validation des firmwares de passerelles domestiques. Au cours du développement d'un firmware de home gateway, les ingénieurs de validation vérifient les nouvelles fonctionnalités réseau implémentées et s'assurent que les fonctionnalités précédemment validées restent fonctionnelles. Dans le cas des protocoles PCP et UPnP, ce processus implique l'exécution de requêtes, l'inspection de la passerelle, l'analyse des paquets, la vérification des journaux et l'exécution répétée de scénarios de régression. Comme ces opérations reposaient sur plusieurs outils indépendants, le processus existant était long, répétitif et difficile à analyser dans son ensemble.

L'objectif de ce projet a donc été de concevoir et de développer **NetSupervision**, une plateforme centralisée dédiée à la validation de PCP et UPnP dans des environnements IPv6 et DS-Lite. La plateforme associe une application web à des **LAN Agents** distribués permettant de préserver le contexte réseau réel de validation, et s'appuie sur un serveur PCP modifié afin de reproduire des scénarios d'erreur contrôlés. Elle automatise les principales opérations de validation et prend en charge des campagnes de régression du firmware, permettant aux ingénieurs de rejouer des scénarios prédéfinis après des modifications du firmware et de vérifier que le comportement attendu est préservé. La plateforme fournit également une vue unifiée des échanges protocolaires, des états de la passerelle, des mappings, des erreurs, des informations de débogage et des résultats de régression.

La solution développée a été évaluée à travers des scénarios représentatifs de validation et de régression sur différents modèles de passerelles domestiques. Les résultats ont montré une réduction importante de l'effort manuel et du temps d'exécution, ainsi qu'une amélioration de la reproductibilité, de la traçabilité et de la visibilité sur le comportement des passerelles. À titre d'exemple, treize scénarios d'erreur PCP qui nécessitaient auparavant environ six heures et demie peuvent être exécutés en approximativement sept minutes grâce au processus automatisé.

Mots-clés : Passerelle domestique, PCP, UPnP, DS-Lite, validation réseau, automatisation des tests, tests de régression.

Contents

List of Figures **x**

List of Tables **xii**

List of Acronyms **xiii**

General Introduction **1**

1 Project Context and Technical Foundations **3**
Introduction . . . 3
1.1 Host Company Presentation . . . 3
1.1.1 Overview of Telnet . . . 3
1.1.2 Telecommunications Department . . . 4
1.2 Project Presentation . . . 4
1.2.1 General Context . . . 5
1.2.2 Problem Statement . . . 5
1.2.3 Project Objectives . . . 6
1.3 Project Methodology . . . 6
1.3.1 Work Approach . . . 6
1.3.2 Development Phases . . . 6
1.4 Technical Foundations . . . 7
1.4.1 IPv4 Exhaustion, NAT and CGNAT . . . 7
1.4.2 IPv6 and Transition Mechanisms . . . 8
1.4.2.1 Main Transition Mechanisms . . . 8
1.4.3 DS-Lite Architecture and Deployment Context . . . 8
1.4.3.1 Overview of DS-Lite . . . 8
1.4.3.2 Main Components of the DS-Lite Architecture . . . 9
1.4.3.3 Operational Principle of DS-Lite . . . 9
1.4.3.4 Integration of PCP in DS-Lite . . . 9
1.4.4 Port Control Protocol (PCP) . . . 10
1.4.4.1 PCP Architecture and Operation . . . 10
1.4.4.2 UPnP–PCP Interworking Function (IWF) . . . 11
1.4.5 Network Supervision Concepts . . . 12
1.4.5.1 Overview of Network Supervision . . . 12
1.4.5.2 Supervision Approaches . . . 12
Conclusion . . . 13

2 Existing Validation Approaches and Requirements Analysis **14**

Introduction . . . 14
2.1 Existing Validation Workflow . . . 14
2.1.1 IPv6 Validation Workflow . . . 15
2.1.1.1 Environment Preparation . . . 15
2.1.1.2 PCP Request Execution . . . 15
2.1.1.3 PCP Exchange Verification . . . 15
2.1.2 IPv4 Validation Workflow in a DS-Lite Environment . . . 16
2.1.2.1 Preparation of the DS-Lite Validation Environment . . . 17
2.1.2.2 UPnP Request Execution . . . 17
2.1.2.3 PCP Translation Verification . . . 18
2.1.2.4 PCP Exchange and Final Verification . . . 19
2.1.3 Validation Tools Overview . . . 20
2.1.4 Collecting Gateway Debug Information . . . 21
2.1.5 Issue Reporting . . . 22
2.2 Limitations of Existing Approaches . . . 23
2.2.1 Lack of PCP Visibility . . . 23
2.2.2 Fragmented Validation Workflow . . . 23
2.2.3 Limited Error Analysis . . . 24
2.2.4 Manual Mapping Verification . . . 24
2.2.5 Lack of Automation . . . 24
2.3 Requirements Analysis . . . 25
2.3.1 Functional Requirements . . . 25
2.3.1.1 PCP Request Management . . . 25
2.3.1.2 PCP Error Handling and Response Analysis . . . 25
2.3.1.3 UPnP Testing and Interworking Verification . . . 26
2.3.1.4 Packet Capture and Logging . . . 26
2.3.1.5 Remote Validation Support . . . 26
2.3.1.6 Supervision and Monitoring Features . . . 26
2.3.1.7 PCP Server Control and Automation . . . 27
2.3.2 Non-Functional Requirements . . . 27
2.3.2.1 Usability and Operational Efficiency . . . 27
2.3.2.2 Reliability . . . 27
2.3.2.3 Maintainability . . . 28
2.3.2.4 Scalability and Extensibility . . . 28
2.3.2.5 Performance . . . 28
2.4 Proposed Solution Overview . . . 28
Conclusion . . . 29

3 System Design and Architecture **30**
Introduction . . . 30
3.1 Overall System Architecture . . . 30
3.2 Architectural and Technological Choices . . . 32
3.2.1 Central Application Technologies . . . 32
3.2.2 Communication Technologies . . . 32
3.2.3 Network Validation Technologies . . . 33
3.2.3.1 Python Sockets for PCP Communication . . . 33
3.2.3.2 MiniUPnPc for UPnP Validation . . . 33
3.2.3.3 Telnet-Based Gateway Access . . . 33

3.2.3.4 Remote Server Administration . . . 33
3.2.4 Persistence Technologies . . . 33
3.2.5 Authentication, Authorization and Traceability . . . 34
3.3 Software Architecture . . . 34
3.3.1 General Software Organization . . . 35
3.3.2 Central Application Architecture . . . 35
3.3.3 Communication Management . . . 37
3.3.4 Persistence Services . . . 38
3.3.5 LAN Agent Architecture . . . 39
3.3.5.1 Agent Communication Interface . . . 40
3.3.5.2 PCP Execution Service . . . 40
3.3.5.3 UPnP-to-PCP Validation Service . . . 40
3.3.5.4 Gateway Inspection and Network Diagnostics . . . 41
3.3.5.5 Packet Capture and Protocol Correlation . . . 41
3.3.5.6 Debug Collection Service . . . 41
3.3.6 Component Interaction Workflows . . . 41
3.3.6.1 Direct PCP Validation Workflow . . . 42
3.3.6.2 UPnP-to-PCP Interworking Workflow . . . 43
3.3.6.3 Automated Regression Workflow . . . 45
3.4 Database Design . . . 45
3.4.1 Database Organization . . . 45
3.4.2 Entity Relationship Model . . . 46
3.4.3 Data Relationships and Integrity . . . 46
3.5 System Modeling . . . 47
3.5.1 Global Use Case Diagram . . . 47
3.5.2 Class Diagram . . . 47
3.6 Deployment Architecture . . . 48
3.6.1 Deployment Strategy . . . 48
3.6.2 Final Deployment Architecture . . . 49
3.6.3 Deployment Components . . . 49
3.6.4 Deployment and Maintenance . . . 51
Conclusion . . . 52

4 Implementation and Validation of the NetSupervision Platform 53
Introduction . . . 53
4.1 Experimental and Validation Environment . . . 53
4.1.1 Overall Validation Architecture . . . 54
4.1.2 Home Gateways Under Test . . . 54
4.1.3 PCP Server . . . 55
4.1.4 Validation Network Topology . . . 55
4.1.5 Development Environment . . . 56
4.1.5.1 Hardware Environment . . . 57
4.1.5.2 Software Environment . . . 57
4.2 Platform Implementation . . . 57
4.2.1 Authentication Module . . . 58
4.2.2 Dashboard Module . . . 58
4.2.3 LAN Agent . . . 58
4.2.4 PCP Server Management Module . . . 60

4.2.5 PCP Validation Module . . . 61
4.2.6 Packet Capture Module . . . 61
4.2.7 UPnP Validation Module . . . 62
4.2.8 Mapping Management Module . . . 63
4.2.9 Home Gateway Inspection Module . . . 64
4.2.10 Regression Campaign Module . . . 65
4.2.11 AI-Assisted Regression Analysis . . . 66
4.2.12 Debug Analysis Module . . . 68
4.2.12.1 Automatic Debug Collection . . . 68
4.2.12.2 Debug File Comparison . . . 69
4.2.13 Monitoring and Traceability . . . 70
4.3 Platform Validation . . . 70
4.3.1 Representative Validation Scenarios and Results . . . 70
4.3.1.1 Scenario 1: Local UPnP Mapping Conflict . . . 71
4.3.1.2 Scenario 2: PCP Resource Exhaustion Propagation . . . 71
4.3.1.3 Scenario 3: Distributed External-Port Conflict . . . 72
4.3.1.4 Scenario 4: Debug Comparison Between MiB4 and LB77 72
4.4 Performance Evaluation . . . 75
4.4.1 Validation Process Improvement . . . 75
4.4.2 Reduction of Configuration and Validation Errors . . . 75
4.4.3 Time Saving Evaluation . . . 76
4.4.4 Regression Execution Performance . . . 77
Conclusion . . . 78

General Conclusion **79**

A Home Gateway Technical Details **80**
A.1 Sagemcom F@st 5670 (MiB4) . . . 80
A.2 LB77 (Sagemcom F@st 5698) . . . 81

B Modified PCP Server Details **83**
B.1 Implemented Extensions . . . 83
B.2 PCP Result Codes Used During Validation . . . 84

C Additional NetSupervision Interfaces and Execution Logs **85**
C.1 LAN Agent Deployment Interfaces . . . 85
C.2 UPnP Execution Logs . . . 86
C.3 UPnP Mapping Management Interface . . . 87
C.4 Regression Execution Logs . . . 87
C.5 Debug Collection Interfaces . . . 88
C.6 Monitoring and Traceability Interfaces . . . 88
C.6.1 Activity Logs . . . 89
C.6.2 Audit and User Traceability . . . 89
C.6.3 LAN Agent Logs . . . 90

Bibliography **91**

List of Figures

1.1 Overview of Telnet's main engineering departments [1]. 4
1.2 Project Schedule and Development Phases 6
1.3 DS-Lite network architecture [2]. 9
1.4 PCP client–server communication architecture [3] 11
1.5 UPnP–PCP Interworking Function message flow [4] 12

2.1 PCP server listening on UDP port 5351 15
2.2 Execution of a successful PCP MAP request using the PCP client. . . . 16
2.3 PCP MAP request and response captured with Wireshark. 16
2.4 IPv4 UPnP–PCP validation workflow. 17
2.5 UPnP AddPortMapping execution 18
2.6 Firewall.PCP mapping information 20
2.7 Gateway debug archive generation 21
2.8 Manual collection of debugging information from the Home Gateway. . 22
2.9 UPnP management page of the Home Gateway WebUI. 23

3.1 Overall architecture of the proposed supervision platform 31
3.2 Software architecture of the proposed validation platform 36
3.3 Internal software architecture of the LAN Agent 39
3.4 Sequence of a direct PCP validation request 42
3.5 Sequence of a UPnP-to-PCP interworking validation request 44
3.6 Entity relationship diagram of the NetSupervision database 46
3.7 Global use case diagram of the NetSupervision platform 48
3.8 Software Design Class Diagram of the NetSupervision Platform 50
3.9 Deployment architecture of the NetSupervision platform. 51

4.1 Overall validation architecture of the NetSupervision platform. 55
4.2 Network topology of the DS-Lite validation testbed. 56
4.3 Main dashboard of the NetSupervision platform. 59
4.4 LAN Agent management and selection interface. 60
4.5 PCP Server Management interface 61
4.6 Successful PCP MAP request 62
4.7 Integrated PCP/UPnP packet capture interface. 62
4.8 Successful UPnP port mapping 63
4.9 PCP Mapping Management interface. 64
4.10 Home Gateway Firewall.PCP inspection 65
4.11 Configuration of an automated PCP and UPnP regression campaign. . 66
4.12 Real-time monitoring of an automated regression campaign. 67
4.13 AI-assisted root-cause analysis of a regression result. 68

4.14 Automatic collection of Home Gateway debug archives. 69
4.15 Local UPnP mapping conflict 71
4.16 PCP NO_RESOURCES propagation 72
4.17 Distributed external-port conflict 73
4.18 MiB4 and LB77 debug comparison 74
4.19 AI-assisted debug comparison 74
4.20 Execution-time comparison 78

A.1 Sagemcom F@st 5670 Home Gateway used during validation. 81
A.2 LB77 Home Gateway used during validation. 82

C.1 User registration interface. 85
C.2 LAN Agent installation interface. 86
C.3 LAN Agent startup and registration with the platform. 86
C.4 Execution logs generated during UPnP validation. 87
C.5 UPnP IGD–PCP mapping management interface. 87
C.6 Detailed execution logs generated by the regression module. 88
C.7 Debug extraction and file management interfaces. 88
C.8 Centralized activity log for PCP and UPnP validation events. 89
C.9 Audit log recording administrative actions performed on the platform. . 89
C.10 Monitoring of distributed LAN Agent activity. 90

List of Tables

1.1 Comparison of IPv6 transition mechanisms 8
1.2 Supervision approaches used in the project 13

2.1 Main parameters used during an UPnP mapping request 18
2.2 Main tools used during PCP validation 21
2.3 Main information included in a validation report 22

4.1 Development workstation specifications 57
4.2 Software environment used during the internship 57
4.3 Comparison between the manual validation workflow and the proposed platform. 76
4.4 Execution-time comparison between manual validation and NetSupervision. 77

A.1 Technical specifications of the Sagemcom F@st 5670 80
A.2 Technical specifications of the LB77 (Sagemcom F@st 5698) 81

B.1 Main PCP result codes used during validation 84

List of Acronyms

- **AFTR:** Address Family Transition Router.
- **AI:** Artificial Intelligence.
- **API:** Application Programming Interface.
- **B4:** Basic Bridging BroadBand element.
- **CGNAT:** Carrier-Grade Network Address Translation.
- **CLI:** Command-Line Interface.
- **CPE:** Customer Premises Equipment.
- **CSV:** Comma-Separated Values.
- **DS-Lite:** Dual-Stack Lite.
- **GPON:** Gigabit Passive Optical Network.
- **HGW:** Home Gateway.
- **HTTP:** Hypertext Transfer Protocol.
- **IGD:** Internet Gateway Device.
- **ISP:** Internet Service Provider.
- **IWF:** Interworking Function.
- **IPv4:** Internet Protocol Version 4.
- **IPv6:** Internet Protocol Version 6.
- **JSON:** JavaScript Object Notation.
- **LAN:** Local Area Network.
- **LLM:** Large Language Model.
- **NAT:** Network Address Translation.
- **NAT44:** IPv4-to-IPv4 Network Address Translation.
- **OS:** Operating System.
- **PCAP:** Packet Capture.

- **PCP:** Port Control Protocol.
- **PDF:** Portable Document Format.
- **PID:** Process Identifier.
- **QA:** Quality Assurance.
- **RFC:** Request for Comments.
- **SSH:** Secure Shell.
- **TCP:** Transmission Control Protocol.
- **TFTP:** Trivial File Transfer Protocol.
- **UDP:** User Datagram Protocol.
- **UI:** User Interface.
- **UML:** Unified Modeling Language.
- **UPnP:** Universal Plug and Play.
- **VDSL2:** Very-high-bit-rate Digital Subscriber Line 2.
- **WAN:** Wide Area Network.

General Introduction

The proliferation of Internet services and connected devices has significantly increased the complexity of modern residential networks. Once mainly used to provide Internet access, the Home Gateway (HGW) has evolved into a multifunctional networking device responsible for routing, firewall management, address translation, and protocol interworking. As broadband networks progressively transition from IPv4 to IPv6 while maintaining compatibility through mechanisms such as Dual-Stack Lite (DS-Lite), ensuring the reliability of these gateway functions has become an important requirement for Internet Service Providers (ISPs) and equipment manufacturers.

A major challenge in this transition is maintaining inbound connectivity. The limited availability of IPv4 addresses has encouraged the deployment of Carrier-Grade NAT (CGNAT), which, together with traditional Network Address Translation (NAT), restricts direct end-to-end communication. To address this limitation, the Port Control Protocol (PCP) was standardized to allow applications and gateways to dynamically create and manage port mappings. PCP therefore plays an important role in IPv6 transition environments where controlled inbound connectivity is required.

As HGW firmware continues to evolve with new networking features and protocol improvements, validation before deployment becomes essential. Quality Assurance (QA) engineers are responsible for validating protocol implementations, executing functional and regression tests, analyzing network traffic, inspecting gateway data models, and reproducing reported issues. Their objective is to ensure that new firmware versions behave as expected without affecting the stability of existing functionalities.

This internship was carried out within the Telecommunications Department of Telnet and focused on PCP and UPnP validation for HGW firmware in a laboratory environment. During validation, QA engineers relied on several independent tools to execute protocol requests, inspect the gateway, capture network traffic, and analyze debugging information. Although each tool provided useful information, validating a single scenario required manually correlating data from several sources. This fragmented workflow increased troubleshooting complexity and made regression testing repetitive and time-consuming, particularly in DS-Lite environments where interactions between UPnP, PCP, and the HGW had to be analyzed together.

To address these limitations, the objective of this internship was to design and develop a centralized supervision platform for PCP and UPnP validation in IPv6 and DS-Lite environments. The proposed solution centralizes protocol request execution, gateway inspection, packet capture, logging, debugging, and automated regression testing within a single web-based interface. By bringing these operations together, the platform reduces manual effort, improves protocol visibility, and simplifies the analysis

of HGW behavior.

This report is organized into four chapters.

Chapter 1 introduces the internship context by presenting the host company, the project objectives, and the technical background required to understand Home Gateway validation, IPv6 transition mechanisms, and the PCP protocol.

Chapter 2 analyzes the existing PCP and UPnP validation workflow, identifies its limitations, and derives the functional and non-functional requirements that guided the development of the proposed platform.

Chapter 3 presents the system design and architecture of the proposed solution. It describes the overall software architecture, the communication between the different components, the selected technologies, and the main design choices adopted during the development of the platform.

Chapter 4 presents the implementation and validation of the developed platform. It describes the main functionalities, including PCP and UPnP validation, Home Gateway inspection, mapping management, distributed execution through LAN Agents, automated regression testing, and debug analysis. The chapter also evaluates the proposed solution through representative validation scenarios and discusses the obtained results.

The report concludes with a summary of the main contributions of the project and outlines several perspectives for future improvements.

Chapter 1

Project Context and Technical Foundations

Introduction

In this chapter, we present the context of the internship and the project carried out at Telnet. We first introduce the host company, then present the project, the problem addressed, the project objectives, and the development methodology adopted throughout the internship. Finally, we introduce the technical foundations required to understand the proposed solution and the remainder of this report.

1.1 Host Company Presentation

This section presents Telnet, the company that hosted this internship, before introducing the department in which the project was carried out.

1.1.1 Overview of Telnet

Telnet is a Tunisian engineering and consulting company specializing in innovation and high technologies. Founded in 1994, the company started as a small engineering firm with an initial team of seven engineers and progressively expanded its activities to serve both national and international markets [5]. Over the years, Telnet has developed significant expertise in several domains, including telecommunications, multimedia, embedded systems, electronics, space technologies, transport, defense and avionics, security, and industrial solutions. The company supports its clients throughout the different phases of the product life cycle, from design and development to integration, validation, and maintenance [5]. Today, Telnet is recognized as one of the leading engineering companies in Tunisia, relying on a multidisciplinary approach and strong technical expertise to deliver reliable and scalable solutions adapted to modern technological challenges.

To support its diverse activities, Telnet is organized into several specialized departments, each dedicated to a specific technological domain, such as telecommunications, embedded systems, multimedia, electronics, and space technologies. Figure 1.1 pro-

vides an overview of the company's main departments. The internship presented in this report was carried out within the Telecommunications Department, which is described in the following subsection.

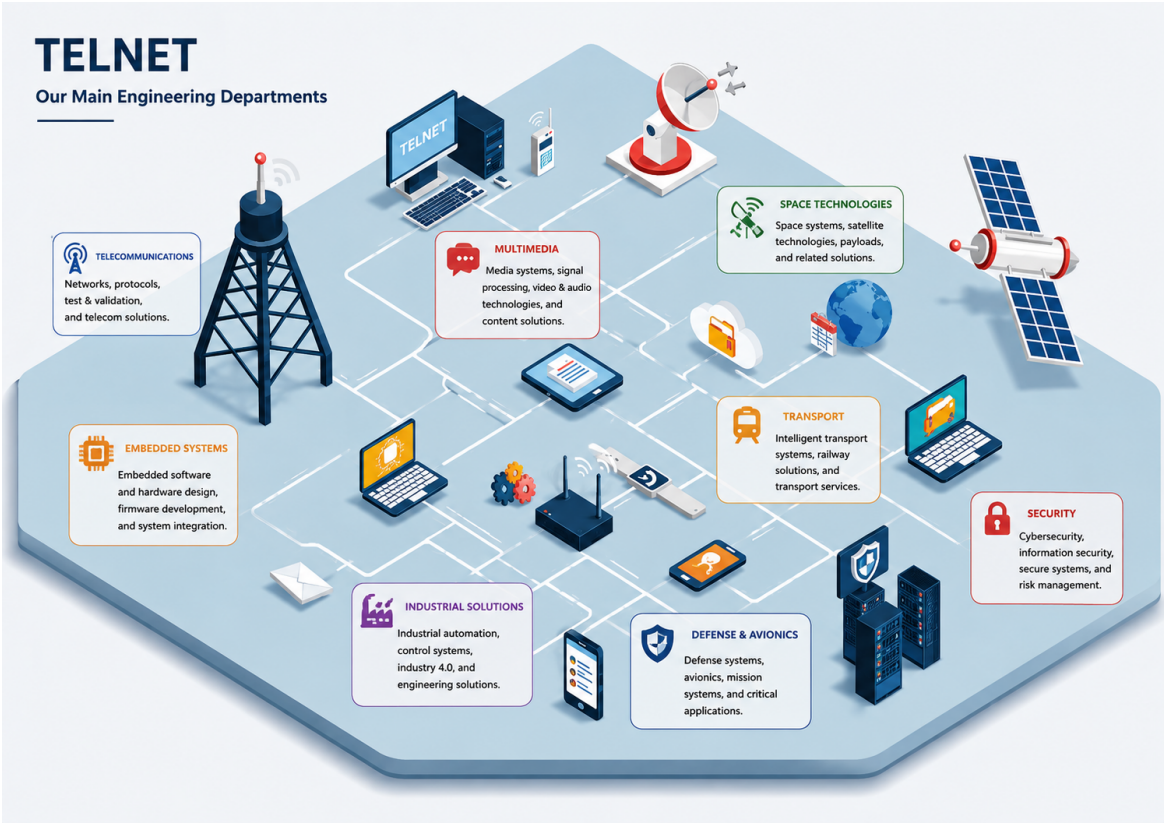


Figure 1.1: Overview of Telnet's main engineering departments [1].

1.1.2 Telecommunications Department

The internship was carried out within the telecommunications department, which is dedicated to network engineering and the validation of communication systems. This department is mainly concerned with testing and verifying the behavior of network devices, particularly HGW, which play a central role in user connectivity. The department organizes its activities around both manual and automated testing approaches. Manual testing allows direct observation and functional verification, while automated testing is used to improve efficiency, repeatability, and test coverage.

In this context, the validation process aims to ensure that network devices operate correctly under different conditions and comply with expected specifications. This environment highlights the need for advanced tools to analyze and supervise network mechanisms, which directly motivated the development of the solution proposed in this project.

1.2 Project Presentation

This section introduces the project by presenting its general context, the problem it addresses, and the objectives pursued during the internship.

1.2.1 General Context

With the rapid growth of the Internet and connected devices, network infrastructures have become increasingly complex. Home networks rely on Home Gateways (HGWs), which provide connectivity between local devices and external networks.

At the same time, the exhaustion of public IPv4 addresses has led operators to adopt address-sharing and IPv6 transition mechanisms such as CGNAT and DS-Lite. These architectures introduce additional challenges for inbound connectivity and port management, making protocols such as PCP particularly important.

IPv6 was introduced as a long-term solution by providing a much larger address space. Since many existing services still depend on IPv4, transition mechanisms such as DS-Lite allow IPv4 traffic to coexist with an IPv6-based operator network by transporting IPv4 traffic over IPv6 while performing address translation at the provider level.

In these environments, inbound connectivity remains difficult because NAT and CGNAT block unsolicited incoming connections by default. PCP addresses this limitation by allowing devices to dynamically create and manage port mappings.

Despite these mechanisms, validating network behavior on Home Gateways remains complex and often requires manual command-line operations, log analysis, and repetitive test execution. This context motivates the need for a more centralized and automated validation approach.

1.2.2 Problem Statement

Within the telecommunications department, the validation of network protocols such as PCP and UPnP on home gateways is performed using predefined test plans covering scenarios such as port mapping creation, lifetime management, error handling, and protocol interoperability.

However, this validation process relies heavily on manual operations and the use of multiple independent tools. Test engineers must generate protocol requests, capture and analyze network traffic, inspect gateway data models through command-line interfaces, and examine system logs to determine whether the observed behavior complies with the expected specifications. As a result, the validation workflow is fragmented, time-consuming, and highly dependent on the engineer's expertise.

Furthermore, the information required for troubleshooting is distributed across different sources, making it difficult to correlate protocol exchanges, gateway states, and debug logs during the analysis of failures or firmware regressions.

These challenges become even more significant in DS-Lite environments, where the interaction between the home gateway, the PCP server, and the operator network increases the complexity of validating port mappings and inbound connectivity.

Consequently, there is a need for a centralized validation platform capable of simplifying test execution, consolidating relevant information, and providing better visibility into gateway behavior throughout the validation process.

1.2.3 Project Objectives

The main objective of this project is to develop a software solution that facilitates the testing and supervision of network mechanisms in modern telecommunication environments, particularly those involving PCP and DS-Lite.

To achieve this objective, the proposed solution aims to simplify the execution of test scenarios and improve the analysis of network behavior on home gateways. It also focuses on providing better visibility of the system by centralizing relevant information and reducing the complexity of manual operations.

In addition, the solution is designed to support engineers in validating different use cases, improving efficiency, and ensuring more reliable results during the testing process.

1.3 Project Methodology

This section describes the methodology adopted throughout the internship, including the general work approach and the development lifecycle followed to design and implement the proposed solution.

1.3.1 Work Approach

The project was conducted using a flexible and iterative approach, allowing the solution to evolve progressively according to technical constraints, validation results, and feedback obtained throughout the internship. Tasks were organized using a workflow inspired by Kanban, with three main states: *To Do*, *In Progress*, and *Done*.

The overall project schedule and its main development phases are illustrated in Figure 1.2.

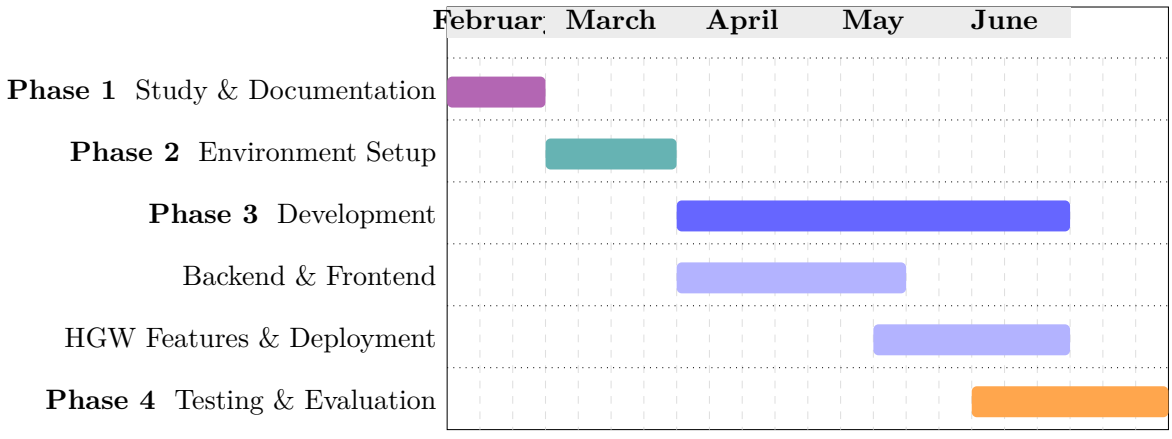


Figure 1.2: Project Schedule and Development Phases

1.3.2 Development Phases

The internship was organized into four main phases. The first phase focused on studying the technical foundations of the project, including IPv6 transition mechanisms, DS-Lite, PCP, and UPnP–PCP interworking.

The second phase consisted of preparing the validation environment by configuring the Home Gateway, validating IPv4/IPv6 and DS-Lite connectivity, and installing the PCP validation server.

The third phase covered the incremental development of the supervision platform. It included the PCP client, Flask web interface, protocol logging and packet capture, PCP server extensions for controlled error scenarios, UPnP validation, Home Gateway inspection, distributed LAN Agent execution, and automated regression features.

The fourth phase focused on functional validation and performance evaluation. Representative PCP and UPnP scenarios were executed and the observed results were compared with the expected protocol behavior using packet captures, gateway data-model information, server logs, and regression results.

1.4 Technical Foundations

This section presents the essential concepts required to understand the context and the technical choices of this project. It focuses on IPv6 transition mechanisms, the PCP, and the deployment environments in which these technologies are used.

The objective is not to provide an exhaustive description of the protocols, but rather to highlight the key elements necessary to understand the behavior of the system and the challenges addressed by the developed tool.

1.4.1 IPv4 Exhaustion, NAT and CGNAT

Initially, Internet communication was mainly based on Internet Protocol version 4 (IPv4) addressing. However, due to its limited 32-bit address space, IPv4 has faced significant exhaustion as the number of Internet users and connected devices has continued to grow [6, 2]. Assigning a unique public IPv4 address to every subscriber has therefore become increasingly difficult.

To mitigate this limitation, NAT has been widely adopted, allowing multiple devices within a local network to share a single public IPv4 address. However, this approach still generally requires a public IPv4 address for each subscriber connection, which becomes difficult to sustain at large scale [7].

To further reduce public IPv4 address consumption, operators deploy CGNAT, where address translation is performed within the provider network. In this architecture, multiple subscribers can share the same public IPv4 address [7]. However, users no longer have direct control over the upstream NAT function. Consequently, traditional port forwarding becomes difficult or impossible, and inbound connectivity is significantly restricted.

These limitations create the need for mechanisms capable of dynamically managing port mappings on upstream network devices, which is one of the main motivations for the use of PCP in modern operator networks [3].

1.4.2 IPv6 and Transition Mechanisms

IPv6 was designed as a long-term solution, offering a 128-bit address space and enabling direct addressing of devices without relying on large-scale address sharing mechanisms.

However, the transition from IPv4 to IPv6 is still ongoing, and both protocols must coexist. This has led to the development of several transition mechanisms.

1.4.2.1 Main Transition Mechanisms

Several mechanisms have been introduced to support the coexistence of IPv4 and IPv6. Table 1.1 summarizes the transition mechanisms most relevant to this project, together with their main advantages and limitations.

Table 1.1: Comparison of IPv6 transition mechanisms

Mechanism	**Principle**	**Advantages**	**Limitations**
Dual Stack [8]	Devices operate simultaneously with IPv4 and IPv6.	Simple deployment and native support for both protocols.	Requires IPv4 resources and dual protocol management.
DS-Lite [2]	IPv4 traffic is encapsulated and transported through an IPv6 network.	Reduces IPv4 address consumption and facilitates IPv6 deployment.	Relies on centralized NAT and reduces visibility on port mappings.
NAT64/DNS64 [9, 10]	Enables communication between IPv6-only clients and IPv4 services through protocol translation.	Supports IPv6-only deployments.	Some applications may experience compatibility limitations.

As shown in Table 1.1, each transition mechanism offers a different approach for ensuring interoperability between IPv4 and IPv6 networks. Among these mechanisms, DS-Lite is particularly relevant because it is the deployment architecture used in the validation platform where this internship was carried out. Since PCP operates in this environment to manage port mappings at the AFTR, understanding DS-Lite is essential before introducing the protocol itself.

1.4.3 DS-Lite Architecture and Deployment Context

1.4.3.1 Overview of DS-Lite

DS-Lite, defined in RFC 6333, is a transition mechanism that enables ISPs to deliver IPv4 connectivity over an IPv6-only access network [2].

In this architecture, end users are no longer assigned public IPv4 addresses. Instead, IPv4 traffic is encapsulated within IPv6 packets and transported through the IPv6 network to a centralized device located in the operator's infrastructure.

This approach allows ISPs to reduce IPv4 address consumption while continuing to support legacy IPv4 services.

The DS-Lite architecture is illustrated in Figure 1.3.

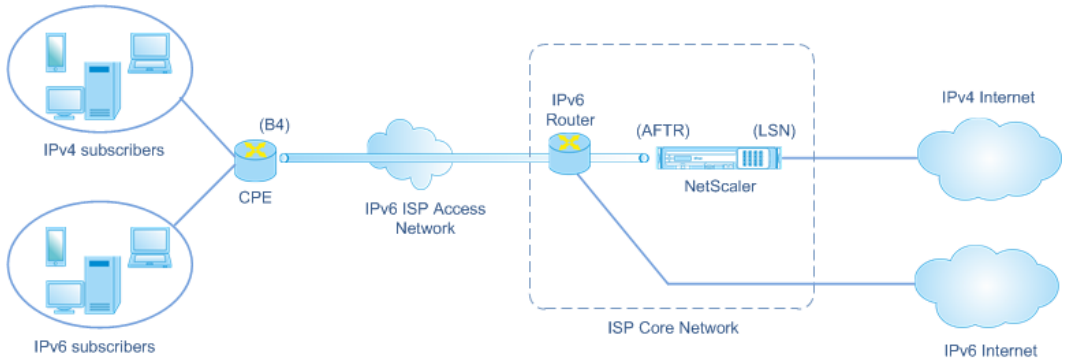


Figure 1.3: DS-Lite network architecture [2].

As shown in Figure 1.3, IPv4 traffic is encapsulated within IPv6 packets and transmitted to the AFTR, where NAT is performed. This centralized architecture reduces IPv4 consumption but limits user control over port mappings.

1.4.3.2 Main Components of the DS-Lite Architecture

A DS-Lite architecture is composed of two main elements:

- **B4 Element (Basic Bridging BroadBand):** located in the home gateway (CPE). It encapsulates IPv4 packets into IPv6 and forwards them to the operator network. In PCP-enabled deployments, the B4 also acts as a PCP client.
- **AFTR (Address Family Transition Router):** located in the ISP network. It decapsulates incoming packets and performs NAT44 using a shared pool of public IPv4 addresses. The AFTR also acts as a PCP server.

Communication between the B4 and the AFTR is carried over an IPv6-only network.

1.4.3.3 Operational Principle of DS-Lite

When a device inside the local network sends an IPv4 packet, the home gateway encapsulates this packet inside an IPv6 packet using the B4 function. The encapsulated packet is then transmitted through the IPv6 network to the AFTR.

At the AFTR level, the packet is decapsulated and translated using NAT44 before being forwarded to the IPv4 Internet. The reverse process is applied for incoming traffic.

This centralized NAT architecture allows multiple users to share a limited number of public IPv4 addresses, but it also introduces limitations in terms of visibility and control over port mappings.

1.4.3.4 Integration of PCP in DS-Lite

In DS-Lite environments, the NAT function is located at the AFTR, outside the user's local network. The B4 element integrated into the Home Gateway therefore acts as a PCP client, while the AFTR provides the PCP server function.

Because users cannot directly configure the centralized NAT, the Home Gateway uses PCP to request the creation, renewal, or deletion of port mappings at the AFTR. This mechanism restores controlled inbound connectivity while keeping address translation within the operator network.

However, the success of a mapping remains dependent on the AFTR configuration, available resources, and network policies. These characteristics make DS-Lite the main deployment environment considered in this project for validating and supervising PCP behavior.

1.4.4 Port Control Protocol (PCP)

In modern networks, the widespread use of CGNAT limits the ability of end users to control inbound connections. Applications such as online gaming, VoIP, and peer-to-peer services require the creation of port mappings to allow external hosts to reach internal devices.

Before PCP, protocols such as UPnP IGD and NAT-PMP were used to manage these mappings. However, these solutions present several limitations, including lack of IPv6 support, security concerns, and limited control over upstream NAT devices.

To address these issues, the PCP, defined in RFC 6887 [3], was introduced. PCP provides a standardized and efficient way for clients to request port mappings directly from NAT devices, including those located in the operator's network.

1.4.4.1 PCP Architecture and Operation

PCP follows a client–server architecture. The PCP client is generally located on an end device or on the HGW, while the PCP server is deployed on the network device responsible for port management, such as the AFTR in DS-Lite environments. The client sends requests to create, renew, or delete port mappings, and the server processes these requests according to its configuration and available resources.

Communication between the PCP client and server is performed over UDP port 5351. For each request received, the server returns a response indicating whether the operation was successfully completed or rejected. This request–response model allows clients to dynamically manage inbound connectivity in environments where traditional manual port forwarding is not possible.

Among the different PCP operations, the MAP request is the most commonly used. It enables a client to request the creation of a mapping between an internal address and port and an external address and port. Each mapping is associated with a lifetime value that determines how long the mapping remains active. To keep a mapping valid, the client must periodically renew it before its expiration.

Several PCP concepts are particularly important for this project. A port mapping represents the association between internal and external communication endpoints. The lifetime parameter defines the duration of this association. PCP responses also contain result codes that indicate whether the operation was successful or rejected due to a specific condition such as resource limitations or invalid request parameters. Analyzing these responses is essential for understanding protocol behavior and identifying potential anomalies during validation activities.

Understanding PCP behavior is essential for the development of the supervision tool proposed in this project. The tool interacts with the PCP server to send mapping requests, analyze responses and error codes, and monitor mapping behavior over time. This allows the detection of inconsistencies such as resource limitations, mapping conflicts, or unexpected gateway behavior, which are difficult to observe using traditional testing methods.

The logical architecture of PCP is illustrated in Figure 1.4. The PCP client communicates with the PCP server through UDP port 5351 using a request–response model.

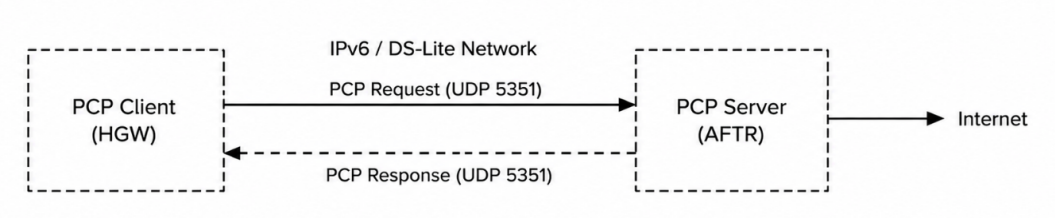


Figure 1.4: PCP client–server communication architecture.

As shown in Figure 1.4, the PCP client sends requests to the PCP server through UDP port 5351. The server processes these requests and returns the corresponding responses. This request–response communication forms the basis of PCP operation in NAT and DS-Lite environments.

1.4.4.2 UPnP–PCP Interworking Function (IWF)

In many home gateway implementations, UPnP IGD is used as a local interface for client applications, while PCP is used to communicate with upstream NAT devices such as the AFTR in DS-Lite environments.

The UPnP–PCP Interworking Function (IWF), defined in RFC 6970 [4], enables the translation of UPnP actions into PCP requests. For example, an *AddPortMapping* request received through UPnP is converted into a PCP MAP request sent to the upstream server.

The corresponding message exchange is illustrated in Figure 1.5.

However, during the experimental phase of this project, it was observed that the home gateway may return a `SUCCESS` response to the UPnP client before receiving the actual response from the PCP server. This behavior differs from the sequence defined in RFC 6970, where the gateway is expected to reflect the actual outcome of the PCP operation.

As a result, real errors such as `NO_RESOURCES` or `CANNOT_PROVIDE_EXTERNAL` may be hidden from the client, leading to inconsistent or misleading results.

This observation directly influenced the design of the supervision tool. To overcome this limitation, an additional verification mechanism was introduced by querying the internal data model of the gateway (e.g., `Firewall.PCP`). This allows retrieving the actual state of the mapping and validating whether the operation was effectively applied at the network level.

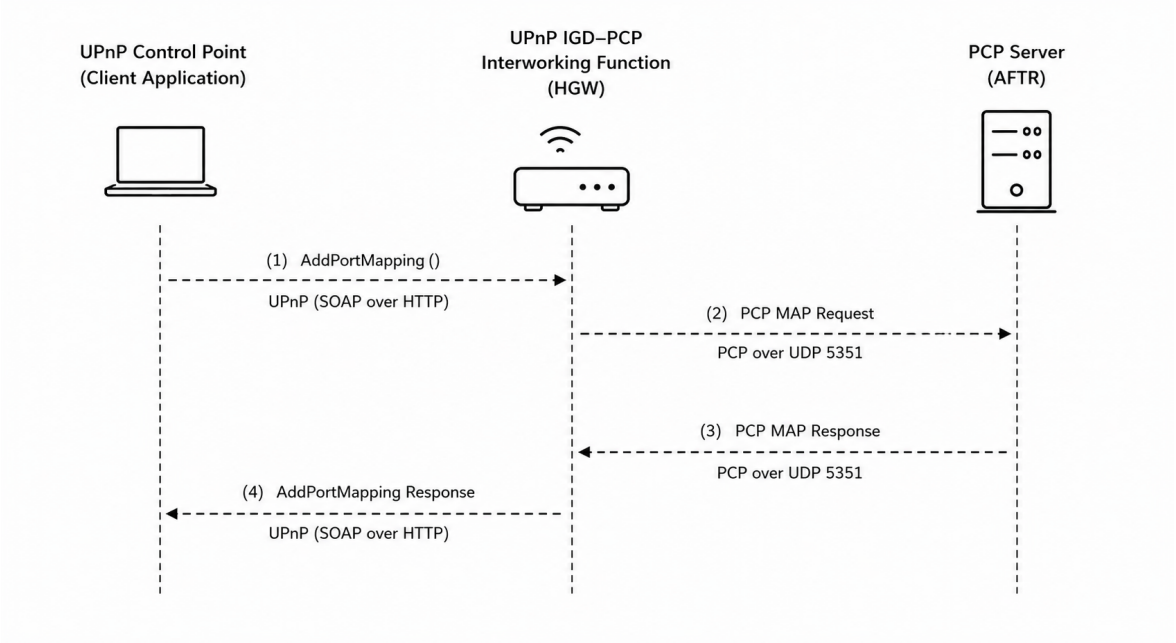


Figure 1.5: UPnP–PCP Interworking Function message flow.

1.4.5 Network Supervision Concepts

1.4.5.1 Overview of Network Supervision

Network supervision refers to the set of techniques used to monitor, analyze, and validate the behavior of network systems in real time. It plays a key role in ensuring reliability, detecting anomalies, and validating the correct operation of network protocols.

In modern ISP environments, supervision is particularly important due to the complexity of architectures such as DS-Lite, where multiple components (home gateway, access network, and operator infrastructure) interact dynamically.

1.4.5.2 Supervision Approaches

In this project, supervision is based on two complementary approaches: active monitoring and passive monitoring. Active monitoring consists of generating test requests in order to evaluate the behavior of the system, while passive monitoring consists of observing existing traffic, logs, and gateway states without generating additional requests.

To better compare these two approaches, Table 1.2 summarizes their role in this project.

Table 1.2: Supervision approaches used in the project

Approach	**Principle**	**Usage in the Project**
Passive monitoring	Observes existing traffic, logs, and gateway states without generating additional requests.	PCP packet capture, log analysis, `Firewall.PCP` inspection, and gateway state verification.
Active monitoring	Generates test requests in order to evaluate protocol behavior under controlled conditions.	PCP MAP requests, UPnP requests, renew/delete operations, and error validation scenarios.

As shown in Table 1.2, the proposed solution combines passive monitoring to observe network behavior with active monitoring to execute PCP and UPnP validation scenarios. This combination provides a comprehensive view of protocol behavior by correlating requests with responses, packet captures, logs, and gateway states.

This combination of active and passive supervision forms the basis of the validation methodology adopted throughout this project.

Conclusion

This chapter presented the general context of the project and introduced the technical foundations required to understand the validation of PCP and UPnP mechanisms in modern network environments. The main concepts related to IPv6 transition mechanisms, DS-Lite, PCP, UPnP–PCP interworking, and network supervision were also discussed. Based on this technical background, the next chapter examines the existing validation approaches, identifies their main limitations, and derives the functional and technical requirements of the proposed solution.

Chapter 2

Existing Validation Approaches and Requirements Analysis

Introduction

This chapter analyzes the existing validation workflow adopted within the telecommunications department, identifies the limitations of the current approach, and derives the functional and non-functional requirements that guided the design of the proposed supervision platform. Finally, it presents a high-level overview of the proposed solution before introducing its architecture in the next chapter.

2.1 Existing Validation Workflow

Before the development of the proposed supervision platform, PCP validation was carried out through a set of manual procedures involving several independent tools and interfaces. The objective of these validation activities was not only to verify the correct exchange of PCP messages but also to ensure that the HGW behaved as expected in different network deployment scenarios.

Depending on the network architecture, two validation workflows were followed. In native IPv6 environments, the HGW operates as an IPv6 router, and PCP is mainly used to create firewall pinholes that allow inbound connectivity. For IPv4 applications running in a DS-Lite environment, the validation process was different. Since the Home Gateway did not implement a native PCP client for IPv4 applications, port mappings were requested through UPnP. The gateway then translated these requests into PCP MAP messages before sending them to the upstream PCP server.

While the two workflows differed in terms of network architecture, both required testers to perform several manual verification steps using separate tools. As a result, validating a single scenario often involved collecting and comparing information from different sources before determining whether the expected protocol behavior had been achieved.

The following subsections describe the validation procedures adopted during the internship for both IPv6 and DS-Lite environments before analyzing their limitations.

2.1.1 IPv6 Validation Workflow

In IPv6 environments, the HGW simply forwards IPv6 packets, while inbound communications are controlled by the firewall. The purpose of this validation was to verify that PCP could successfully request the opening of inbound connectivity through the firewall. Since no NAT is involved in this scenario, the PCP client communicates directly with the upstream PCP server over IPv6, making it possible to validate the protocol behavior independently of the gateway translation mechanisms.

2.1.1.1 Environment Preparation

Before starting a validation scenario, the PCP server had to be accessible from the test environment. Testers first connected to the company server through an SSH session and manually started the PCP server from the Linux terminal. Figure 2.1 shows the server after it was launched and confirms that it was listening on the standard PCP UDP port (5351), making it ready to receive incoming PCP requests.

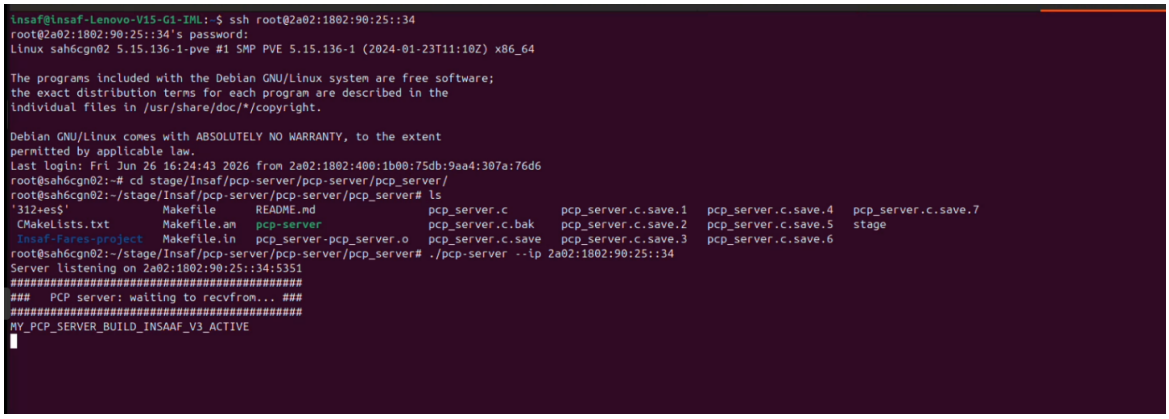


Figure 2.1: PCP server listening on UDP port 5351

Once the PCP server was running, the tester used the PCP client installed on the validation workstation to generate PCP requests toward the configured server. Depending on the validation scenario, different parameters such as the transport protocol, internal port, external port, and requested lifetime could be specified.

2.1.1.2 PCP Request Execution

After preparing the environment, the tester manually generated a PCP MAP request using the PCP client. The request included the IPv6 address of the PCP server, the transport protocol, the internal port, the requested external port, and the desired lifetime. These parameters could be modified depending on the validation scenario.

Figure 2.2 shows the terminal output obtained after executing a PCP MAP request. The exchanged messages, the returned result code, and the created mapping are displayed directly by the PCP client. In this example, the request completed successfully.

2.1.1.3 PCP Exchange Verification

Receiving a successful response from the PCP client was only the first step of the validation process. During each test, the PCP server console was also monitored to

Figure 2.2: Execution of a successful PCP MAP request using the PCP client.

verify that the request had been correctly received and processed. This made it possible to compare the information displayed by the client with the events reported by the server.

The exchanged PCP packets were also captured using Wireshark. The capture was started on the network interface used for the test and filtered on UDP port 5351. Figure 2.3 shows the PCP MAP request together with the corresponding response, which were examined to verify the exchanged protocol fields.

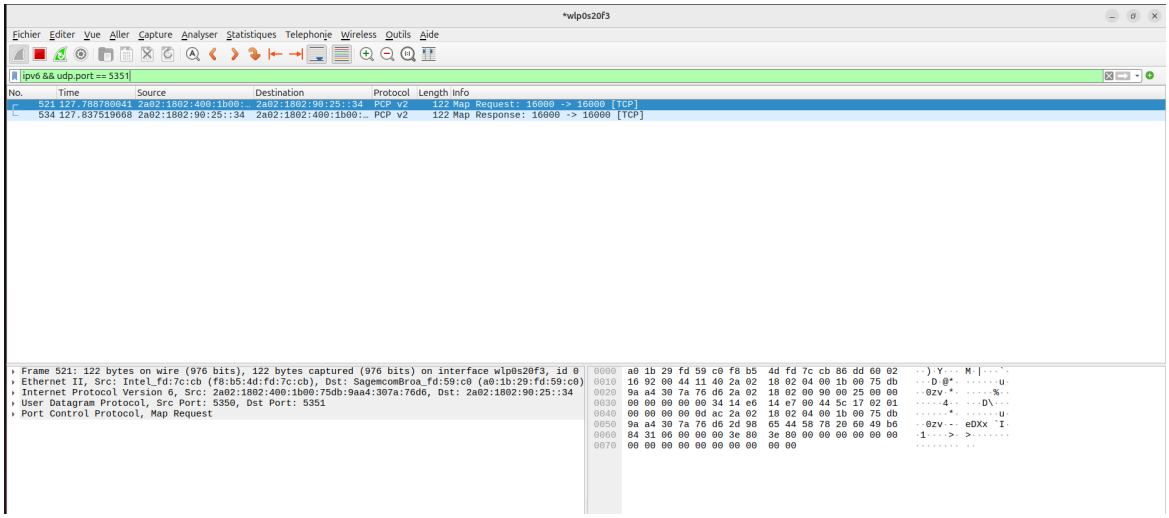


Figure 2.3: PCP MAP request and response captured with Wireshark.

Although the IPv6 validation workflow was relatively straightforward, it still required the simultaneous use of several tools, including SSH, the PCP client, the PCP server console, and Wireshark. As a result, even simple validation scenarios involved multiple manual operations before the correctness of a PCP exchange could be confirmed. The next subsection presents the validation workflow used for IPv4 applications in a DS-Lite environment, where the Home Gateway performs the UPnP–PCP interworking.

2.1.2 IPv4 Validation Workflow in a DS-Lite Environment

In the IPv4 validation scenario, the tested application was located behind the Home Gateway and operated using IPv4. In this environment, inbound connectivity was affected by NAT, while the upstream PCP server was reached through the IPv6 network used by the DS-Lite architecture.

Unlike the IPv6 workflow, the IPv4 application did not send a PCP request directly to the upstream server. Instead, an UPnP client was used to request a port mapping

from the Home Gateway. The gateway then relied on its UPnP–PCP Interworking Function to translate the received UPnP action into a PCP MAP request and forward it to the PCP server.

2.1.2.1 Preparation of the DS-Lite Validation Environment

Before executing the validation scenario, the tester first verified that the PCP server was running on the company server and listening on UDP port 5351. The server was accessed and monitored through an SSH session.

On the local workstation, an UPnP client based on the `miniupnpc` command-line library was required. This client was used to discover the Home Gateway and execute UPnP operations such as mapping creation and deletion.

The Home Gateway also had to be correctly connected to the DS-Lite environment, with the UPnP service enabled and PCP operational on the WAN side.

Figure 2.4 presents the main components involved in this validation scenario.

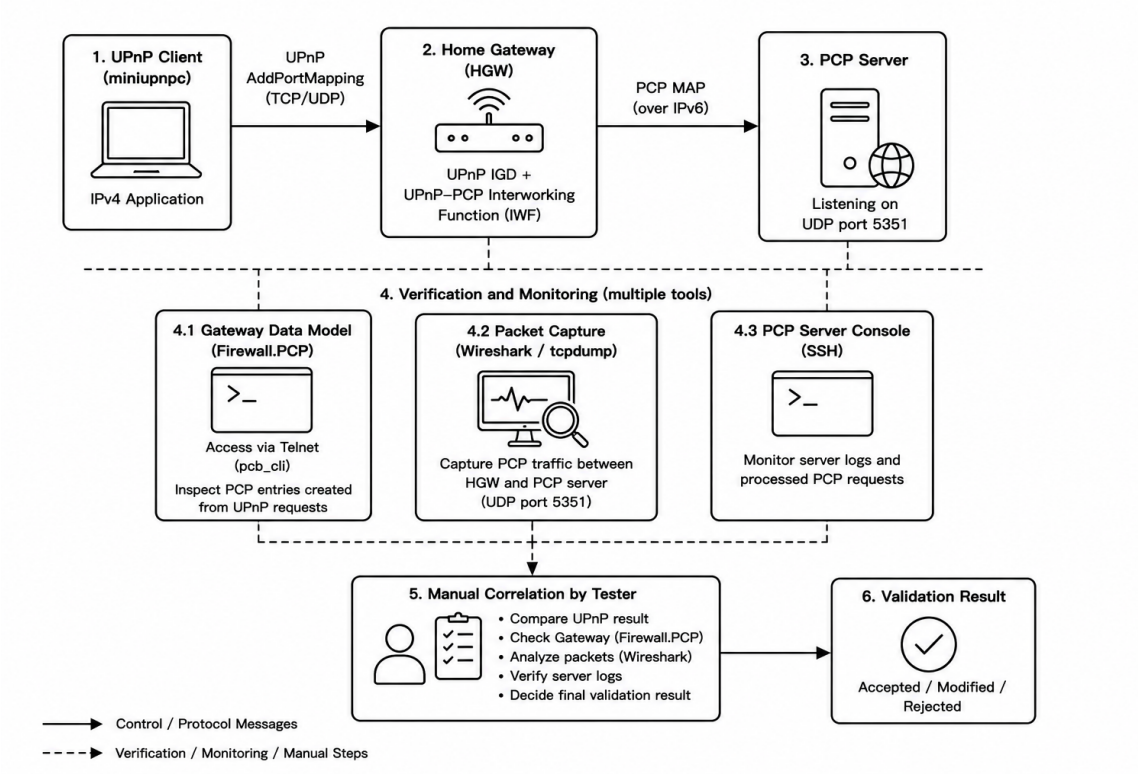


Figure 2.4: IPv4 UPnP–PCP validation workflow.

2.1.2.2 UPnP Request Execution

Once the test environment was ready, the tester generated an UPnP *AddPortMapping* request from the IPv4 workstation using the `miniupnpc` command-line client. The request specified the local IPv4 address of the application, the internal port, the requested external port, and the transport protocol.

Figure 2.5 shows the execution of an *AddPortMapping* request during the validation process. The terminal output displays the discovered Internet Gateway Device (IGD)

together with the created port mapping.

Figure 2.5: Execution of an UPnP AddPortMapping request using the miniupnpc client.

After receiving the request, the Home Gateway processed the UPnP operation through its IGD interface. The gateway then translated the request into a PCP MAP message and forwarded it to the upstream PCP server over IPv6.

The main parameters used during an UPnP mapping request are summarized in Table 2.1.

Table 2.1: Main parameters used during an UPnP mapping request

Parameter	**Description**
Internal IPv4 address	IPv4 address of the application located inside the local network.
Internal port	Local port on which the application receives incoming traffic.
Requested external port	Port requested on the external side of the mapping.
Transport protocol	Protocol associated with the mapping, generally TCP or UDP.
Lease duration	Requested duration of the UPnP mapping.

After executing the command, the UPnP client displayed the result returned by the Home Gateway. Although the request appeared to succeed locally, this response alone did not confirm that the corresponding PCP operation had been completed successfully on the upstream PCP server.

2.1.2.3 PCP Translation Verification

The response returned by the UPnP client did not provide enough information to confirm that the corresponding PCP operation had been completed successfully. To verify the effective PCP state, the tester connected to the Home Gateway through Telnet and accessed the `pcb_cli` interface.

The `Firewall.PCP` data model was then inspected to retrieve the PCP mappings maintained by the gateway.

Each mapping contained several fields, including the PCP status, the internal port, the proposed external port, the assigned external port, the transport protocol, and both the requested and assigned lifetimes.

Since several mappings could be present at the same time, the corresponding entry had to be identified manually. The tester compared the values displayed by the gateway with those used in the UPnP request, mainly:

- the internal port;
- the proposed external port;
- the assigned external port;
- the transport protocol;
- the requested and assigned lifetimes;
- the PCP status or returned error code.

This comparison confirmed whether the generated UPnP request produced the expected PCP state on the Home Gateway and whether the mapping had been accepted, modified, or rejected.

Figure 2.6 shows an example of the information available during this verification.

2.1.2.4 PCP Exchange and Final Verification

When further verification was required, the PCP server console was examined to confirm that the translated PCP request had been received and processed correctly. The information reported by the server was then compared with the state observed on the Home Gateway.

The exchanged PCP packets were also captured using Wireshark or `tcpdump`. These captures allowed the tester to inspect the PCP MAP request generated by the gateway together with the corresponding response returned by the PCP server.

The final validation result was obtained by comparing information collected from several independent sources:

- the result returned by the UPnP client;
- the PCP request and response captured on the network;
- the PCP server console;
- the corresponding entry in `Firewall.PCP`.

Compared with the direct IPv6 workflow, IPv4 validation in a DS-Lite environment required additional verification steps because both the UPnP protocol and the PCP protocol were involved. A single validation scenario required the tester to generate an UPnP request, verify its translation into a PCP MAP request, inspect the gateway data model, analyze the PCP server response, and compare the information collected from different tools.

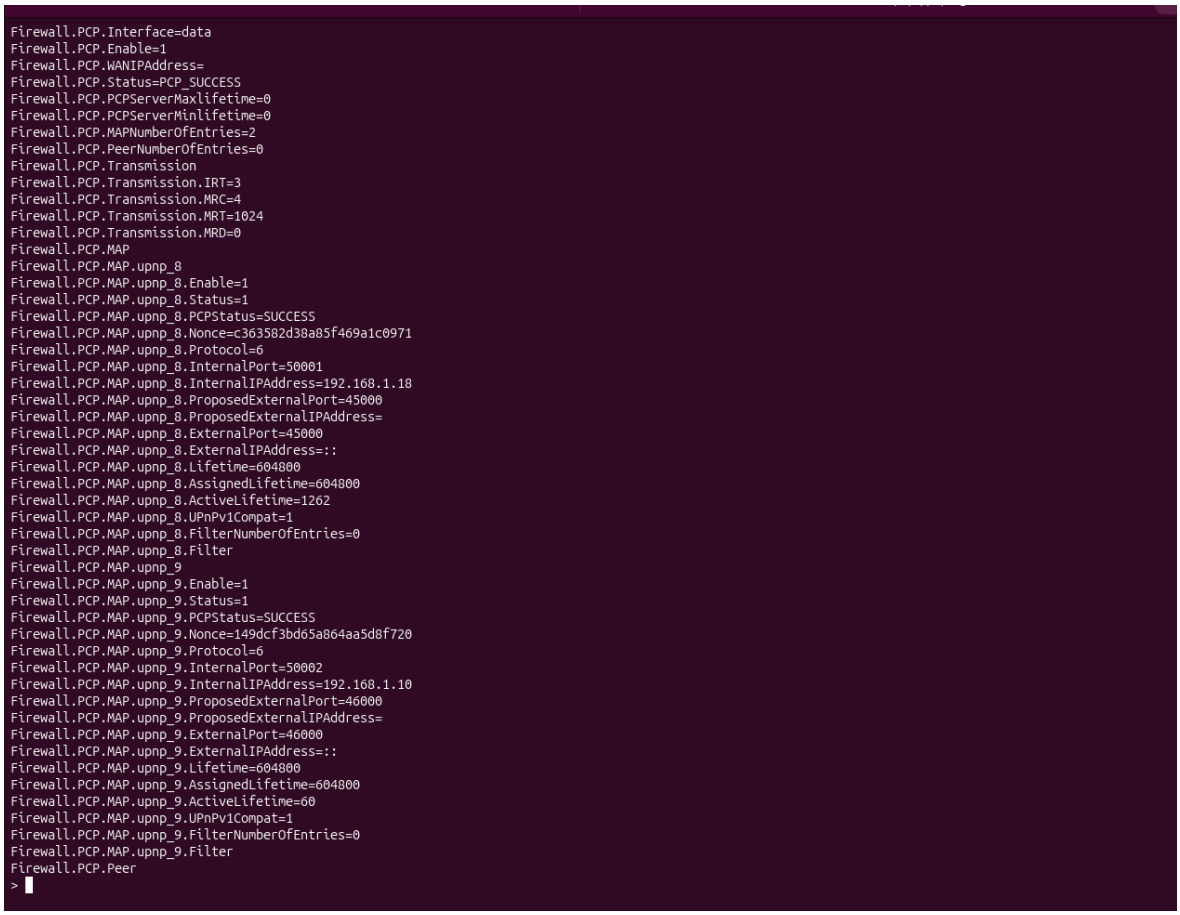


Figure 2.6: PCP mapping information available through the `Firewall.PCP` data model.

Although this workflow made it possible to validate the UPnP–PCP interworking mechanism, it remained largely manual. The tester had to correlate information obtained from the UPnP client, the Home Gateway, the PCP server, and packet captures before determining whether the validation scenario had succeeded. This process became increasingly time-consuming when multiple mappings were active or when the same tests had to be repeated after firmware updates.

These limitations highlighted the need for a centralized validation platform capable of collecting and presenting all the required information through a single interface.

2.1.3 Validation Tools Overview

Table 2.2 summarizes the main tools used during the validation process and their respective roles.

Table 2.2: Main tools used during PCP validation

Tool	**Purpose**	**Main Usage**
PCP Client	Generate PCP MAP requests	Direct PCP validation over IPv6
miniupnpc	Generate UPnP port mapping requests	IPv4 validation through UPnP–PCP IWF
PCP Server	Receive and process PCP requests	Return PCP responses during validation
Wireshark	Capture and decode network packets	Analyze PCP and UPnP exchanges
Telnet (pcb_cli)	Access the HGW data model	Inspect `Firewall.PCP` information
HGW WebUI	Configure UPnP	Enable/disable UPnP and display mappings

2.1.4 Collecting Gateway Debug Information

When the previous verification steps did not provide enough information, the tester collected the debugging files generated by the Home Gateway. This operation required a TFTP server to be available on the validation workstation.

The tester first connected to the Home Gateway through Telnet and executed the `getDebugInformation -a` command to generate a new debugging archive. Figure 2.7 shows an example of this operation during the validation process.

Figure 2.7: Generation of a gateway debugging archive using the getDebugInformation command.

Once the archive had been generated, it was transferred to the validation workstation using TFTP. The downloaded archive was then extracted so that its contents could be inspected and the relevant logs analyzed.

If the workstation normally used for debugging was already occupied by another engineer, a different machine had to be prepared and configured with a TFTP server before starting the transfer. This dependency reduced the flexibility of the debugging process, especially when several engineers were working simultaneously.

The overall debug collection procedure is summarized in Figure 2.8.

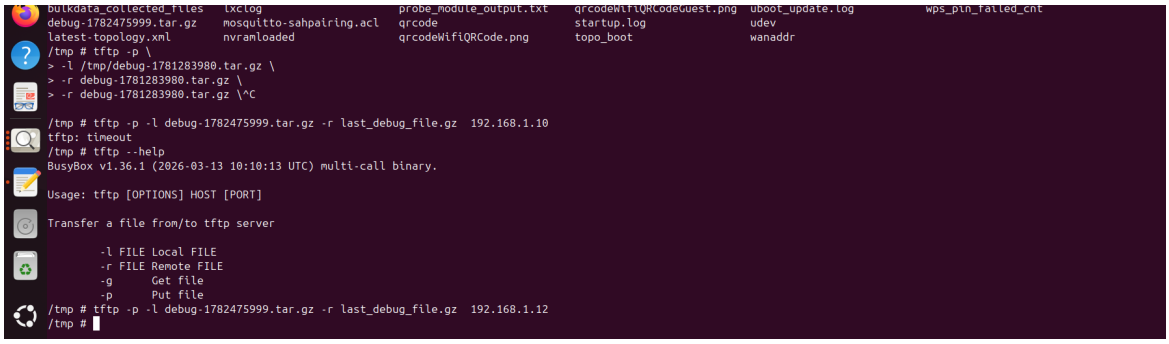


Figure 2.8: Manual collection of debugging information from the Home Gateway.

2.1.5 Issue Reporting

When an unexpected behavior was observed during validation, the tester first reproduced the scenario and collected the technical information required to document the issue. This information was then transmitted to the development team to facilitate the analysis and reproduction of the reported behavior.

The reported issue generally contained the information summarized in Table 2.3.

Table 2.3: Main information included in a validation report

Information	**Description**
Validation scenario	Description of the executed test case and its configuration.
Observed behavior	PCP or UPnP responses together with the behavior observed on the Home Gateway.
Expected behavior	Expected protocol behavior according to the validation scenario.
Gateway information	Relevant information collected from the Home Gateway, including PCP mappings and gateway status.
Packet capture	Wireshark or PCAP traces of the exchanged protocol messages.
Debug information	Gateway debug logs and additional diagnostic information.

Preparing this report required collecting information from several independent tools before organizing the technical evidence into a single document. This manual process increased the time required to report validation results and motivated the integration of centralized reporting capabilities in the proposed platform.

For example, a mapping created with an automatically assigned external port could remain active after a deletion request using a lifetime equal to zero. To report this behavior, the tester had to reproduce the scenario, capture the PCP exchange, inspect the corresponding gateway mapping, and attach the relevant logs to the ticket.

2.2 Limitations of Existing Approaches

Although the existing workflow allowed PCP functionality to be tested and validated, several operational limitations were identified during the internship. These limitations made debugging and validation more difficult, especially in DS-Lite environments involving multiple interacting components.

2.2.1 Lack of PCP Visibility

One of the main limitations identified during the internship was the limited visibility into PCP behavior provided by the Home Gateway WebUI.

Figure 2.9 shows the UPnP page available in the gateway WebUI. The interface allows testers to enable or disable the UPnP service and displays the list of active UPnP mappings. However, no equivalent page is available for PCP. Information such as PCP mappings, PCP result codes, assigned lifetimes, or the status of the UPnP–PCP interworking mechanism cannot be accessed directly from the WebUI.

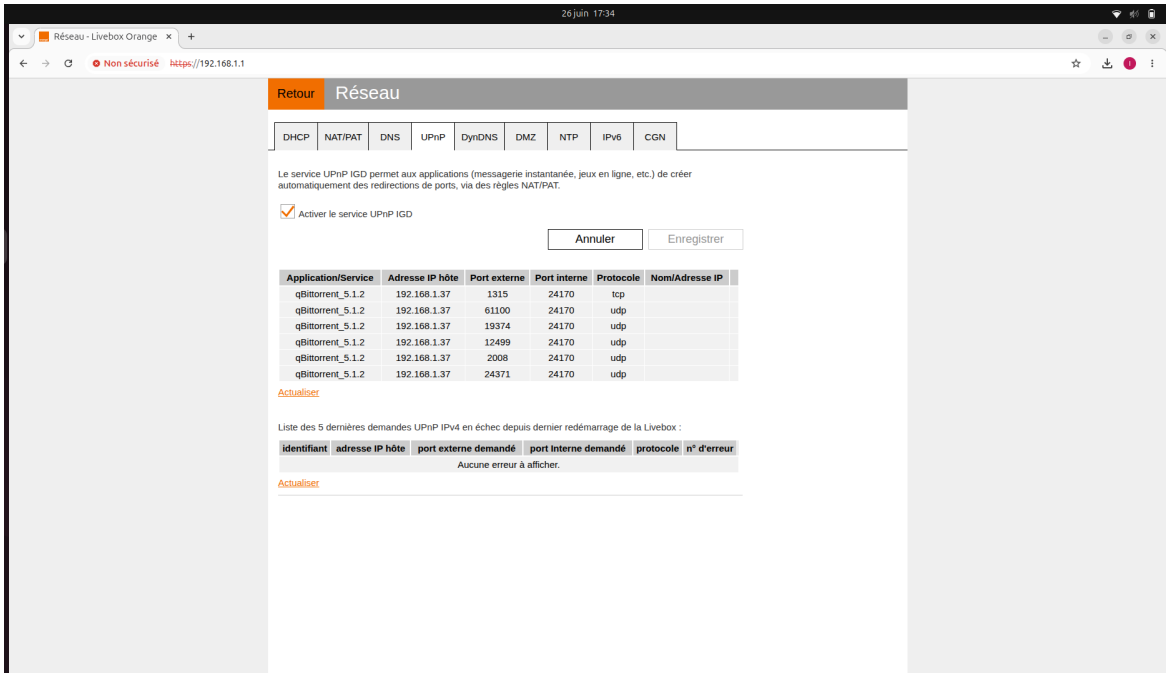


Figure 2.9: UPnP management page of the Home Gateway WebUI.

Consequently, testers had to connect to the Home Gateway through Telnet and inspect the `Firewall.PCP` data model in order to verify whether PCP mappings had been created, modified, or deleted correctly.

This limitation became even more significant in DS-Lite environments. Although the validation scenario started with an UPnP request, the corresponding PCP operation performed internally by the Home Gateway was not visible through the WebUI. Testers therefore had to rely on additional tools to verify the actual PCP behavior.

2.2.2 Fragmented Validation Workflow

Another important limitation was the fragmentation of the validation workflow.

Several separate tools were required during testing and debugging phases, including telnet access, packet capture tools, TFTP debugging mechanisms, and gateway data model inspection commands.

Since the collected information was distributed across different interfaces and tools, testers had to manually compare logs, packet captures, and gateway states in order to understand the origin of failures or abnormal behavior.

This process required additional manual analysis and increased the time needed to validate test scenarios.

2.2.3 Limited Error Analysis

Another limitation of the existing environment was related to PCP error handling and response analysis.

The PCP server deployed on the company server provided limited information about request failures and error conditions. In several scenarios, PCP errors were either minimally reported or difficult to interpret during troubleshooting operations.

As a result, testers often had to manually inspect gateway states, packet captures, and debugging logs in order to determine whether the issue originated from the HGW, the upstream PCP server, or the network environment.

This lack of detailed error visibility made debugging operations more complex and increased the time required to analyze PCP behavior.

2.2.4 Manual Mapping Verification

Another limitation observed during the validation process was the manual identification of PCP mappings on the Home Gateway.

After executing an UPnP port mapping request, testers inspected the `Firewall.PCP` data model to verify that the corresponding PCP mapping had been correctly created. However, the returned output could contain several active mappings, and the newly created entry did not necessarily appear at the end of the list or follow a chronological order.

As a result, testers had to manually locate the correct mapping by comparing several fields, including the internal port, the proposed external port, the external port assigned by the PCP server, the transport protocol, the assigned lifetime, and the PCP status. This verification required careful comparison between the request parameters and the values stored by the gateway.

When multiple mappings were active simultaneously, this process became increasingly time-consuming and error-prone. Identifying the wrong entry could lead to incorrect conclusions regarding the result of the validation scenario, especially during regression testing where numerous mappings were created and removed repeatedly.

2.2.5 Lack of Automation

Most validation and debugging operations relied on manual procedures.

After each firmware update, previously executed PCP and UPnP validation scenarios had to be repeated in order to verify that reported issues had been corrected and that no new regressions had been introduced.

This process required testers to manually execute the same validation scenarios, inspect the corresponding PCP mappings, analyze packet captures, and compare the obtained results with those collected during previous firmware versions.

Since no centralized mechanism was available to execute validation campaigns automatically or consolidate their results, regression validation became repetitive and increasingly time-consuming as the number of test scenarios increased.

2.3 Requirements Analysis

Based on the limitations identified previously, the requirements of the proposed platform are presented in this section.

2.3.1 Functional Requirements

The functional requirements define the main services and capabilities that the proposed platform must provide. They are presented in the following subsections.

2.3.1.1 PCP Request Management

The platform must allow the tester to generate and manage PCP requests from a web interface. The user can send PCP MAP and PEER requests by selecting the protocol, the internal port, the requested external port, and the mapping lifetime.

The system must also support mapping renewal and deletion. For each request, the platform displays both the raw PCP response and the decoded result, including the result code, assigned lifetime, external address, and assigned external port when available.

Finally, successful mappings and relevant response information must be stored in the database in order to keep a history of PCP operations and support later analysis.

The platform must also analyze whether the PCP server assigned the same external port requested by the tester or selected another available port. When the Prefer Failure option is not used, the server may assign a different external port. However, when Prefer Failure is used, the server is expected to reject the request if the requested external port cannot be provided.

2.3.1.2 PCP Error Handling and Response Analysis

The platform must decode PCP responses and display the corresponding result code in a clear way. The handled result codes are defined in RFC 6887 [3].

The system must also compare the requested parameters with the values returned by the PCP server. This includes checking the assigned lifetime and the assigned external port, since the server may return values different from those initially requested.

This analysis helps the tester understand whether the mapping was accepted exactly as requested or modified by the server according to its policies.

2.3.1.3 UPnP Testing and Interworking Verification

The platform must support UPnP testing by allowing the tester to create, renew, and delete UPnP port mappings from the web interface. These operations are performed using a Python UPnP client based on the `miniupnpc` library.

For each UPnP operation, the platform must display the returned result and any error reported by the HGW.

Since the HGW uses an UPnP–PCP interworking mechanism, the platform must also verify whether a UPnP action triggers a corresponding PCP state on the gateway side. This verification is performed by checking the `Firewall.PCP` data model after the UPnP operation.

This feature is important because the HGW may return a successful UPnP response even when the upstream PCP operation fails. The platform therefore improves validation by correlating the UPnP result with the PCP state exposed by the gateway.

2.3.1.4 Packet Capture and Logging

The platform must provide PCP packet capture capabilities in order to help testers analyze protocol exchanges during validation scenarios.

The user must be able to start and stop packet capture directly from the web interface. Captured PCP packets must include timestamps, addresses, ports, and PCP request or response data.

The platform must also support exporting captured traffic in `PCAP` format for additional analysis using external tools such as Wireshark.

In addition, the system must store activity logs and validation results in the database. The platform must also allow exporting logs in `CSV` format according to different categories, such as PCP logs, UPnP logs, or combined activity logs.

2.3.1.5 Remote Validation Support

The platform must support remote validation operations from different machines inside the local network environment.

The system should allow testers to execute PCP and UPnP validation scenarios remotely and collect the corresponding responses and packet captures through the centralized platform.

2.3.1.6 Supervision and Monitoring Features

The platform must provide supervision and monitoring features in order to simplify PCP and UPnP validation operations.

The system must display PCP and UPnP mapping tables containing information about active mappings, assigned ports, protocols, lifetimes, and mapping states.

The platform must also provide PCP status verification capabilities through the HGW data model.

In addition, the platform must include gateway diagnostic features allowing testers to inspect network information such as WAN state, IPv6 addressing information, MAC address, DNS configuration, and DS-Lite operating mode.

The system must also provide live activity logs and summarized diagnostic information in order to simplify troubleshooting and validation operations.

2.3.1.7 PCP Server Control and Automation

The platform must provide PCP server management features in order to simplify validation operations during testing phases.

Instead of manually connecting to the server through SSH commands to start or stop the PCP service, the platform must allow these operations to be triggered directly from the web interface.

2.3.2 Non-Functional Requirements

The non-functional requirements define the quality attributes that the proposed platform must satisfy. They are presented in the following subsections.

2.3.2.1 Usability and Operational Efficiency

The platform must simplify the validation workflow by centralizing different operations inside a single interface. Instead of switching between multiple independent tools such as telnet sessions, packet capture utilities, and manual debugging procedures, testers can access the required information directly from the dashboard.

The system must also reduce repetitive manual operations, especially for common validation scenarios that need to be repeated after firmware updates.

In addition, the platform must facilitate access to logs, packet captures, and gateway information in order to simplify troubleshooting operations.

Finally, the dashboard must provide clearer visibility into PCP and UPnP behavior through summarized information, mapping tables, activity logs, and gateway diagnostic features.

2.3.2.2 Reliability

The platform must provide reliable validation and supervision operations during PCP and UPnP testing phases.

The system must correctly handle PCP responses, request failures, and timeout situations in order to avoid incomplete or inconsistent validation results.

In addition, the platform must maintain stable communication with remote validation environments and ensure that collected information is properly stored and accessible.

2.3.2.3 Maintainability

The platform must be designed using a modular architecture in order to simplify future maintenance and feature integration.

Different functionalities such as PCP validation, UPnP testing, packet capture, logging, and remote execution should remain separated into reusable components.

2.3.2.4 Scalability and Extensibility

The platform must support future extensions and additional validation features without requiring major architectural modifications.

The system should allow the integration of new PCP test scenarios, additional supervision modules, and remote validation environments.

2.3.2.5 Performance

The platform must reduce the time required for PCP validation and troubleshooting operations.

By automating repetitive validation tasks and centralizing supervision features, the system should simplify debugging procedures and reduce the number of manual operations required during testing phases.

2.4 Proposed Solution Overview

The objective of our platform is to improve the supervision and validation of PCP and UPnP mechanisms in IPv6 environments, especially in DS-Lite architectures.

The current validation workflow relies heavily on manual testing procedures and the use of multiple separate tools, which makes testing operations time-consuming and increases the risk of human error. In addition, the existing HGW WebUI provides limited visibility into PCP behavior and DS-Lite information, making troubleshooting and protocol analysis more difficult.

To address these limitations, we propose a centralized web-based supervision and validation platform that groups PCP and UPnP testing features inside a single interface. The platform simplifies validation operations by automating repetitive test scenarios, centralizing logs and packet captures, and providing clearer visibility into protocol behavior and gateway states.

The proposed solution also improves debugging operations by helping testers better understand the origin of failures and analyze PCP and UPnP interactions more efficiently. In addition, the platform provides diagnostic and monitoring features that simplify the analysis of network behavior in DS-Lite environments.

The platform is organized around several supervision and validation modules. A PCP request page allows testers to generate PCP MAP and PEER requests and analyze the returned responses. An UPnP validation page is also provided to test UPnP mappings and verify the behavior of the UPnP–PCP interworking mechanism implemented on the HGW.

In addition, the platform includes a PCP packet capture module allowing testers to monitor PCP exchanges and export captured traffic for further analysis. Mapping tables are also available to display PCP and UPnP states and simplify the inspection of active mappings.

The proposed solution also integrates diagnostic and monitoring features through a Smart Gateway analysis page and live activity logs. These features help testers inspect gateway information, analyze DS-Lite operating mode, and simplify troubleshooting operations.

Finally, the platform supports remote validation scenarios through an agent dashboard allowing validation operations to be executed from different machines inside the local network environment.

The proposed solution is designed to support both manual validation operations and automated testing scenarios.

During manual testing phases, testers can directly visualize PCP requests, responses, assigned mappings, and protocol states through the centralized dashboard. This improves visibility into PCP behavior and simplifies troubleshooting operations without relying exclusively on low-level debugging procedures.

The platform can also simplify repetitive validation tasks by automating common PCP and UPnP test scenarios. This helps reduce validation time after firmware updates and facilitates regression testing operations.

To illustrate the added value of the proposed solution, consider a validation scenario performed after a firmware update on the HGW.

Using the traditional workflow, testers had to manually send PCP requests, inspect packet captures, and analyze logs separately in order to understand whether a mapping failure originated from the HGW or from the upstream PCP environment.

With the proposed platform, PCP requests, responses, packet captures, and gateway states are centralized inside the same interface. This allows testers to quickly identify returned PCP errors, inspect mapping behavior, and simplify troubleshooting operations during validation phases.

As a result, the platform improves visibility into PCP and UPnP interactions and simplifies both manual validation and repetitive testing operations.

Conclusion

This chapter analyzed the existing PCP and UPnP validation approaches adopted during the internship and identified the main limitations affecting the efficiency of the validation process. Based on these observations, the functional and non-functional requirements of the proposed platform were established, and an overview of the proposed solution was presented. Building upon these requirements, the next chapter focuses on the design of the NetSupervision platform by presenting its overall architecture, the adopted technological choices, the software organization, the database design, the UML models, and the deployment architecture.

Chapter 3

System Design and Architecture

Introduction

Following the analysis of the existing validation workflow and system requirements presented in the previous chapter, this chapter introduces the design of the proposed supervision platform. It presents the overall architecture, main design choices, UML models, database structure, and deployment architecture that form the basis for the implementation described in the next chapter.

3.1 Overall System Architecture

To support centralized supervision, distributed network execution, traceability, and multi-user access, we designed a distributed layered architecture. Instead of concentrating all responsibilities inside a single application, the solution separates the different concerns into independent layers. Each layer is responsible for a specific part of the validation workflow while cooperating with the others to provide a complete supervision environment. This organization improves maintainability, simplifies future extensions, and clearly separates the user interface from the network operations executed inside the local validation environment.

Figure 3.1 illustrates the overall architecture of the proposed platform. The solution is organized into five functional layers: the presentation layer, the application layer, the communication layer, the execution layer, and the persistence layer.

Each layer fulfills a specific responsibility while collaborating with the others to support the complete validation workflow. The role of each layer is described below:

- **Presentation Layer:** Provides the web interface used by validation engineers to launch validation operations, monitor the Home Gateway, consult mappings, execute regression campaigns, and visualize validation results. It only manages user interaction and forwards requests to the application layer.
- **Application Layer:** Acts as the central coordination layer of the platform. It validates user requests, manages authentication, selects the appropriate LAN Agent, coordinates validation workflows, processes execution results, and prepares the data displayed by the interface.

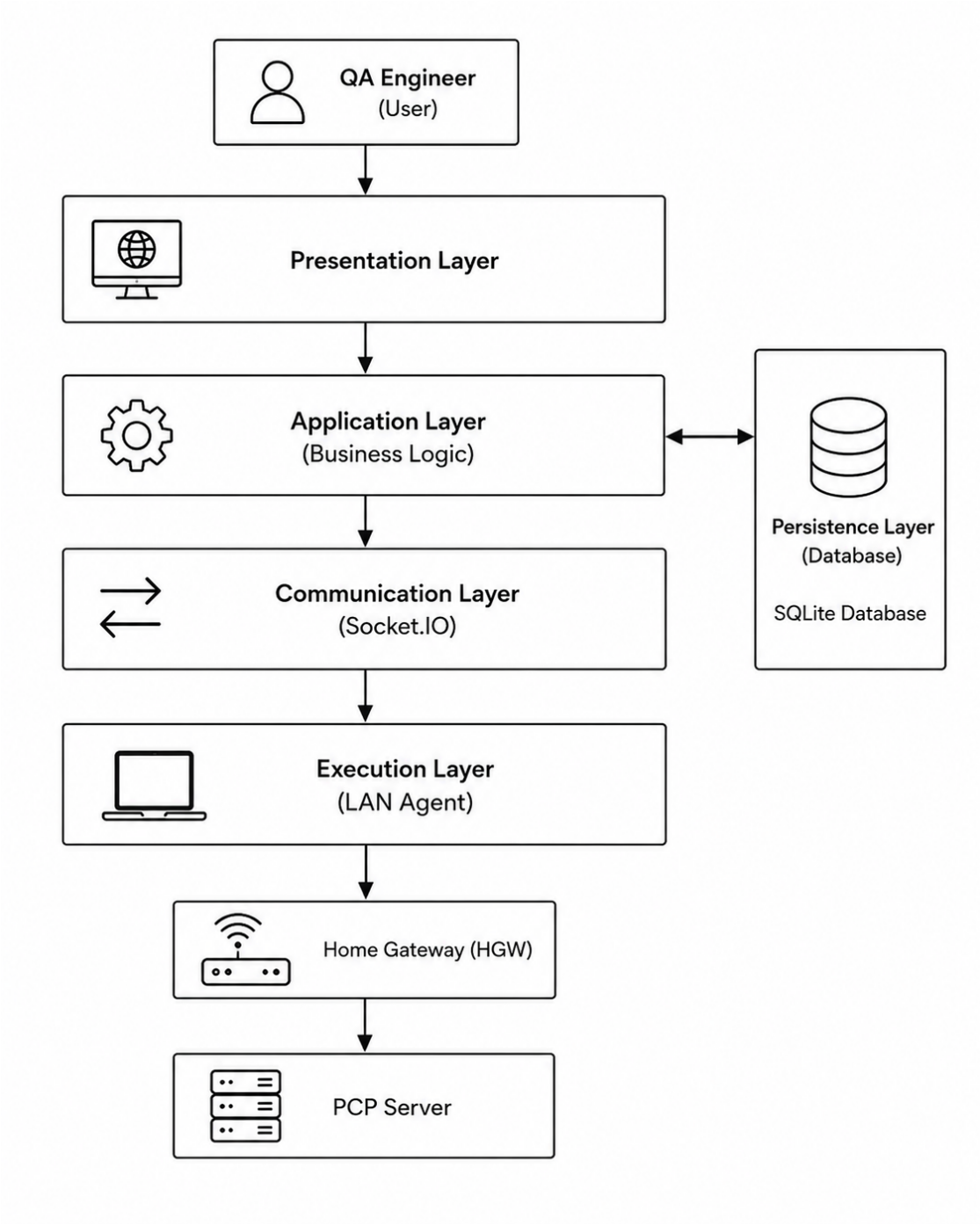


Figure 3.1: Overall architecture of the proposed supervision platform

- **Communication Layer:** Ensures communication between the centralized platform and the remote LAN Agents through Socket.IO. It exchanges commands, execution results, logs, and status information using a persistent bidirectional communication channel.
- **Execution Layer:** Corresponds to the LAN Agent deployed inside the validation network. It executes PCP and UPnP operations, inspects the Home Gateway through Telnet, performs packet captures, collects debugging information, and executes network diagnostics while preserving the local network context.

- **Persistence Layer:** Stores the data generated during validation, including users, agents, protocol mappings, campaigns, gateway information, logs, audit records, and debugging history using an SQLite database managed through SQLAlchemy.

3.2 Architectural and Technological Choices

The implementation of the proposed platform relies on several complementary technologies selected according to the project requirements. The selection was guided by compatibility with the existing validation environment, modularity, ease of deployment, support for network programming, and efficient communication between the centralized application and distributed LAN Agents. The following subsections present the principal technologies adopted for each architectural layer.

3.2.1 Central Application Technologies

Python was selected as the main programming language because it is already used by the company's development team for internal software projects. Adopting the same language ensures compatibility with the existing development environment, facilitates future maintenance by engineers already familiar with the company's technology stack, and provides the networking libraries required to implement PCP, UPnP, gateway management, and automation within a single environment [11].

The backend was implemented using Flask. Since Flask is the framework adopted by the development team for similar internal applications, using it ensures consistency with the company's software practices and simplifies future evolution of the platform. Its Blueprint architecture also enables the application to be organized into independent functional modules, such as PCP validation, UPnP validation, Smart Gateway inspection, regression testing, agent management, reporting, and authentication [12].

Jinja was selected as the server-side templating engine because the platform mainly consists of validation forms, monitoring dashboards, protocol results, mapping tables, and administrative pages. Its native integration with Flask allows dynamic web pages to be generated without introducing the complexity of a separate frontend framework [13].

The graphical interface was developed using HTML, CSS, and JavaScript. These technologies provide a lightweight and highly customizable interface adapted to the validation workflow. JavaScript is mainly used for interactive features such as confirmation dialogs, filtering, asynchronous updates, and dynamic visualization of validation results [14, 15, 16].

3.2.2 Communication Technologies

Figure 3.2 presents the internal organization of the central application and the communication mechanisms between its principal components.

Communication inside the platform relies on two complementary channels. The web browser communicates with the Flask application through standard HTTP requests, while real-time communication between the central application and the LAN Agent is achieved using Socket.IO.

Socket.IO provides bidirectional event-based communication and allows the server to communicate with a specific connected LAN Agent through its Socket.IO session identifier (SID) [17]. Unlike a traditional request-response model, it enables the server to initiate communication with the selected LAN Agent and receive asynchronous validation results. This mechanism is essential because protocol validation operations are executed remotely and may require several seconds before returning a response.

Internally, the communication mechanism relies on three complementary components. The Agent Registry maintains information about connected LAN Agents, the Command Dispatcher forwards validation requests to the selected agent, and the Pending Request Manager correlates asynchronous responses with their corresponding requests while handling concurrent executions and timeout management.

3.2.3 Network Validation Technologies

The LAN Agent executes protocol validation operations locally inside the validation network using several specialized technologies.

3.2.3.1 Python Sockets for PCP Communication

Direct PCP operations are implemented using Python sockets over UDP. A custom packet builder constructs PCP requests while a dedicated decoder interprets PCP responses according to RFC 6887 [3]. This implementation provides complete control over protocol fields and validation parameters required during testing.

3.2.3.2 MiniUPnPc for UPnP Validation

UPnP IGD operations are performed using MiniUPnPc [18]. The library provides the client operations required to create, inspect, renew, and delete port mappings while allowing the platform to verify the corresponding PCP behavior on the gateway.

3.2.3.3 Telnet-Based Gateway Access

Python's `telnetlib` module is used to establish Telnet sessions with the Home Gateway (HGW). It allows the platform to execute management and diagnostic commands, retrieve gateway information, and inspect `Firewall.PCP` entries during protocol validation. This approach matches the management interface available on the gateways used in the company's validation laboratory [19].

3.2.3.4 Remote Server Administration

Paramiko is used to establish SSH connections with the remote PCP validation server. Through this secure connection, the platform performs administrative operations required during validation, such as starting or stopping the PCP server, configuring validation scenarios, and executing remote maintenance operations [20].

3.2.4 Persistence Technologies

The persistence layer was implemented using SQLite and SQLAlchemy. Since the platform is intended for deployment inside the company's validation laboratory, using a

dedicated database server was unnecessary. SQLite therefore provides a lightweight and portable solution while remaining sufficient to store users, validation campaigns, protocol mappings, gateway information, debug files, activity logs, and audit records [21].

SQLAlchemy was adopted as the Object Relational Mapper (ORM) to separate the application logic from database management. It simplifies the manipulation of persistent objects while providing a clear and maintainable data model [22].

This combination minimizes database administration while preserving the possibility of migrating to another relational database management system if the platform evolves toward larger-scale deployments.

3.2.5 Authentication, Authorization and Traceability

Access to the platform is controlled through username and password authentication. Passwords are securely stored as hashes using Werkzeug security functions, while Flask sessions maintain the authenticated user's identity and associated role between successive HTTP requests [23, 24].

Two user roles are currently implemented: *superadmin* and *engineer*. Access-control decorators restrict administrative functionalities to authorized users while protecting the remaining application resources.

Operational traceability is ensured through activity logs and audit records. Important operations store the responsible user, executed action, timestamp, and additional contextual information. At the communication level, unique request identifiers and agent verification mechanisms ensure that each response is correctly associated with its originating validation request while timeout management prevents indefinite waiting for unavailable agents.

The selected technologies provide a modular software foundation adapted to the requirements of the proposed validation platform. Their combination enables centralized management of PCP and UPnP validation, efficient communication with distributed LAN Agents, Home Gateway supervision, remote server administration, and reliable storage of validation data while facilitating future maintenance and evolution of the platform.

3.3 Software Architecture

The layered architecture presented in the previous section provides a global view of the proposed platform and identifies the responsibilities assigned to each architectural layer. This section translates this organization into concrete software components and explains how these components collaborate during PCP and UPnP validation operations.

The design was mainly driven by three constraints. First, the central validation server cannot directly access the local network of every validation engineer. Second, PCP and UPnP operations must be executed from the same network context as the tested Home Gateway. Third, validation results must be centralized to support traceability, regression analysis, and comparison between firmware versions.

To address these constraints, the proposed solution follows a centralized orchestration and distributed execution model. The central application manages users, validation workflows, results, and historical data, while network-dependent operations are delegated to LAN Agents installed on validation workstations.

The software architecture is therefore divided into two main execution domains:

- the **central application**, which manages user interaction, validation orchestration, authentication, result processing, persistence, and traceability;
- the **LAN Agent**, which executes operations requiring direct access to the validation network, the Home Gateway, the PCP server, and local diagnostic tools.

This separation allows the platform to centralize validation management without moving network execution away from the environment in which the tested equipment is deployed.

3.3.1 General Software Organization

Figure 3.2 presents the main software components of the platform and their interactions.

The central application contains functional modules corresponding to the main validation activities. These modules share common authentication, communication, persistence, and logging services.

On the remote side, each LAN Agent maintains a Socket.IO connection with the central application and provides execution services for PCP, UPnP, Home Gateway inspection, packet capture, network diagnostics, and debug collection.

The Home Gateway and PCP server remain outside the software boundary because they are part of the validation infrastructure rather than components of the developed platform.

The architecture separates four principal responsibilities:

- the functional modules define and coordinate validation workflows;
- the communication service exchanges commands and results with LAN Agents;
- the LAN Agent performs operations requiring local network access;
- the persistence service stores results and validation histories.

This organization prevents protocol execution from being embedded directly in the web interface. It also enables the same central application to supervise several validation workstations.

3.3.2 Central Application Architecture

The central application represents the coordination point of the platform. Its role is not limited to displaying web pages. It receives validation requests, checks the user's session and permissions, identifies the selected LAN Agent, prepares the execution parameters, processes the returned result, stores relevant information, and presents the final outcome through the web interface.

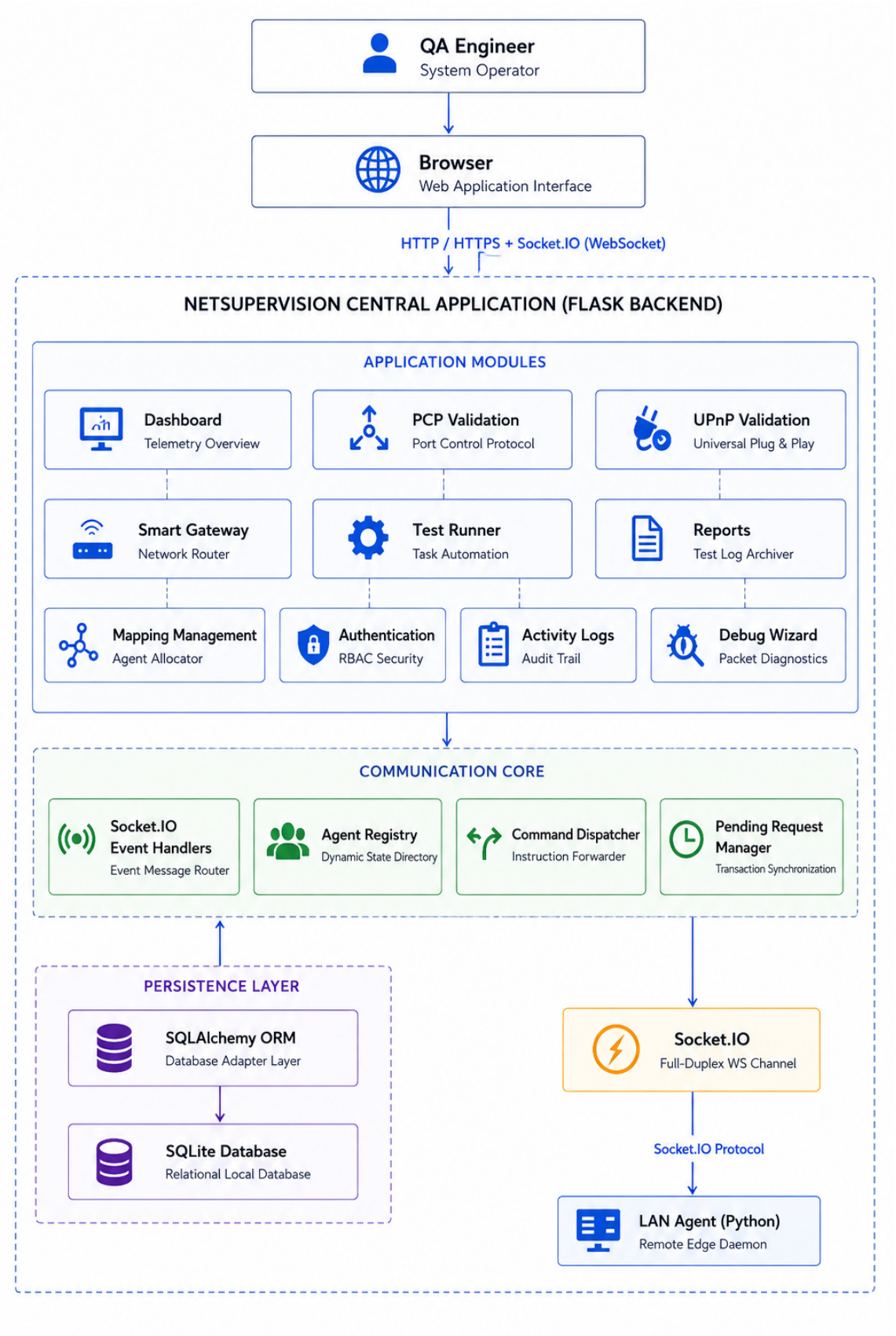


Figure 3.2: Software architecture of the proposed validation platform

The application was progressively organized into functional modules. Each module corresponds to a specific part of the validation process while reusing the same shared services.

The main functional modules include:

- authentication and role-based access control;
- dashboard and validation statistics;
- PCP request execution and response analysis;
- UPnP IGD operations and PCP-state verification;
- PCP and UPnP mapping management;
- LAN Agent registration and monitoring;
- Smart Gateway inspection;
- packet capture and protocol analysis;
- automated regression campaigns;
- debug-file collection and comparison;
- activity and audit logging;
- capability comparison and reporting.

Although these modules provide different functions, they follow a similar processing sequence. A user request is first received by the corresponding controller. The submitted parameters are then validated, and the controller determines whether the operation can be processed centrally or must be delegated to a LAN Agent.

For a remote operation, the controller builds a command containing the parameters required by the selected service. Once the result is returned, the controller interprets it according to the validation context, stores the relevant data, records the activity, and renders the result through the appropriate interface.

This modular decomposition reduces coupling between features and allows the platform to be extended progressively without redesigning the entire application.

3.3.3 Communication Management

Communication between the central application and the LAN Agents is based on Socket.IO. This technology provides bidirectional and event-based communication, allowing the central application to send validation commands to a selected workstation and receive the corresponding results.

The communication mechanism internally relies on an Agent Registry, a Command Dispatcher, and a Pending Request Manager. These components are grouped into a shared communication service so that the functional modules do not need to manage Socket.IO sessions or asynchronous responses directly.

When a LAN Agent connects, the platform associates its persistent agent identifier with its active Socket.IO session. The persistent identifier allows the application to

recognize the same workstation after a restart, while the Socket.IO session identifies its current connection.

For each remote operation, the communication service:

1. verifies that the selected LAN Agent is connected;
2. retrieves its current communication session;
3. generates a unique request identifier;
4. sends the command and its parameters to the agent;
5. waits for the corresponding result;
6. returns the response to the originating functional module.

The request identifier is returned by the LAN Agent with the execution result. It allows the central application to associate asynchronous responses with their original validation requests, even when several users or agents are active at the same time.

A timeout is applied to remote operations to avoid indefinite waiting when an agent becomes unavailable or when a network operation does not return a result.

This shared mechanism is reused by PCP execution, UPnP operations, gateway inspection, packet capture, network diagnostics, and debug collection. It therefore provides consistent communication behavior across the platform without duplicating the same logic inside every module.

3.3.4 Persistence Services

The central application uses a shared persistence service based on SQLAlchemy. Its purpose is to preserve information that must remain available after the completion of a web request or the disconnection of a LAN Agent.

The main stored entities include:

- users and roles;
- activity and audit records;
- registered LAN Agents and execution logs;
- PCP and UPnP mappings;
- test cases, campaigns, and validation results;
- Home Gateway capability snapshots;
- captured PCP packet information;
- Firewall PCP entries;
- debug executions and analysis histories.

Persistence is particularly important for regression validation. An individual protocol response may be displayed immediately, but mappings, test campaigns, gateway snapshots, packet captures, and debug comparisons must remain available for later consultation.

The detailed organization and relationships between these entities are presented in Section 3.4.

3.3.5 LAN Agent Architecture

The LAN Agent was introduced to preserve the real network context of the validation workstation. Unlike the central application, which is responsible for orchestration and supervision, the agent executes operations that require direct access to the local validation environment.

These operations include PCP and UPnP execution, communication with the Home Gateway management interface, inspection of PCP-related gateway data-model entries, packet capture, debug collection, and IPv4/IPv6 connectivity diagnostics.

Several of these operations cannot be reliably executed by the central server. UPnP discovery and gateway management require access to the validation LAN, while PCP requests must follow the same IPv6 path and source-address context as the engineer's workstation. Delegating these operations to the LAN Agent therefore preserves the network conditions of the original manual validation process.

The LAN Agent receives commands from the central application, invokes the appropriate local service, and returns the collected results for visualization, storage, and comparison.

Figure 3.3 illustrates the main internal components of the LAN Agent.

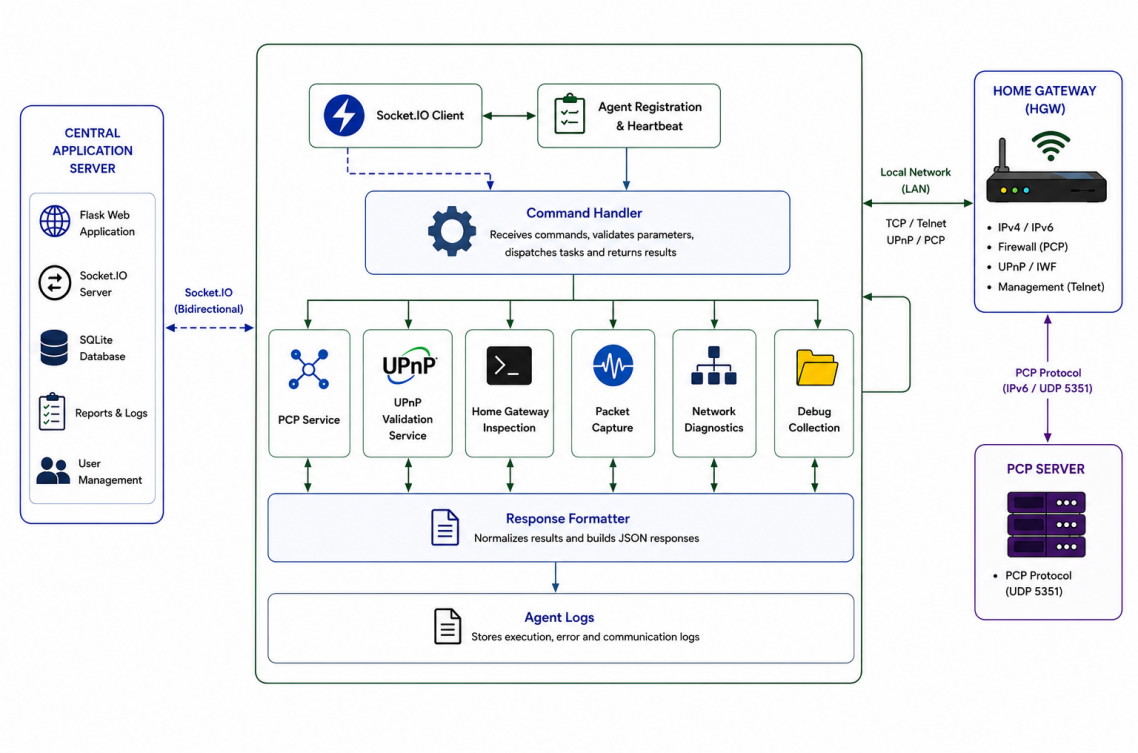


Figure 3.3: Internal software architecture of the LAN Agent

The LAN Agent contains a Socket.IO communication interface, command handlers, and specialized services for protocol execution and network inspection.

3.3.5.1 Agent Communication Interface

The Socket.IO client manages the connection with the central application, agent registration, command reception, and result transmission. At connection time, the agent sends its persistent identifier, hostname, operating system, software version, and available IPv4 and IPv6 addresses.

Each command contains a request identifier and the parameters required by the selected execution service. The same identifier is returned with the execution result, allowing the central application to associate the response with the correct validation request. This interface remains independent of protocol-specific services and can therefore be reused by additional validation functions.

3.3.5.2 PCP Execution Service

The PCP execution service constructs and transmits PCP requests over UDP from the LAN Agent workstation, preserving its network path and IPv6 source-address context.

The service supports MAP, PEER, and ANNOUNCE operations and receives parameters such as the protocol version, opcode, transport protocol, client address, internal and requested external ports, lifetime, and selected PCP options. These parameters are converted into a binary PCP message and transmitted to the validation server over UDP port 5351.

After receiving the response, the service decodes the PCP header and opcode-specific fields, including the result code, server epoch time, assigned external address and port, accepted lifetime, and returned protocol options. The decoded response and relevant raw protocol data are then returned to the central application.

The same service supports mapping creation, renewal, and deletion, providing direct control over PCP packet construction and validation parameters.

3.3.5.3 UPnP-to-PCP Validation Service

The UPnP service performs Internet Gateway Device operations against the Home Gateway from the validation LAN. It supports `AddPortMapping`, `AddAnyPortMapping`, mapping inspection, renewal, and deletion. The service runs on the LAN Agent because UPnP discovery and IGD communication require direct access to the Home Gateway's LAN-side interface.

In a DS-Lite environment, the immediate IGD result represents the local interaction between the client and the Home Gateway, while the gateway may translate the UPnP request into a PCP operation toward the external PCP server. Consequently, a successful local UPnP response does not necessarily prove that the corresponding PCP operation completed successfully; the WAN-side operation may return an error, allocate a different external port, or apply a different lifetime.

For this reason, the platform combines the UPnP result with gateway-side inspection. It retrieves the PCP state from the gateway data model and compares it with the immediate IGD result, providing visibility into both the LAN-side UPnP interaction and the WAN-side PCP operation.

3.3.5.4 Gateway Inspection and Network Diagnostics

The gateway inspection service communicates with the Home Gateway through its management interface and provides shared access for the different validation modules. It retrieves `Firewall.PCP` mapping entries, `Device.PCP.Client` parameters, gateway identity and software version, WAN status, IPv4/IPv6 configuration, routing information, DS-Lite configuration, and PCP server reachability.

This information supports Smart Gateway inspection, mapping verification, UPnP-to-PCP validation, PCP error-propagation analysis, and diagnostic workflows. In particular, `Firewall.PCP` entries provide additional visibility into the PCP processing performed toward the WAN side during UPnP validation.

Network diagnostics complement gateway inspection by verifying addressing, routing, IPv4 and IPv6 connectivity, and reachability of the Home Gateway and PCP server. This helps distinguish protocol failures from connectivity or network-configuration problems.

3.3.5.5 Packet Capture and Protocol Correlation

The packet capture service records PCP exchanges from a local network interface, providing raw network evidence associated with a validation operation. Tools such as `tcpdump` or `tshark` are used to capture traffic exchanged with the PCP server over UDP port 5351, with filtering by server address and port.

Packet capture is executed from the LAN Agent so that the observed packets correspond to the real validation traffic. The captured data complement the decoded PCP result by allowing correlation between submitted parameters, transmitted packets, server responses, and the state observed in the Home Gateway.

3.3.5.6 Debug Collection Service

The debug collection service automates the acquisition of diagnostic information from the Home Gateway. The LAN Agent performs the operations requiring direct access to the gateway and the local file-transfer environment, replacing the manual process of triggering debug generation, transferring the resulting archive, and extracting its content.

The collected information is returned to the central application, where it can be stored, parsed, displayed, and compared with previous executions or firmware versions. This keeps network-dependent collection on the LAN Agent while centralizing historical analysis and comparison.

3.3.6 Component Interaction Workflows

The previous subsections described the responsibilities of the principal software components. This subsection presents representative PCP and UPnP workflows to show how these components collaborate during validation.

3.3.6.1 Direct PCP Validation Workflow

A direct PCP validation request illustrates how the different software components collaborate to execute a protocol operation. Unlike the previous subsections, which described the responsibilities of each component individually, this workflow focuses on their interactions during a complete PCP validation. Figure 3.4 presents the sequence of interactions between the validation engineer, the web application, the communication service, the LAN Agent, the Home Gateway, the PCP server, and the persistence layer. The workflow begins when the validation engineer submits a PCP request through

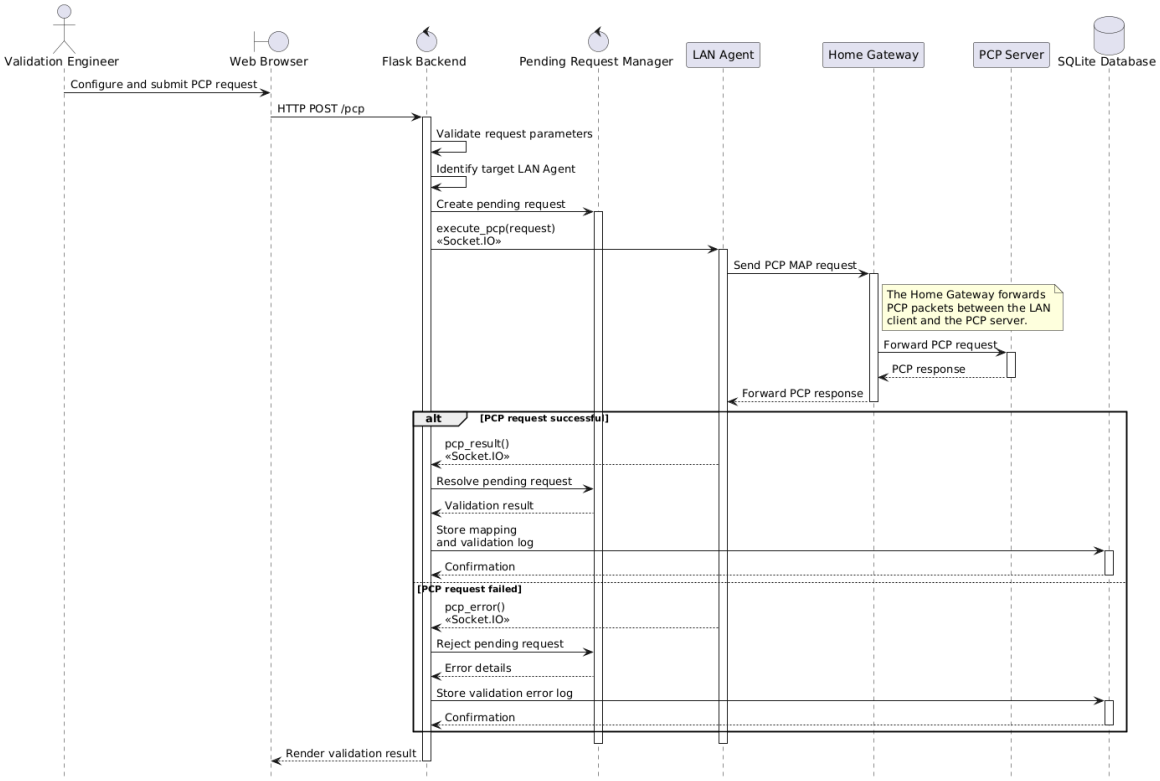


Figure 3.4: Sequence of a direct PCP validation request

the web interface. After receiving the HTTP request, the Flask backend validates the submitted parameters, identifies the LAN Agent selected for the current user session, and creates a pending request. The command is then transmitted to the selected LAN Agent through the Socket.IO communication service, together with a unique request identifier allowing the platform to correlate the asynchronous response with the originating request.

The LAN Agent invokes the PCP execution service, constructs the binary PCP message containing the server address, client address, opcode, transport protocol, ports, lifetime, and selected options, and sends it over UDP through the Home Gateway toward the PCP server. The Home Gateway simply forwards the PCP packets between the validation workstation and the server without modifying the application workflow.

Once the PCP server returns its response, the LAN Agent decodes the protocol result and sends it back to the central application using the same request identifier. The Pending Request Manager associates the asynchronous response with the originating request before the backend processes the returned data.

If the operation succeeds, the platform stores the mapping information and the associated validation log in the SQLite database before presenting the decoded result to the user. If the PCP request fails, the platform records the error details instead while still preserving the execution trace for later analysis.

This interaction model is reused for PCP mapping creation, renewal, deletion, PEER requests, ANNOUNCE requests, and protocol error analysis. The execution flow remains identical, while only the PCP parameters and the expected protocol responses vary.

3.3.6.2 UPnP-to-PCP Interworking Workflow

The UPnP validation workflow illustrates how the platform handles the additional complexity introduced by the UPnP-to-PCP interworking function. Unlike a direct PCP request, a UPnP operation produces two distinct sources of validation information that the platform must combine into a single result.

Figure 3.5 presents the complete sequence of interactions between the validation engineer, the web application, the LAN Agent, the Home Gateway UPnP IGD-PCP interworking function [4], the PCP server, and the persistence layer. The workflow begins when the validation engineer submits a UPnP mapping request through the web interface. The Flask backend creates a pending request and forwards the command to the selected LAN Agent through the Socket.IO communication service. The LAN Agent performs the UPnP operation against the Home Gateway IGD interface using either `AddPortMapping` or `AddAnyPortMapping` and collects the immediate UPnP response.

In a DS-Lite environment, the Home Gateway translates the UPnP request into a PCP MAP request toward the external PCP server. The returned PCP result is reflected in the gateway's internal PCP state. However, the immediate UPnP response does not necessarily represent the final WAN-side PCP outcome. Therefore, NetSupervision performs an additional inspection of the Firewall.PCP data model to retrieve the effective PCP status and correlate it with the local UPnP result. When the UPnP operation completes locally, the platform performs an additional Home Gateway inspection to retrieve the result actually returned by the PCP server. The LAN Agent connects to the gateway through Telnet and queries its data model to read the PCP mapping status, including the assigned external port, the granted lifetime, and any error code returned by the server. This step is necessary because a locally accepted UPnP operation may correspond to a modified external port, a reduced lifetime, or a mapping state that remains unsuccessful on the WAN side.

The central application then combines the immediate UPnP response with the PCP state observed in the gateway data model to produce a unified validation result. If both sources are consistent, the platform stores the UPnP mapping, the PCP status, and the associated logs before presenting the successful result to the engineer. If an inconsistency is detected between the UPnP response and the PCP state reported by the gateway, the platform records a validation warning together with the mismatch details. In the event of a communication failure, the error is logged and the execution trace is preserved for later analysis.

This two-stage validation model is reused for all UPnP mapping operations and pro-

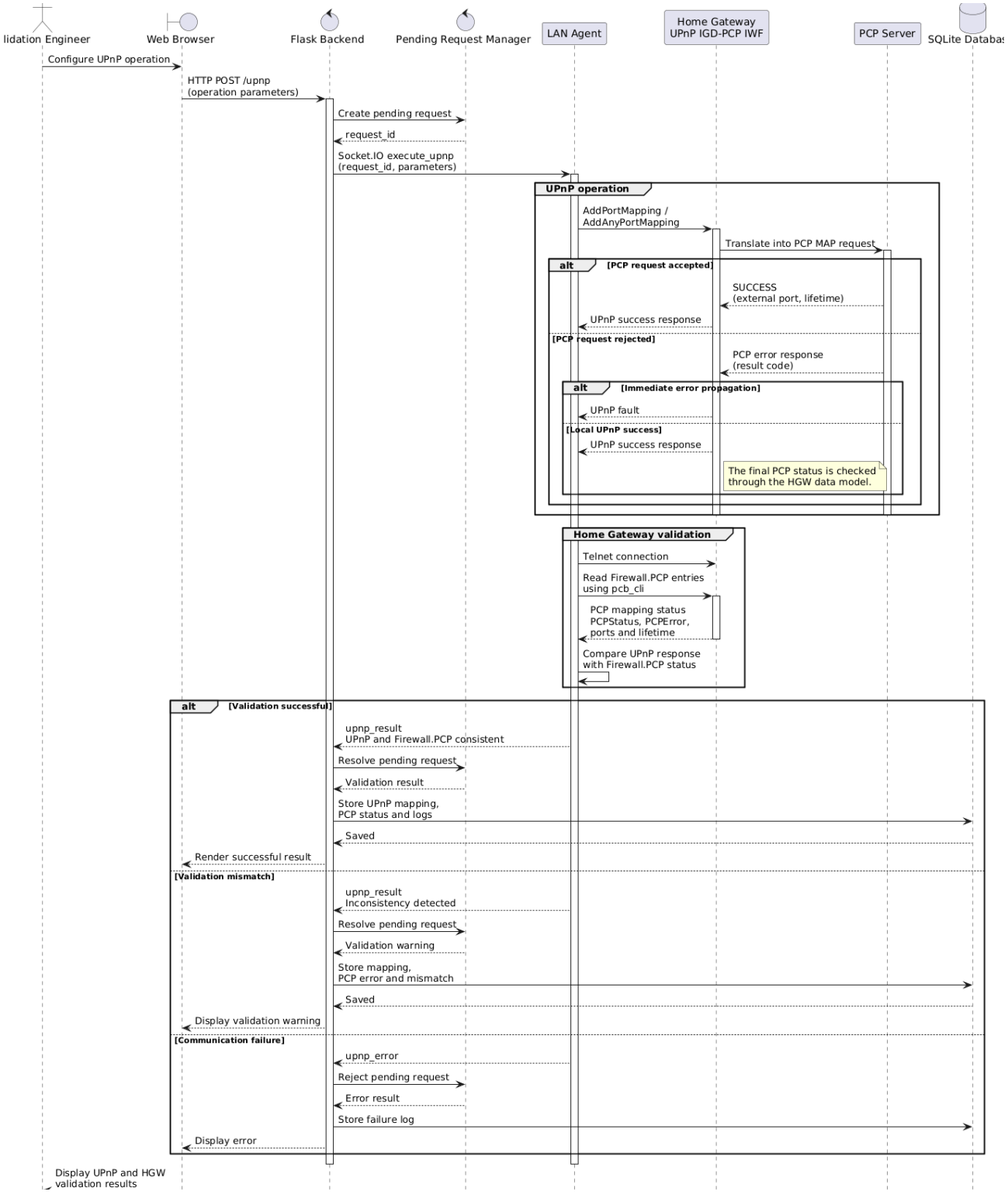


Figure 3.5: Sequence of a UPnP-to-PCP interworking validation request

vides visibility into both the local IGD operation and the PCP processing performed toward the WAN side.

3.3.6.3 Automated Regression Workflow

The Test Runner reuses the same software components to execute complete validation campaigns.

A campaign contains a sequence of predefined test cases. Each test case describes an operation, its input parameters, and the expected validation result.

During execution, the Test Runner:

1. loads the selected test cases;
2. sends the required operation to the selected LAN Agent;
3. waits for the protocol or gateway result;
4. compares the returned values with the expected result;
5. records the test status and execution details;
6. continues with the following test case.

A campaign may combine direct PCP operations, UPnP-to-PCP operations, mapping renewal, deletion, lifetime verification, and error-propagation scenarios.

The results are stored centrally so that complete campaigns can be reviewed and compared after a firmware update.

This workflow transforms the platform from an individual protocol-testing interface into a reusable regression-validation environment.

3.4 Database Design

The NetSupervision database preserves the information generated during PCP and UPnP validation, including protocol mappings, gateway states, packet information, regression results, LAN Agent data, capability snapshots, debug information, and activity records. Its main purpose is to support traceability, protocol analysis, regression comparison, and troubleshooting across different Home Gateway firmware versions.

The choice of SQLite and SQLAlchemy was already discussed in Section 3.2. This section therefore focuses on the organization of the stored data and the relationships between the principal entities.

3.4.1 Database Organization

The database is managed by the central application. Network-dependent operations are executed by the LAN Agent, while the resulting information is returned to the central platform and stored centrally.

The stored data covers the main platform functions: user and audit management, PCP and UPnP validation, regression testing, LAN Agent monitoring, Home Gateway analysis, packet capture, debug management, and optional AI-assisted analysis.

This organization preserves the execution context of the validation workstation while maintaining a centralized history of the collected evidence.

3.4.2 Entity Relationship Model

Figure 3.6 presents the principal database entities and their relationships.

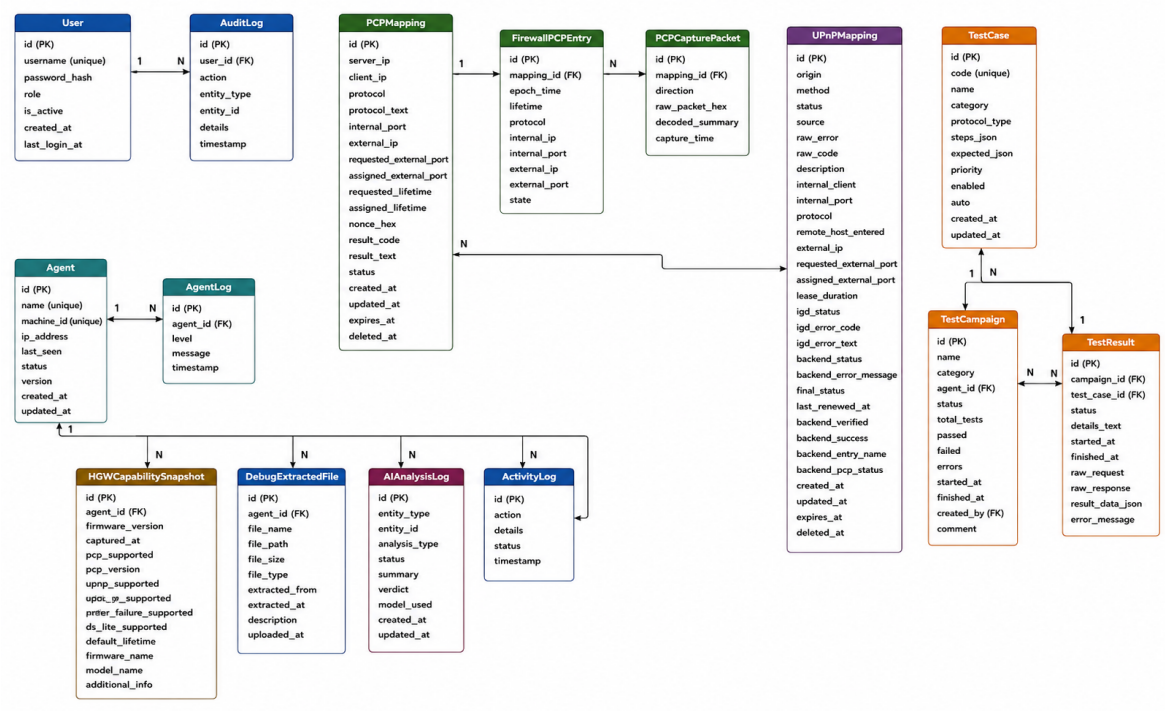


Figure 3.6: Entity relationship diagram of the NetSupervision database

PCP validation mainly relies on the `PCPMapping`, `PCPCapturePacket`, and `FirewallPCPEntry` entities, which preserve the mapping state, exchanged packet information, and PCP state observed inside the Home Gateway. UPnP validation is represented by `UPnPMapping`, which stores both the immediate IGD result and the corresponding PCP-related state retrieved from the gateway.

Regression testing is represented by `TestCase`, `TestCampaign`, and `TestResult`, while the `Agent` and `AgentLog` entities preserve information about the workstation and its execution events. Authentication and traceability are supported by `User`, `ActivityLog`, and `AuditLog`.

Additional entities such as `HGWCapabilitySnapshot`, `DebugExtractedFile`, and `AIAnalysisLog` support firmware comparison, debug analysis, and optional AI-assisted interpretation.

3.4.3 Data Relationships and Integrity

Explicit foreign keys are used for stable relationships, particularly between users and audit records, test campaigns and their results, and test cases and execution results. Logical identifiers are used when distributed information, such as the LAN Agent identity, must remain available even when the corresponding agent is offline.

Mapping records preserve their lifecycle instead of being immediately removed, allowing the platform to distinguish active, renewed, expired, and deleted states. Flexible information such as test steps, expected results, gateway data, packet details, and analysis results can also be stored in structured JSON form when their content varies between executions.

Overall, the database centralizes the evidence generated during validation while keeping protocol execution within the appropriate network environment. This supports traceability, comparison between firmware versions, regression testing, and troubleshooting.

3.5 System Modeling

To complement the software architecture presented in the previous section, the proposed solution is described through UML diagrams. These diagrams provide complementary views of the NetSupervision platform by illustrating its main functionalities, the organization of its persistent data, and the interactions between its components during the execution of the principal validation scenarios.

3.5.1 Global Use Case Diagram

The NetSupervision platform provides a set of functionalities that support PCP and UPnP validation throughout the different stages of the validation process. Figure 3.7 presents the global use case diagram of the proposed solution.

The validation engineer is the main actor interacting with the platform. After authentication, the engineer can perform PCP and UPnP-to-PCP validation, inspect the Home Gateway, execute regression campaigns, capture PCP packets, collect debug information, manage protocol mappings, control the PCP validation server, and consult validation results and activity logs.

Several validation operations require a LAN Agent to execute the requested task within the local validation network. Consequently, selecting a LAN Agent is a mandatory step before executing network-dependent operations. The platform also interacts with the Home Gateway during gateway inspection and validation, while the PCP validation server is responsible for processing PCP requests during the validation scenarios.

3.5.2 Class Diagram

Figure 3.8 presents the software design class diagram of the NetSupervision platform. It highlights the main software classes and services, their responsibilities, and their interactions across agent management, PCP and UPnP validation, automated regression, gateway inspection, and debug analysis.

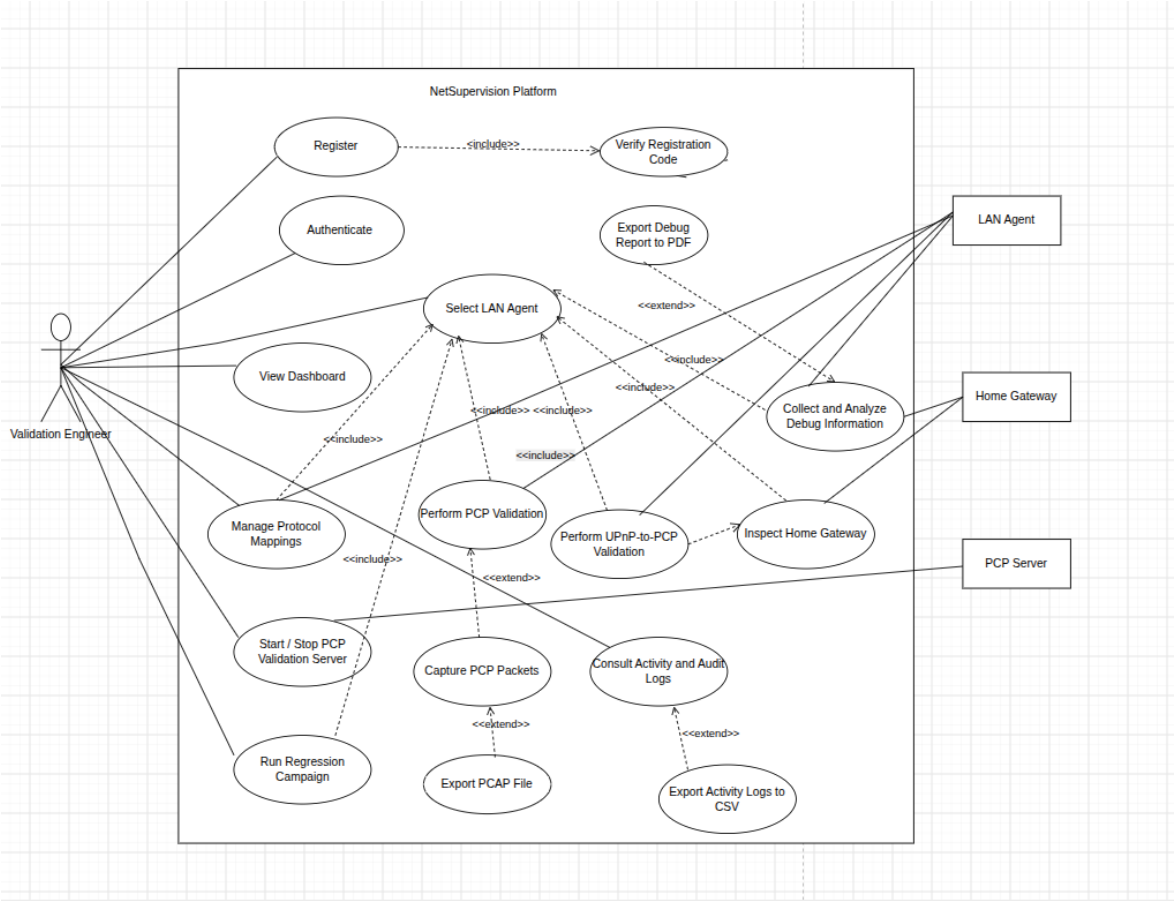


Figure 3.7: Global use case diagram of the NetSupervision platform

3.6 Deployment Architecture

This section presents the physical deployment of NetSupervision and the distribution of its components within the validation environment.

3.6.1 Deployment Strategy

The deployment architecture of NetSupervision evolved progressively throughout the internship according to the operational needs of the validation team. During the first stage of the project, the platform was implemented as a local application running entirely on a single validation workstation. This architecture was sufficient for developing the different validation features and performing the initial PCP and UPnP experiments.

As the project progressed, the objective shifted from a personal development prototype to a collaborative validation platform intended to be shared by multiple validation engineers. Deploying the application on the company's central server provided a single access point through a web browser while centralizing user management, validation campaigns, reports, activity logs, and mapping history. This centralized deployment also simplified maintenance since application updates are performed only on the server without requiring reinstallation on each workstation.

However, the central server does not share the local network context of the Home Gateway under test. Network-dependent operations such as UPnP discovery, gateway

access, and PCP execution therefore could not be reliably performed from the server. In particular, PCP requests executed centrally originated from the server address rather than from the validation workstation, preventing the reproduction of the expected IPv6 validation conditions.

Another important requirement was to support collaborative validation. Multiple engineers may simultaneously execute independent validation campaigns while sharing the same PCP validation server. Without a centralized platform, actions performed by one engineer, such as starting or stopping the PCP server, would remain invisible to other engineers, making troubleshooting considerably more difficult.

To address these challenges, the platform evolved towards a distributed deployment architecture. The central server manages the application and stores all validation data, whereas protocol execution is delegated to lightweight LAN Agents deployed on the validation workstations connected to the target Home Gateway.

3.6.2 Final Deployment Architecture

Figure 3.9 illustrates the final deployment architecture adopted for NetSupervision. The solution follows a distributed client-server model in which the web application is hosted on the company's central server while the validation operations are executed locally by LAN Agents installed on the engineers' workstations.

When an engineer launches a PCP or UPnP operation, the central application forwards the request through Socket.IO to the selected LAN Agent. The agent executes the operation within the local validation network, communicates with the Home Gateway and PCP server when required, and returns the collected results to the central application for visualization and storage.

3.6.3 Deployment Components

The deployment architecture is composed of four main components:

- **Central Server:** Hosts the Flask web application, the Socket.IO communication service, the SQLite database, and the reporting modules. It acts as the coordination point for all validation activities while maintaining a centralized repository of users, mappings, validation campaigns, and activity logs.
- **LAN Agent:** A lightweight Python service deployed on each validation workstation. The agent performs all operations requiring local network access, including PCP requests, UPnP validation, Home Gateway inspection through Telnet, packet capture, and debug information collection.
- **Home Gateway (HGW):** The device under validation. It receives PCP and UPnP requests from the LAN Agent and forwards PCP messages to the external PCP validation server.
- **PCP Validation Server:** Processes PCP requests and returns the corresponding protocol responses, allowing the platform to validate different operational and error scenarios.

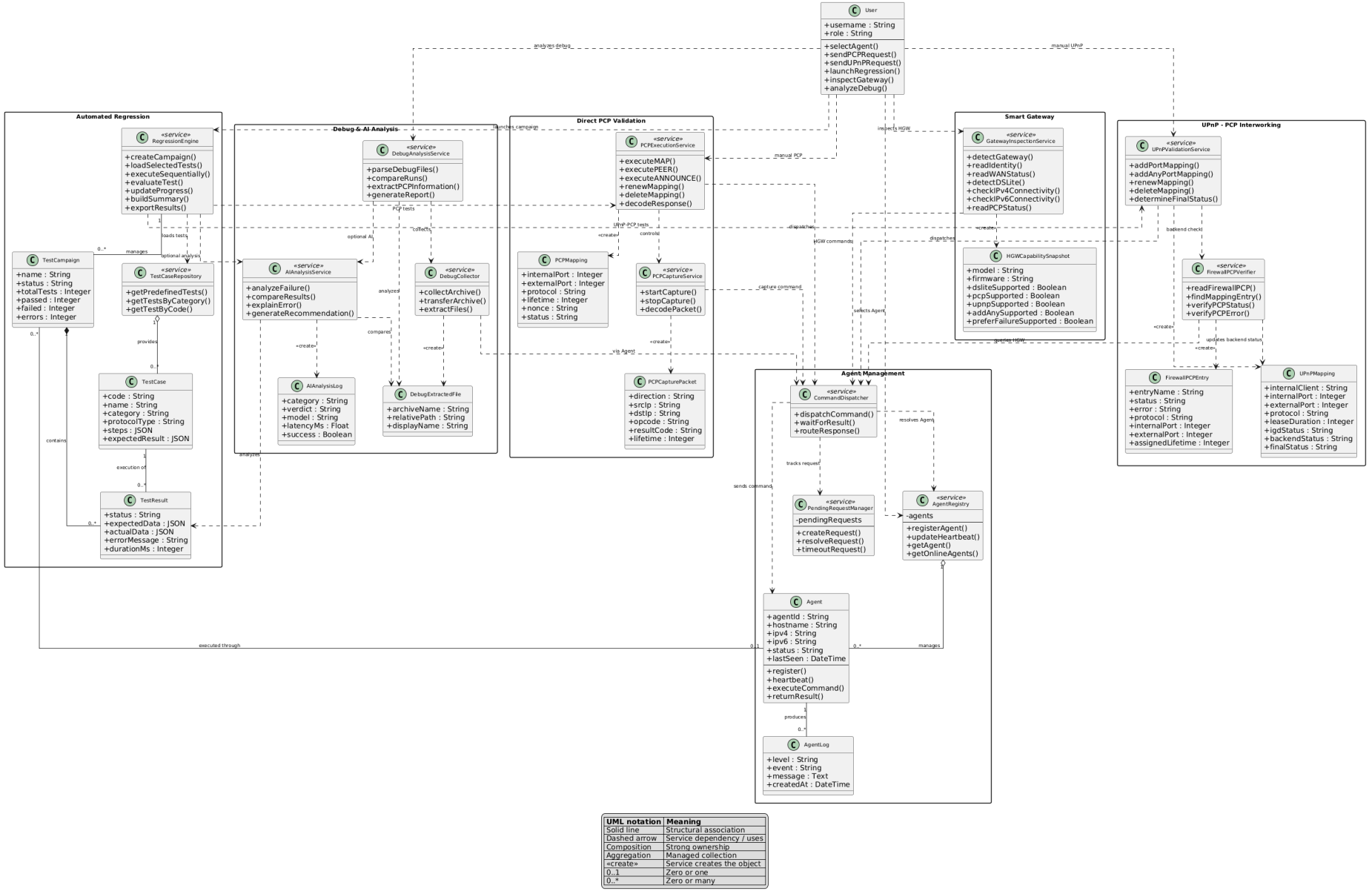


Figure 3.8: Software Design Class Diagram of the NetSupervision Platform

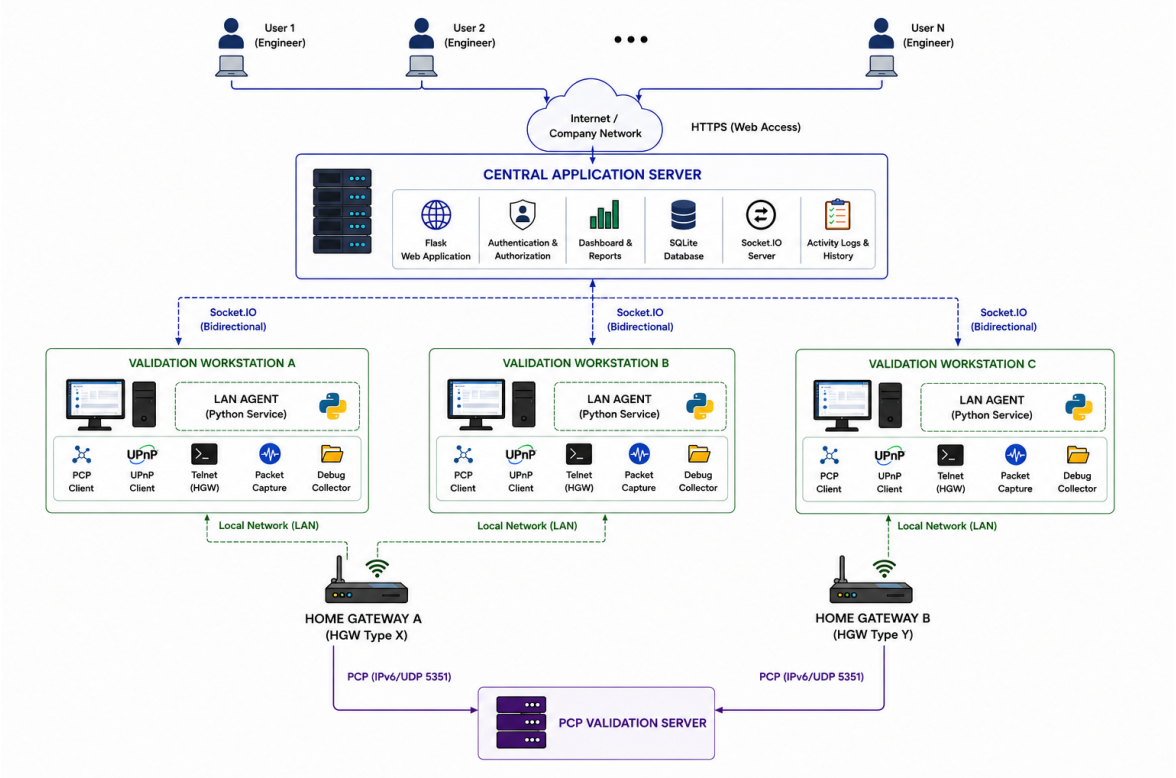


Figure 3.9: Deployment architecture of the NetSupervision platform.

This separation of responsibilities allows each component to execute only the tasks that correspond to its deployment environment while maintaining centralized supervision of the overall validation process.

3.6.4 Deployment and Maintenance

The proposed deployment architecture was also designed to simplify platform administration and future maintenance.

A new validation workstation can be integrated by installing the LAN Agent using either the native installation script or the automated Docker deployment. During installation, the deployment script verifies the execution environment and automatically installs the required dependencies whenever necessary.

After installation, the LAN Agent automatically establishes a Socket.IO connection with the central server and registers itself by transmitting its execution information, including its hostname, operating system, software version, IPv4 and IPv6 addresses, and current status. These data are stored by the central application and displayed in the management dashboard.

Each validation engineer selects the associated LAN Agent only during the first execution. The selected agent is then preserved throughout the user's session, allowing subsequent validation requests to be automatically routed to the appropriate workstation. If the selected agent becomes unavailable, the application immediately informs the user that the agent is offline and prevents the execution of validation operations.

Finally, the centralized deployment considerably simplifies software maintenance. Up-

dates to the Flask application are deployed only on the central server, allowing all engineers to benefit immediately from the latest version by simply refreshing their web browser. Likewise, whenever the network configuration changes, such as after a Home Gateway reboot or DHCP renewal, the LAN Agent automatically reports its updated network information to the central platform.

Overall, this deployment combines centralized supervision with distributed execution, preserving the network conditions required for PCP and UPnP validation while supporting collaborative use, traceability, and simplified maintenance.

Conclusion

This chapter presented the design of the proposed NetSupervision platform by describing its software architecture, the adopted architectural and technological choices, the database model, and the deployment architecture. Together, these design decisions establish a modular, scalable, and maintainable foundation for the implementation of the platform. The next chapter presents the implementation of the developed solution and describes its main functionalities together with the validation results.

Chapter 4

Implementation and Validation of the NetSupervision Platform

Introduction

In this chapter, we present the implementation and validation of the NetSupervision platform developed during the internship. We first introduce the experimental environment used to evaluate the proposed solution. Next, we describe the implementation of the main components of the platform and their integration within the overall system. Finally, we present the validation scenarios and discuss the obtained results to evaluate the effectiveness of the proposed solution.

4.1 Experimental and Validation Environment

During our internship, we worked in a laboratory validation environment designed to reproduce the interactions between an ISP infrastructure and subscriber HGWs. In a real deployment, HGWs installed at subscribers' premises communicate with network-side components such as CGNAT platforms and PCP servers. Before each firmware release, validation engineers execute predefined test scenarios to ensure that the Home Gateway behaves according to the expected specifications and to detect functional regressions before deployment to end users.

Rather than reproducing an entire production network, the objective of the laboratory was to emulate the essential ISP components in a controlled environment. The company's physical server represented the ISP-side infrastructure by hosting the PCP server together with the AFTR and NAT44 components, while the Home Gateway represented the subscriber network. This configuration allowed realistic validation of the communication between the customer premises equipment and the ISP infrastructure under repeatable testing conditions.

The main validation scenarios focused on the DS-Lite architecture, which represents the target deployment environment of this project. In this configuration, communication between the ISP infrastructure and the Home Gateway is based on IPv6, whereas end-user devices continue to operate over IPv4 within the local network. Since IPv4 traffic is transported through the ISP translation infrastructure, the PCP protocol is used to

dynamically create, update, and remove port mappings. To reproduce this behavior, the Home Gateway was configured in DS-Lite mode while the company server simulated the corresponding ISP-side services.

Because the tested Home Gateways do not expose a PCP server directly to devices on the LAN, validation relied on the PCP Interworking Function (IWF) implemented by the gateway. This mechanism automatically translates IPv4 UPnP port mapping requests generated by local devices into PCP requests exchanged with the IPv6 PCP server located in the ISP infrastructure, enabling transparent communication between IPv4 clients and the IPv6-based network.

4.1.1 Overall Validation Architecture

Figure 4.1 illustrates the overall validation architecture used throughout this work. Rather than describing each component independently, it presents how they collaborate during the execution of a typical validation scenario.

The validation process begins when the validation engineer launches a test from the NetSupervision web interface. The platform receives the request and forwards it in real time to the selected LAN Agent through Socket.IO. Running on the validation workstation located behind the Home Gateway, the LAN Agent executes the requested operation directly from the subscriber environment.

Depending on the selected test scenario, the LAN Agent may generate PCP requests, perform UPnP validation, communicate with the Home Gateway management interface, capture network packets, or collect diagnostic information. The Home Gateway then processes the request and communicates with the ISP-side infrastructure. In PCP validation scenarios, the request is forwarded directly to the PCP server. For UPnP validation, the Home Gateway first translates the UPnP request into a PCP message through its Interworking Function before forwarding it to the PCP server over the DS-Lite infrastructure.

Once the validation operation is completed, the response is returned to the LAN Agent together with any collected diagnostic information. The agent sends these results back to the NetSupervision platform, where they are analyzed, displayed to the engineer, and stored in the database to ensure complete traceability of the executed validation scenario.

4.1.2 Home Gateways Under Test

The validation activities were conducted using two Home Gateway models provided by Orange: the **Sagemcom F@st 5670 (MiB4)** and the **LB77 (Sagemcom F@st 5698)**. Using two different gateway generations allowed the proposed platform to be evaluated under different hardware and firmware environments and to verify that its operation does not depend on a specific Home Gateway implementation.

Both devices support the IPv6 and DS-Lite environments required for the PCP and UPnP validation scenarios presented in this chapter. The MiB4 was used as the main validation gateway, while the LB77 provided an additional implementation for distributed validation and comparison between gateway generations.

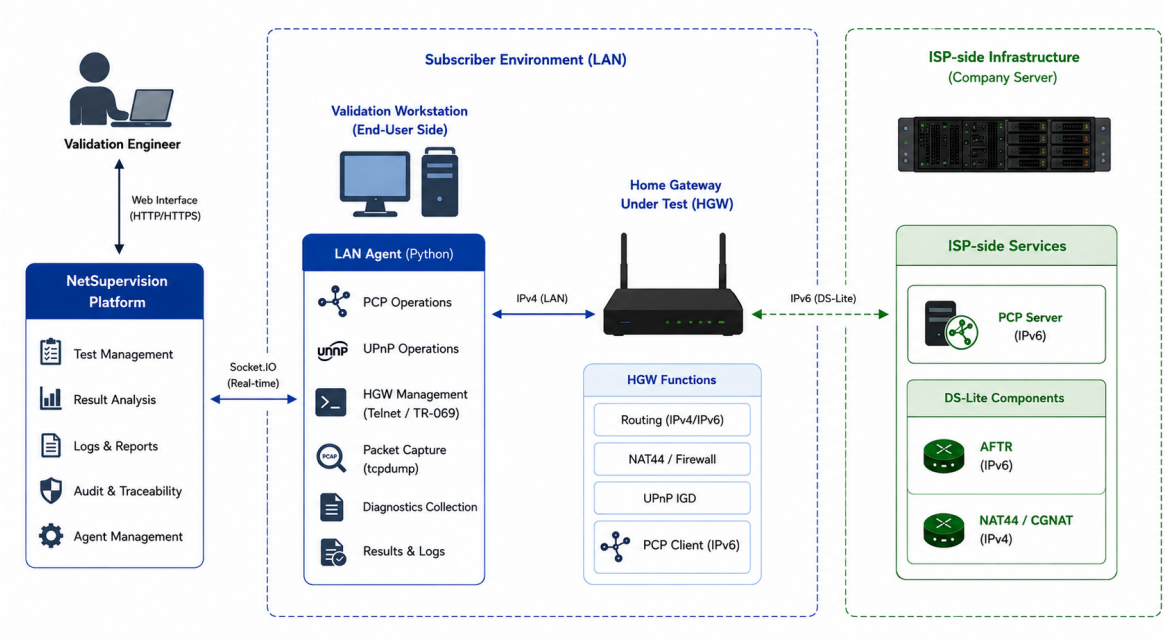


Figure 4.1: Overall validation architecture of the NetSupervision platform.

Detailed hardware specifications and illustrations of both Home Gateways are provided in Appendix A.

4.1.3 PCP Server

The validation environment relies on a PCP server derived from the open-source *PCP Test Server*, part of the *libpcpnatpmp* project developed by Cisco Systems [25]. The original implementation was mainly intended for basic protocol testing: it processed incoming requests and returned configurable responses without maintaining persistent mapping state. This was not sufficient to reproduce the behaviors required for advanced Home Gateway validation.

During this project, the server was therefore extended to support stateful mapping management, resource and quota control, external-port allocation, PCP option validation, request consistency checks, controlled failure injection, and extended execution logging. These extensions make it possible to reproduce both nominal and error scenarios under controlled conditions and to validate how the PCP client implemented in the Home Gateway reacts to network-side behavior.

In particular, the modified server can reproduce result codes such as `NO_RESOURCES`, `USER_EX_QUOTA`, `CANNOT_PROVIDE_EXTERNAL`, and `NETWORK_FAILURE`. This capability is especially important for validating error propagation through the Home Gateway without modifying its firmware. Detailed server extensions, supported validation conditions, and execution-log examples are provided in Appendix B.

4.1.4 Validation Network Topology

The validation testbed was deployed using the network topology illustrated in Figure 4.2. This topology reproduces the subscriber-side and ISP-side connectivity required to evaluate PCP and UPnP–PCP interworking under DS-Lite conditions.

On the subscriber side, the validation workstation is connected behind the Home Gateway under test. The LAN uses the private IPv4 network `192.168.1.0/24`, allowing the workstation to emulate an end-user device. The LAN Agent is executed on this workstation so that validation operations originate from the same network context as the subscriber.

The Home Gateway provides the boundary between the subscriber network and the ISP-side infrastructure. Its WAN interface is connected to the IPv6 access network used in the validation laboratory. As illustrated in Figure 4.2, the communication path traverses several intermediate routing nodes before reaching the company validation server hosting the PCP services.

On the ISP side, the modified PCP server and the AFTR are deployed on the same company validation server. The PCP service listens on UDP port 5351 over IPv6, while the AFTR performs the DS-Lite NAT44 translation required to provide IPv4 connectivity toward external networks. In the laboratory configuration, the NAT44 side of the AFTR uses the IPv4 address `10.255.25.34`.

During validation, PCP signaling is exchanged between the Home Gateway and the PCP server through the IPv6 network, while IPv4 user traffic is forwarded through the AFTR using the DS-Lite architecture. Since the PCP server is not directly accessible from the subscriber LAN, the Home Gateway relies on its UPnP IGD–PCP Interworking Function to translate UPnP port mapping requests received from IPv4 clients into PCP messages before forwarding them to the network-side PCP server.

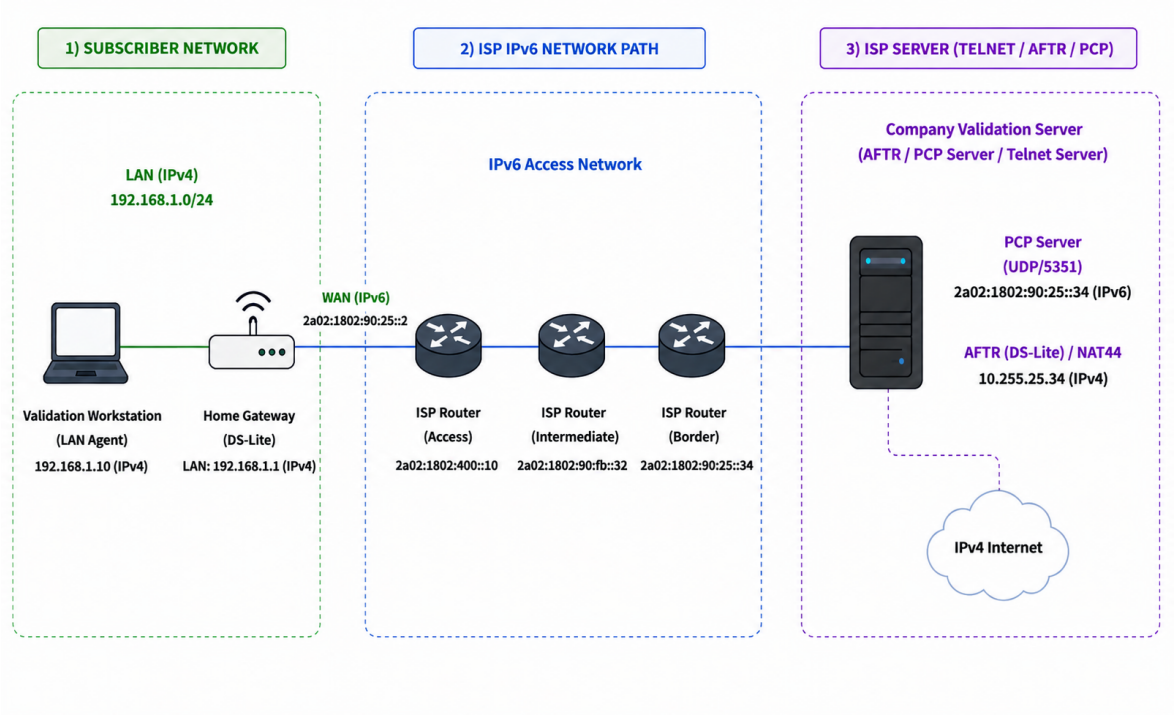


Figure 4.2: Network topology of the DS-Lite validation testbed.

4.1.5 Development Environment

The implementation and validation of the NetSupervision platform were carried out on a workstation running Ubuntu Linux. This environment was used for software

development, debugging, source code management, and deployment to the company's validation laboratory.

The following subsections summarize the hardware and software environment used throughout the internship.

4.1.5.1 Hardware Environment

The development activities were performed on a Lenovo V15-G1-IML laptop. This workstation was used for the implementation of the NetSupervision platform, source code editing, debugging, and interaction with the company's validation infrastructure.

Table 4.1 summarizes the main hardware characteristics of the development workstation.

Table 4.1: Development workstation specifications

Component	**Specification**
Computer	Lenovo V15-G1-IML
Processor	Intel® Core™ i5-10210U @ 1.60 GHz
CPU Cores / Threads	4 Cores / 8 Threads
Memory	20 GB RAM
Architecture	64-bit (x86_64)

4.1.5.2 Software Environment

The development environment was based on Ubuntu Linux and included the tools required for source code management, implementation, and deployment. Git was used for version control, while the validated source code was stored in the company's GitLab repository before being deployed to the validation server for testing.

Table 4.2 summarizes the main software tools used during the internship.

Table 4.2: Software environment used during the internship

Component	**Tool / Purpose**
Operating system	Ubuntu 24.04.4 LTS
Code editor	Visual Studio Code
Version control	Git
Source code repository	Company's GitLab
Modeling tool	StarUML (software architecture and UML diagrams)
Documentation	Overleaf (LaTeX report writing)
Terminal	Bash Shell
Deployment target	Company validation server

4.2 Platform Implementation

After presenting the experimental and development environments, we now describe the implementation of the NetSupervision platform. The following subsections present the

main modules developed during the internship and their role in the validation workflow.

4.2.1 Authentication Module

We implemented an authentication module to identify validation engineers, manage access to the NetSupervision platform and ensure the traceability of validation activities. Since the platform is intended for internal use within the company's validation laboratory, the authentication mechanism provides a simple and practical way to distinguish users and associate validation activities with the corresponding engineer.

When using the platform for the first time, a new engineer creates an account by providing a username, a password, and the validation team access code. As discussed in Chapter 3, the access code restricts the registration process to members of the validation team.

After registration, the engineer authenticates using the created credentials. Once the authentication succeeds, the platform creates a user session and redirects the engineer to the main dashboard, where the different validation modules become available.

The registration interface is presented in Appendix C.

4.2.2 Dashboard Module

After authentication, we redirect the validation engineer to the main dashboard, which provides a centralized overview of the validation environment and quick access to the different modules of the platform.

The dashboard summarizes the most important information required during daily validation activities. It displays global statistics such as the number of active PCP and UPnP mappings, recent validation events, and detected errors. It also continuously monitors the status of the main components involved in the validation environment, including the Flask backend, the Home Gateway, the PCP server, the AFTR, and the connected LAN Agents.

To facilitate troubleshooting, the dashboard provides a health summary of the selected Home Gateway by displaying information such as DS-Lite status, IPv4 and IPv6 connectivity, PCP server reachability, and WAN status. It also shows information about the currently selected LAN Agent, allowing the engineer to verify that validation requests will be executed from the correct workstation.

Finally, the dashboard offers direct access to the platform's principal functionalities, including Smart Analysis, validation reports, and the latest AI-assisted analysis results, allowing engineers to quickly navigate to the appropriate module without manually searching through the application.

Figure 4.3 presents the main dashboard of the NetSupervision platform.

4.2.3 LAN Agent

The LAN Agent is installed on the workstation connected to the Home Gateway under test. To simplify its deployment, we developed an installation page that generates a single deployment command. By executing this command, the workstation automatically

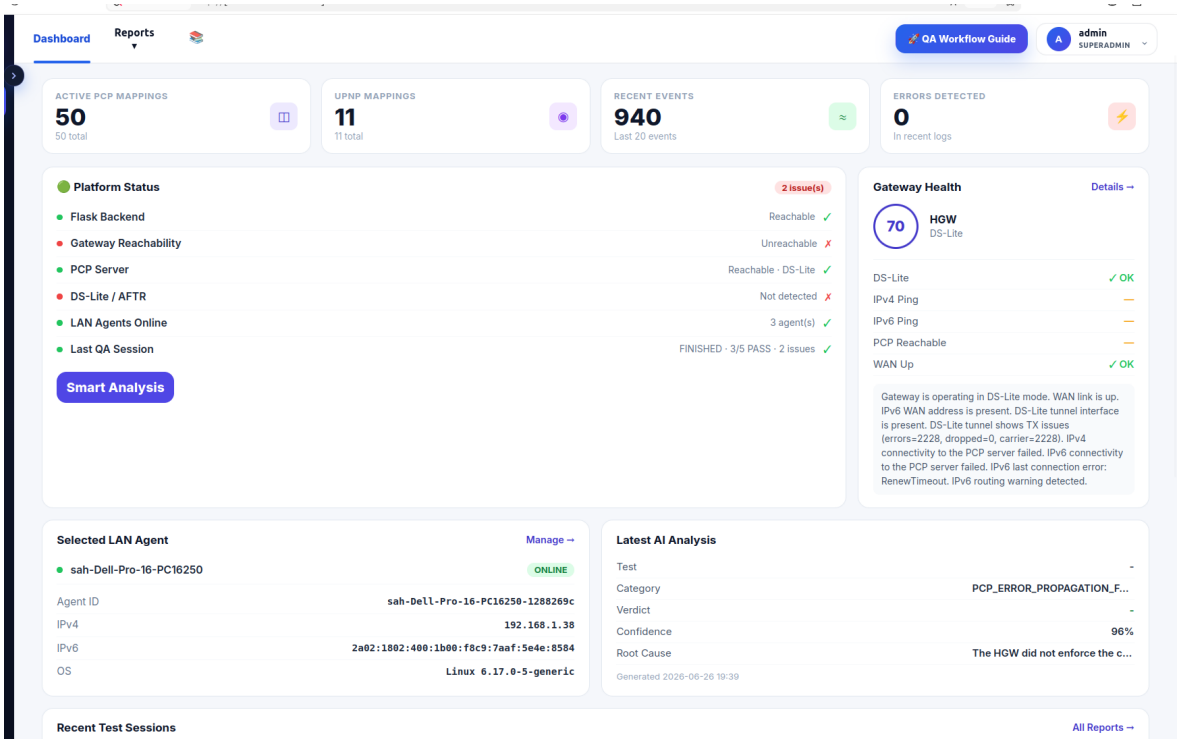


Figure 4.3: Main dashboard of the NetSupervision platform.

downloads the required files from the Flask server, installs the necessary dependencies, and configures the agent. The installer first attempts a native Python installation and automatically switches to a Docker-based deployment whenever the local environment is not compatible.

The LAN Agent installation interface is presented in Appendix C.1.

After the installation is completed, the LAN Agent starts automatically and establishes a persistent connection with the central platform. During its initialization, it registers the workstation by sending its hostname, operating system, IPv4 address, IPv6 address, and a unique Agent ID. This information allows the platform to identify the workstation and use it as the execution point for PCP and UPnP validation scenarios.

The successful startup and registration of the LAN Agent are illustrated in Appendix C.1.

Once connected, every available workstation appears in the *LAN Agents* page of the platform. Before executing any validation scenario, the engineer selects the workstation that will perform the tests. To preserve the physical validation environment, the platform verifies the identity of the current workstation and only allows the selection of the corresponding LAN Agent. For example, an engineer accessing the platform from PC1 can select only the agent installed on PC1, while an engineer working from PC2 can select only the agent associated with PC2. If an attempt is made to select an agent registered on another workstation, the operation is rejected and an access-denied message is displayed.

This verification prevents validation commands from being executed from an incorrect network context and ensures that PCP requests, UPnP validation, packet capture, and gateway management operations are always performed from the workstation physically connected to the Home Gateway under test.

Figure 4.4 presents the interface used to monitor the registered LAN Agents and select the workstation that will execute the validation scenarios.

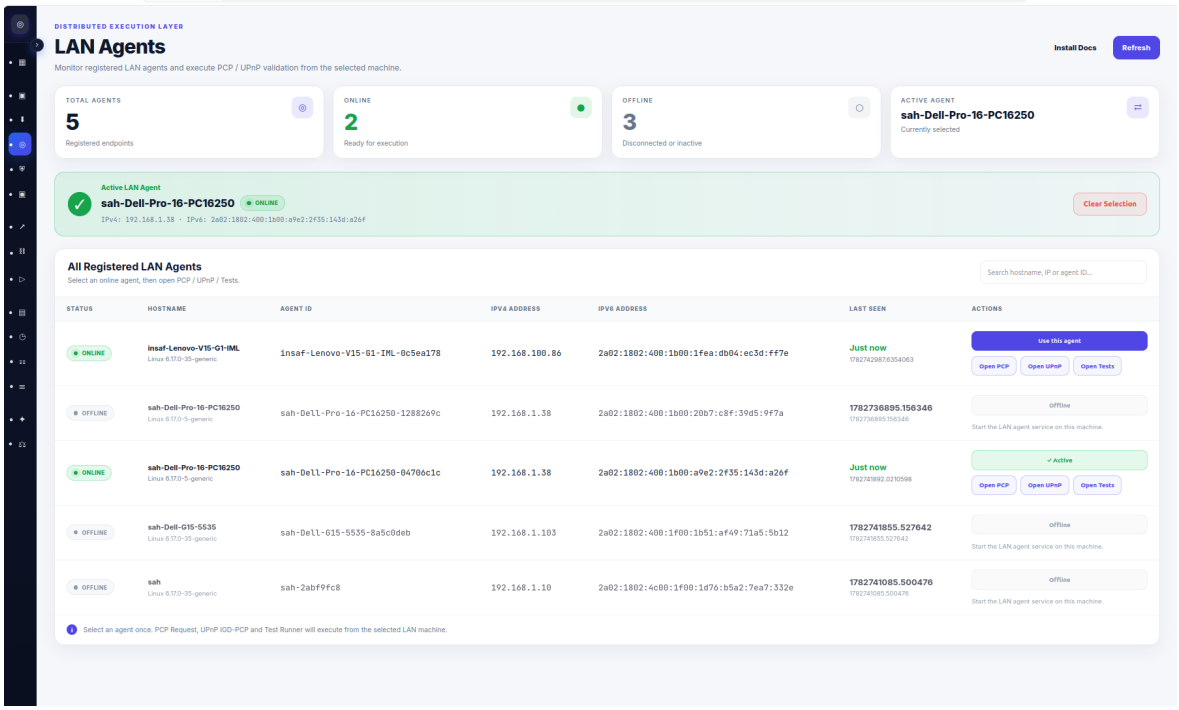


Figure 4.4: LAN Agent management and selection interface.

4.2.4 PCP Server Management Module

To simplify the execution of validation scenarios, the proposed platform integrates a dedicated interface for managing the PCP server used during testing. Instead of manually connecting to the validation server through SSH, QA engineers can control the PCP server directly from the web interface.

The interface displays both the current server status and its configuration, including the jump host, target host, listening IPv6 address, and executable path. This information enables engineers to verify that the validation environment is correctly configured before launching a test campaign.

Using the same interface, the engineer can start, stop, or refresh the PCP server with a single click. The platform continuously updates the execution state and displays operational information such as the process identifier (PID) and the listening status on UDP port 5351. These indicators allow the engineer to quickly confirm that the PCP server is running and ready to receive validation requests.

To facilitate troubleshooting, the platform also displays the execution logs generated by the PCP server in real time. The logs contain the received PCP requests together with the decoded protocol information, allowing QA engineers to verify that requests are correctly received and processed during validation. When required, the complete execution log can be downloaded directly from the interface for offline analysis or attached to validation reports.

Figure 4.5 presents the PCP Server Management interface implemented in the proposed platform.

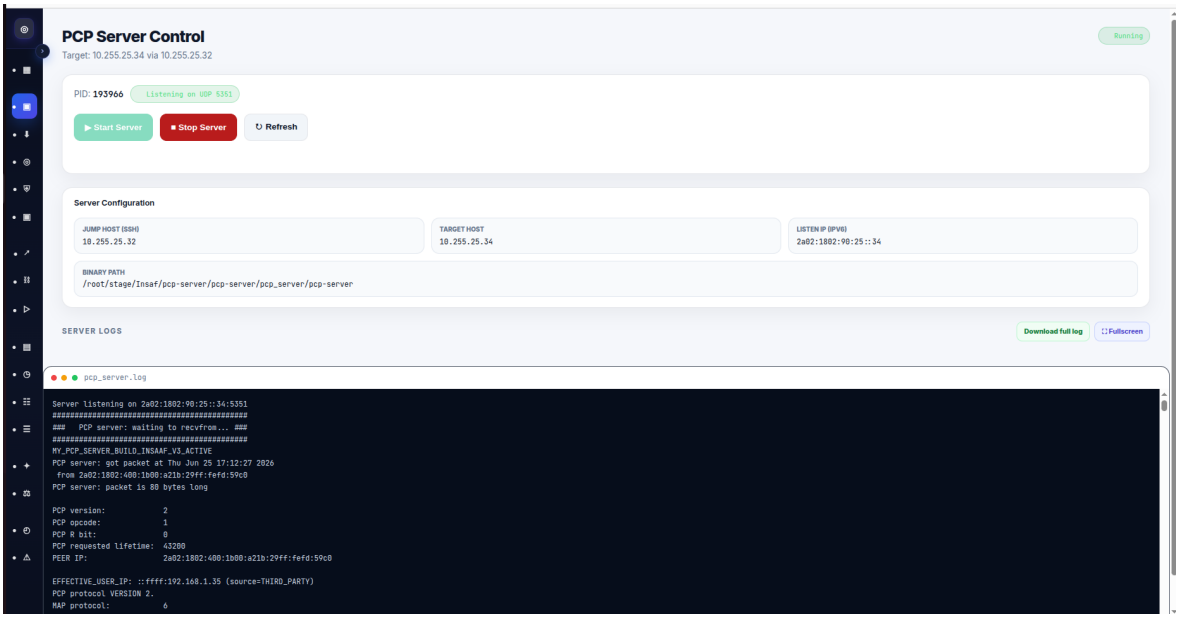


Figure 4.5: PCP Server Management interface showing server control, configuration, and real-time execution logs.

4.2.5 PCP Validation Module

The PCP Validation Module provides a graphical interface for executing PCP validation scenarios directly from the selected LAN Agent. The QA engineer configures the request parameters and sends the PCP request to the server through the IPv6 network in order to validate the Home Gateway firewall behavior, as described in Chapter 2. The platform then displays the server response together with the decoded PCP result.

In addition to the execution interface, all PCP operations are recorded in an execution log. This log allows QA engineers to identify each request, its associated response, and the returned PCP result code, simplifying validation and troubleshooting activities.

4.2.6 Packet Capture Module

The Packet Capture Module provides a real-time view of the PCP and UPnP traffic exchanged during validation sessions. Packet capture is performed using Scapy and is fully integrated into the validation platform, eliminating the need to launch an external packet analysis tool during most validation activities.

Captured packets are displayed through a graphical interface showing the source and destination addresses, protocol, opcode, timestamps, and returned result codes. Filtering options allow QA engineers to quickly locate specific requests or responses generated during a validation session.

In addition to live monitoring, captured traffic can be exported in PCAP format for offline analysis or long-term storage. The exported capture can be opened with tools such as Wireshark whenever a deeper protocol analysis is required.

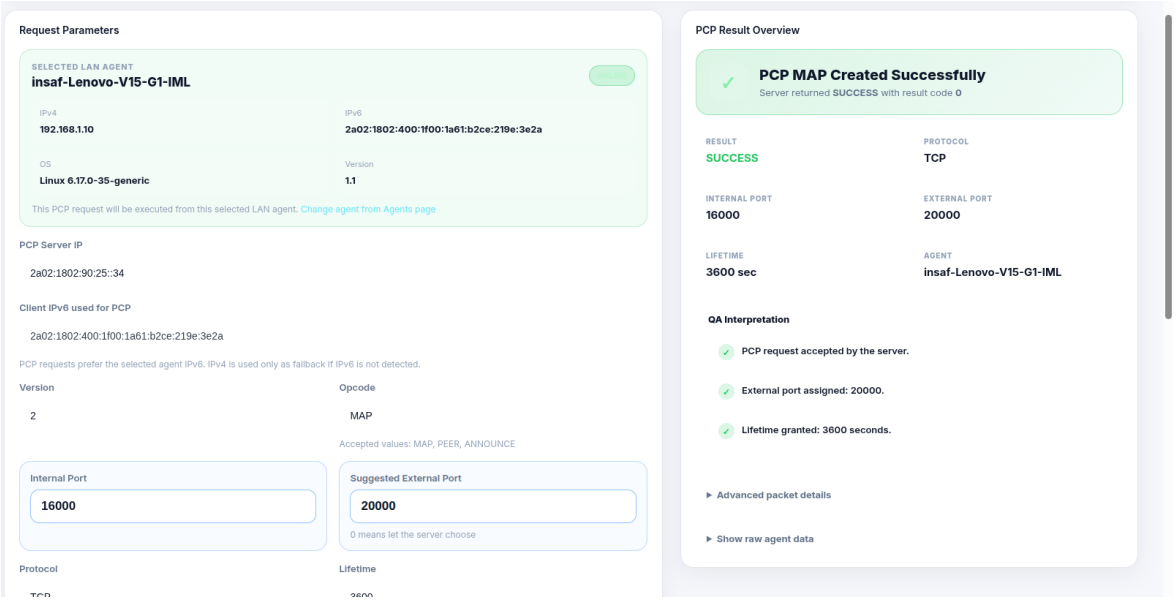


Figure 4.6: PCP request execution interface showing the parameters of a successful MAP request and the corresponding response returned by the PCP server.

By integrating packet capture directly into the platform, QA engineers can execute PCP validations, monitor network traffic, and analyze protocol exchanges from a single centralized environment.

Figure 4.7 presents the packet capture interface.

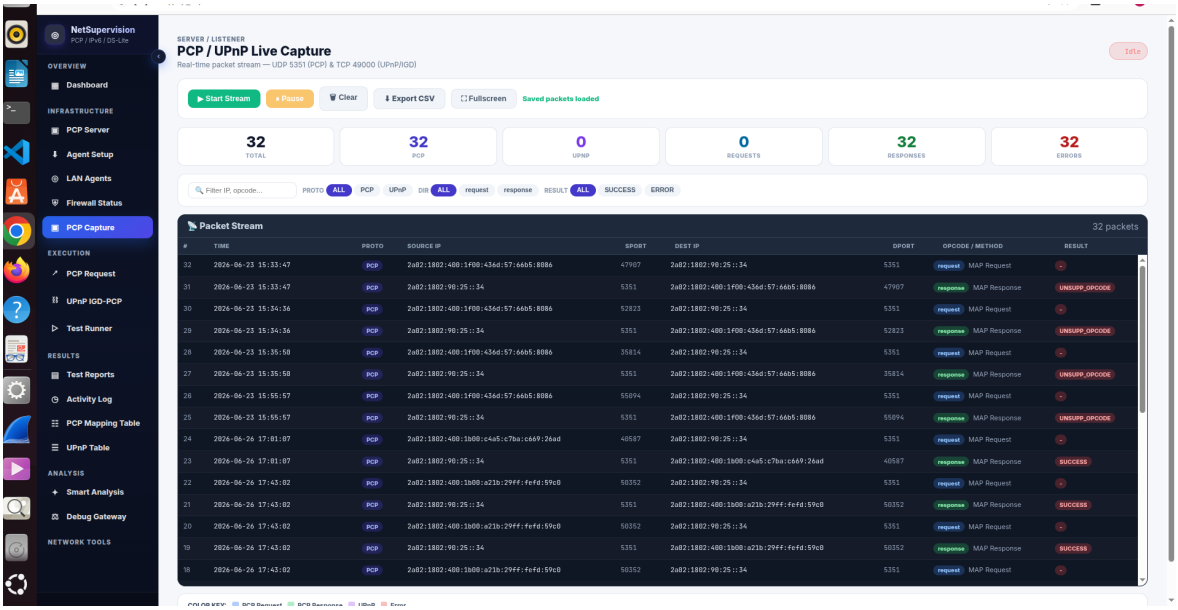


Figure 4.7: Integrated PCP/UPnP packet capture interface.

4.2.7 UPnP Validation Module

The UPnP Validation Module allows QA engineers to execute port-mapping requests from the selected LAN Agent toward the Home Gateway. The request is issued from an IPv4 client and translated by the gateway into a PCP operation through the IGD–PCP

interworking function. The detailed protocol operation has already been presented in Chapter 2; therefore, this section focuses on its implementation within the validation platform.

Figure 4.8 illustrates the validation interface. The left side allows the engineer to configure the mapping parameters, including the internal port, the requested external port, the transport protocol, and the lease duration. After the request is executed, the right side immediately displays the validation result, indicating whether the operation succeeded and whether the corresponding PCP mapping was successfully verified in the Home Gateway `Firewall.PCP` data model. This verification confirms that the gateway correctly translated the UPnP request into a PCP mapping instead of relying only on the UPnP response. Detailed UPnP execution logs are provided in Appendix C.2. The execution logs generated by the platform. Every request, response, verification step, and error is recorded with the associated parameters, making it possible to analyze the complete execution sequence and simplify debugging during validation campaigns.

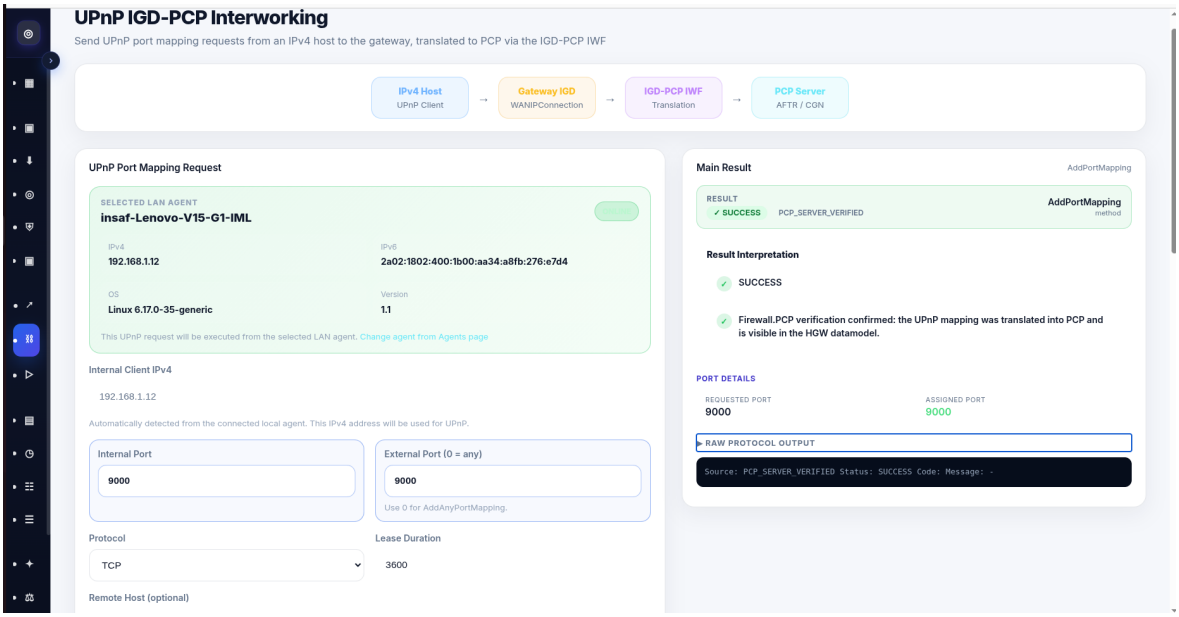


Figure 4.8: UPnP validation interface showing a successful port mapping and its verification through the `Firewall.PCP` data model.

4.2.8 Mapping Management Module

Besides executing PCP and UPnP requests, the platform provides dedicated interfaces for managing existing port mappings. These interfaces replace manual gateway CLI operations by allowing QA engineers to inspect, renew, verify, and delete mappings directly from the web application.

The PCP Mapping Table displays all mappings created through the platform together with their current state, remaining lifetime, expiration time, and renewal information. Active and expired mappings are clearly distinguished using color indicators, allowing the engineer to monitor the lifecycle of each mapping during validation. A mapping can also be renewed or deleted directly from the interface without interacting with the gateway command line.

To preserve execution isolation, each LAN Agent is authorized to manage only the mappings that it created. Consequently, a QA engineer cannot renew or delete mappings belonging to another validation workstation, ensuring that concurrent validation sessions remain independent.

Figure 4.9 illustrates the PCP mapping management interface.

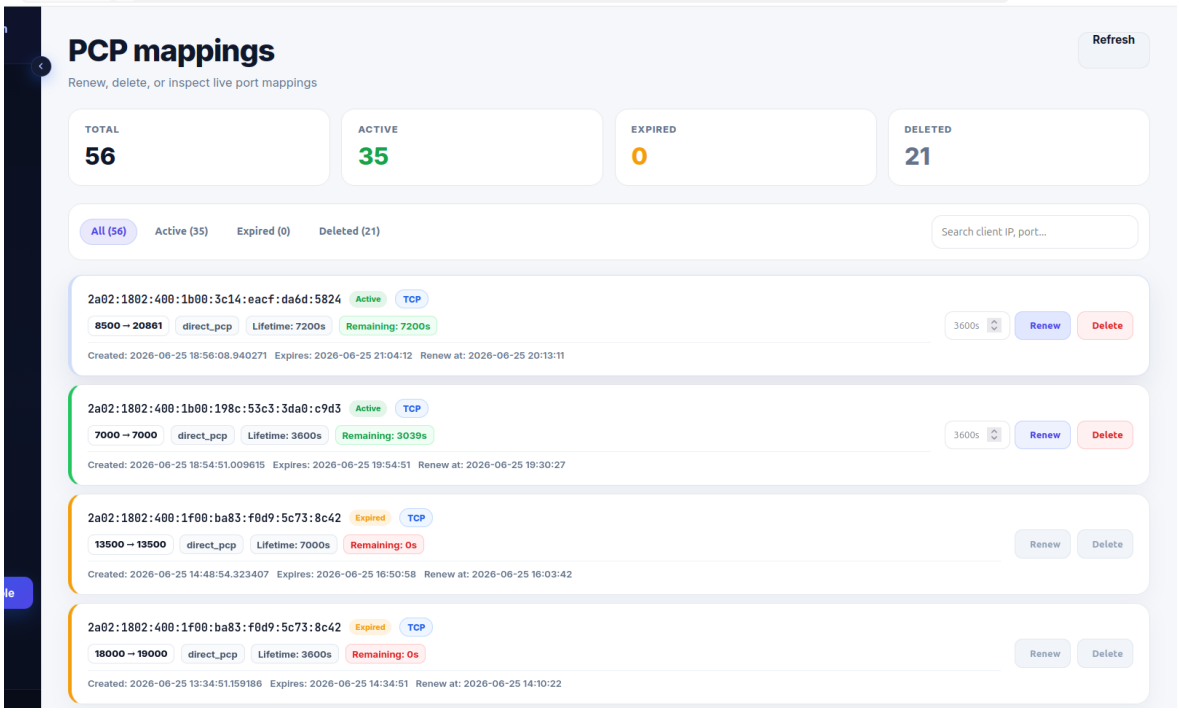


Figure 4.9: PCP Mapping Management interface.

The platform also provides a dedicated table for mappings created through the UPnP IGD–PCP interworking function. In addition to displaying the current mappings and their remaining lifetime, the interface allows QA engineers to verify the mapping against the PCP server, renew the lease, delete individual mappings, or remove a complete range of mappings when executing large validation campaigns.

As with PCP mappings, only the workstation that created a mapping is authorized to modify or remove it. This prevents one validation session from affecting another while maintaining a centralized view of all active UPnP mappings. The UPnP mapping management interface is presented in Appendix C.3.

4.2.9 Home Gateway Inspection Module

The Home Gateway Inspection module allows the QA engineer to inspect the data model of the Home Gateway after executing PCP or UPnP validation scenarios. Instead of relying on manual CLI commands, the platform retrieves the gateway configuration directly and displays all registered PCP mappings in a centralized interface.

Each mapping includes the internal client address, internal and external ports, protocol, lifetime, and the PCP status returned by the gateway. Status labels are displayed using color codes, allowing successful, pending, and failed mappings to be identified immediately. This enables the QA engineer to verify that the mapping created during

validation is actually present in the Home Gateway data model and to quickly detect gateway-side issues such as initialization failures or quota limitations.

Figure 4.10 illustrates the inspection interface of the `Firewall.PCP` data model.

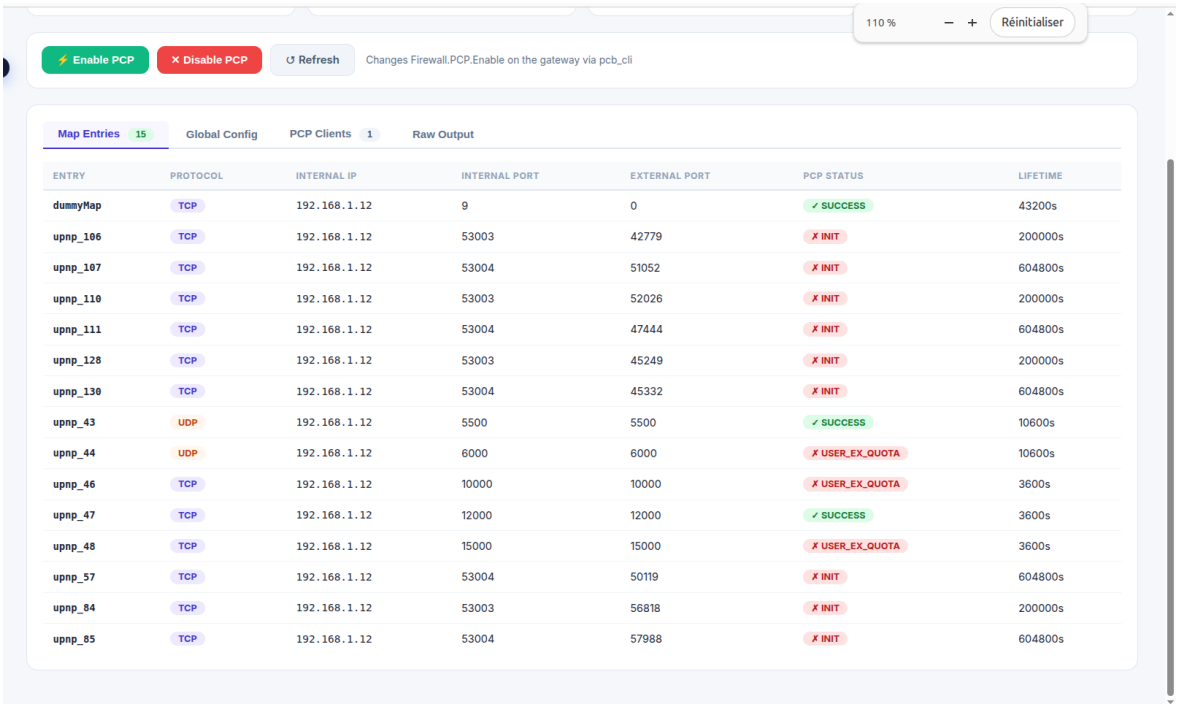


Figure 4.10: Inspection of the `Firewall.PCP` data model retrieved from the Home Gateway.

4.2.10 Regression Campaign Module

During Home Gateway validation, a corrected firmware version may be delivered after the development team resolves a previously reported problem. In this situation, repeating all validation operations manually is unnecessary and time-consuming. We therefore implemented a regression module that allows QA engineers to rerun predefined PCP and UPnP test cases and verify whether the corrected behavior now matches the expected result.

The platform provides a structured test library organized into validation categories. Each test case defines its preconditions, execution steps, protocol parameters, expected result, and validation criteria. The library contains scenarios related to PCP mapping creation, renewal and deletion, UPnP-to-PCP interworking, lifetime management, error propagation, retransmission, recovery, mapping conflicts, and robustness of the LAN Agent.

Figure 4.11 presents the campaign configuration interface. The engineer selects the LAN Agent, specifies the PCP server, chooses the required test suites, and starts the regression campaign from the same centralized interface.

Once the campaign is started, the selected test cases are executed automatically through the LAN Agent. For each scenario, the regression engine compares the expected behavior with the observed protocol result and assigns a deterministic verdict: `PASS`, `FAIL`,

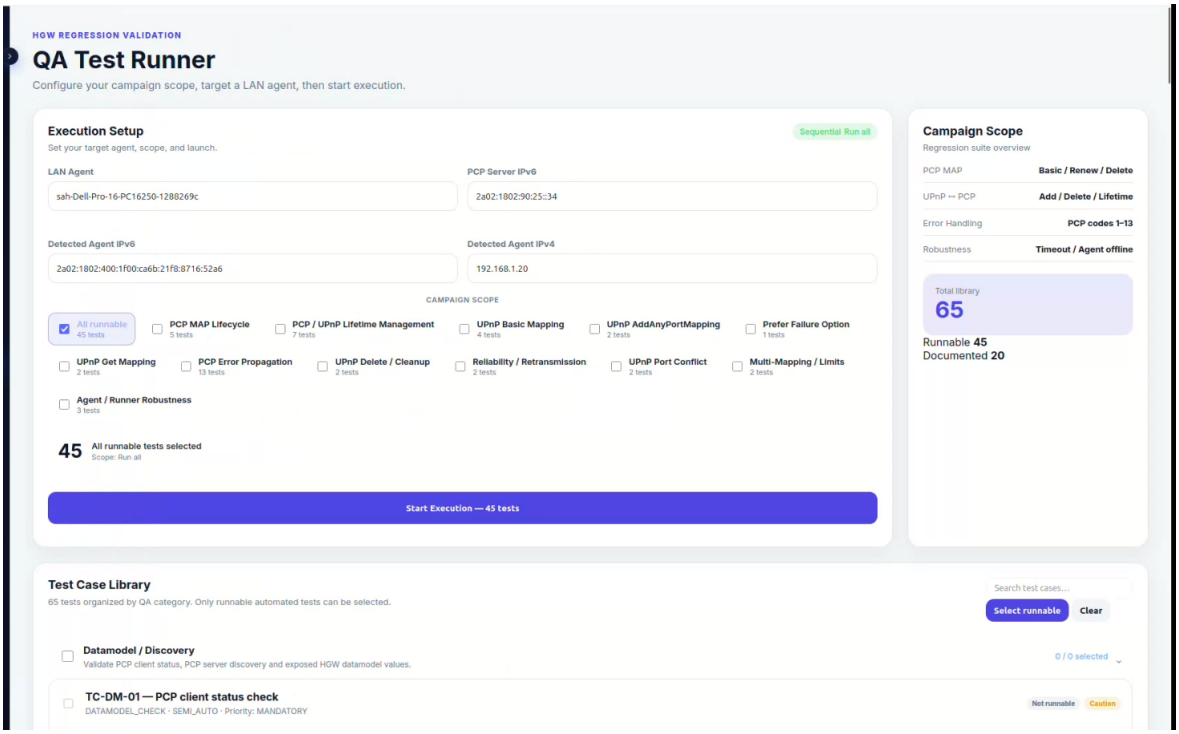


Figure 4.11: Configuration of an automated PCP and UPnP regression campaign.

or `ERROR`. An expected PCP error can therefore produce a `PASS` verdict when the same error is correctly returned and propagated by the Home Gateway.

Figure 4.12 shows the real-time execution dashboard. The interface displays the current progress, cumulative statistics, the running test case, the execution duration, and the verdict of each completed scenario. From this page, the engineer can inspect the corresponding logs, request an AI-assisted analysis for a complex result, or export the complete campaign report.

The execution details are preserved for every test case. They contain the submitted request, expected behavior, observed response, decoded protocol values, Home Gateway verification results, execution duration, and detected errors. These records provide the technical evidence required to investigate failed tests and preserve complete traceability between successive firmware validation campaigns.

Detailed execution logs generated by the regression module are provided in Appendix C.4.

Although these logs contain the information needed for troubleshooting, their manual interpretation can require detailed knowledge of PCP, UPnP, DS-Lite, and the Home Gateway data model. This limitation motivated the integration of an AI-assisted analysis mechanism, presented in the following subsection.

4.2.11 AI-Assisted Regression Analysis

During the validation of the regression module, we observed that detecting a failed test was not sufficient to immediately identify its root cause. A failure may originate from the Home Gateway, the PCP server, the LAN Agent, the network configuration, the `Firewall.PCP` data model, or the test environment itself. Manually correlating all

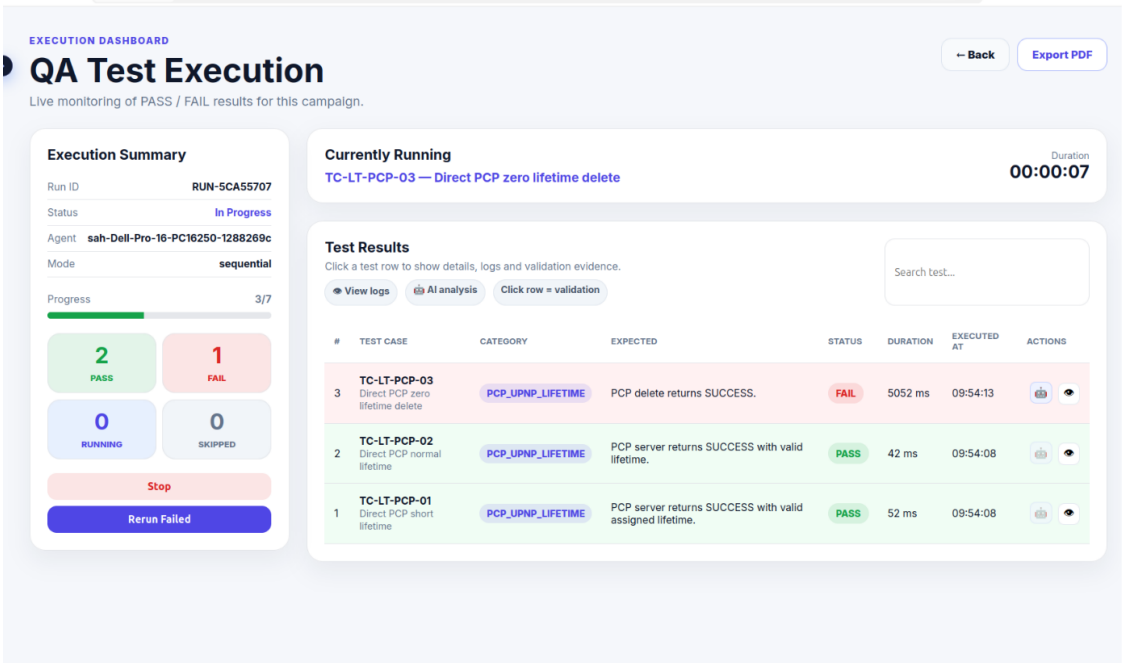


Figure 4.12: Real-time monitoring of an automated regression campaign.

execution evidence can therefore become difficult, particularly during large regression campaigns.

To simplify this investigation, we integrated an AI-assisted diagnostic pipeline into the regression workflow. The AI does not determine the test verdict: `PASS`, `FAIL`, and `ERROR` remain fully deterministic and are produced by comparing the expected and observed results. The AI is used only to interpret the collected evidence and identify the probable cause of complex or failed executions.

Before invoking the model, a rule-based classifier associates the result with a technical category such as PCP server unavailability, LAN Agent failure, network connectivity issues, `Firewall.PCP` synchronization problems, mapping conflicts, lifetime negotiation issues, or PCP error-propagation failures. The platform then constructs a structured context containing the test identity, verdict, expected and observed values, PCP/UPnP parameters, gateway status, server state, execution timing, and relevant evidence.

A sanitization stage removes sensitive laboratory information before the context is sent to the external AI service. Credentials, API keys, IP and MAC addresses, device identifiers, hostnames, and local paths are replaced by redaction markers, while oversized fields are truncated. The sanitized context is then inserted into a specialized prompt that instructs the model to rely only on the supplied evidence, avoid unsupported assumptions, and return a structured diagnosis.

The generated diagnosis includes the probable component, severity, confidence level, root cause, supporting evidence, recommended corrective actions, related standards, and an engineering summary. The analysis is stored together with the test identifier, model, prompt version, execution latency, user, and LAN Agent to preserve complete traceability.

Figure 4.13 presents the diagnosis displayed to the QA engineer after analyzing a

regression result.

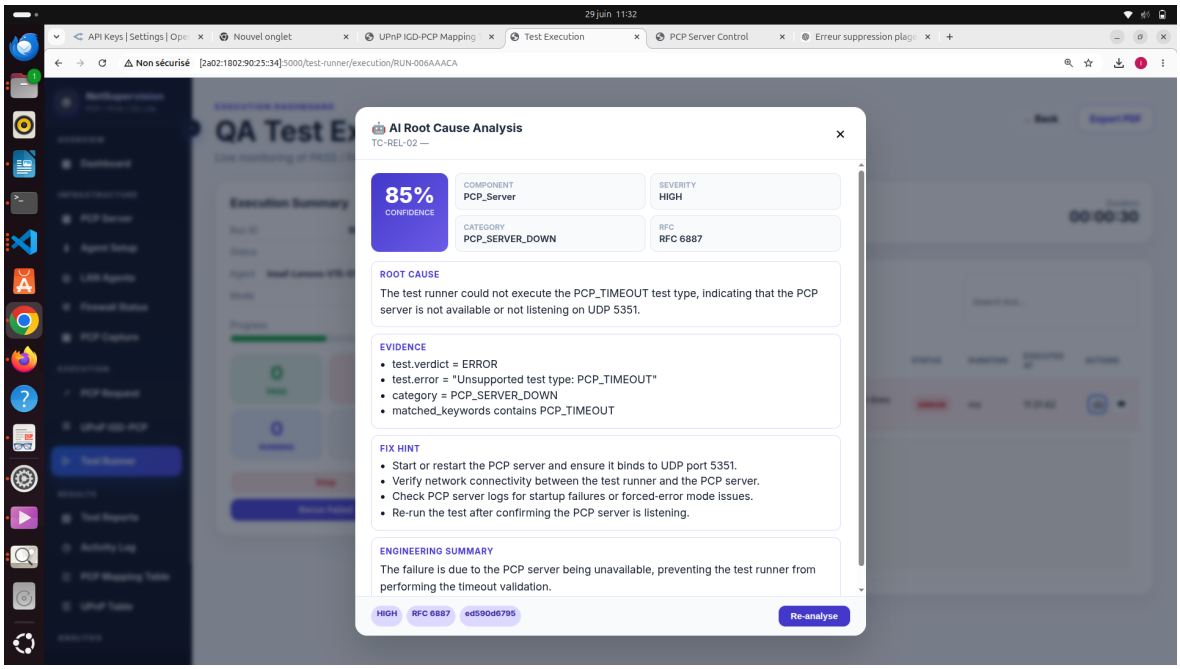


Figure 4.13: AI-assisted root-cause analysis of a regression result.

The interface clearly identifies the probable component and summarizes the technical evidence behind the diagnosis. It also provides recommended actions that help the engineer continue the investigation without replacing the final technical judgment of the validation team. Since the AI-generated interpretation remains advisory, an inaccurate or unsupported suggestion cannot modify the deterministic test verdict. The QA engineer can always verify the original expected and observed values, protocol results, gateway status, and execution evidence before accepting the proposed interpretation. This design limits the impact of potential LLM hallucinations on the validation decision.

4.2.12 Debug Analysis Module

As discussed in Chapter 2, investigating Home Gateway issues using debug files was previously a manual and time-consuming process. After generating a debug archive, QA engineers had to extract its contents, inspect numerous log files, and manually identify the information related to PCP, UPnP IGD-PCP interworking, and DS-Lite. Comparing two firmware versions further increased the complexity of this task.

To simplify this workflow, a dedicated Debug Analysis module was integrated into the proposed platform. The module automates the collection of debug archives, extracts the relevant debugging information, compares different gateway versions, and assists QA engineers in interpreting the detected differences. The complete workflow is presented in the following implementation stages.

4.2.12.1 Automatic Debug Collection

The first stage consists of collecting the debug archive directly from the Home Gateway. Instead of manually connecting to the gateway, generating the archive, configuring

a TFTP server, and retrieving the generated file, the entire procedure is executed automatically through the platform.

As shown in Figure 4.14, the QA engineer initiates the operation by selecting the *Generate Debug* action. The platform communicates with the selected LAN Agent, generates the debugging archive on the Home Gateway, transfers it automatically to the validation server, and stores it for future analysis. Previously collected archives remain available for download, deletion, or extraction.

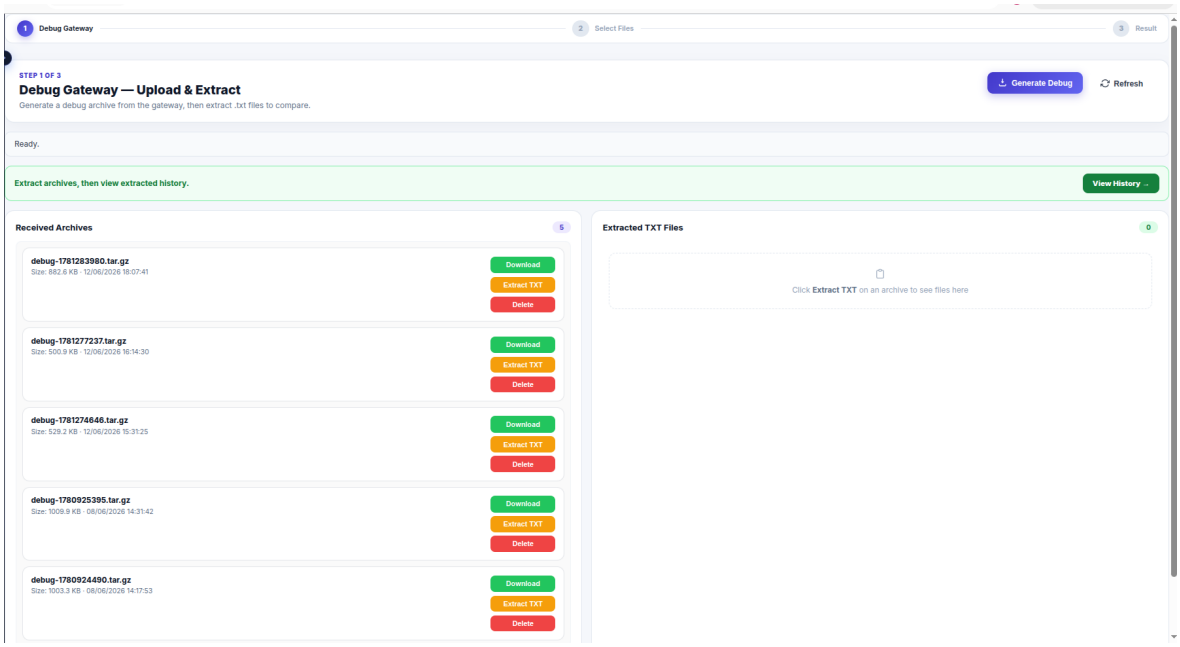


Figure 4.14: Automatic collection of Home Gateway debug archives.

Once the archive has been collected, the platform automatically extracts its contents and identifies the textual debug files required for protocol analysis. The extracted files are stored in a centralized repository, allowing them to be reused later without repeating the collection process.

The platform also maintains a history of previously extracted debug files, enabling QA engineers to reuse existing debug sessions when comparing different firmware versions or gateway configurations. The extraction and debug-file repository interfaces are presented in Appendix C.5.

4.2.12.2 Debug File Comparison

After the required debug files have been extracted, the QA engineer selects a baseline file and a target file for comparison. This mechanism can be used to compare different firmware versions of the same Home Gateway or to analyze differences between different gateway implementations.

Rather than comparing the complete debug files, the platform first performs a deterministic analysis using a Python parser. The parser extracts only the protocol information relevant to PCP, UPnP IGD-PCP interworking, and DS-Lite validation, including firmware metadata, Firewall.PCP parameters, PCP client status, mapping entries, lifetime configuration, UPnP services, DS-Lite information, and runtime events. This

preprocessing transforms large debug files into a structured representation containing only the information required for protocol validation.

The extracted information is transformed into a structured representation, allowing the comparison engine to automatically identify differences between the baseline and target files. These differences may concern protocol capabilities, configuration parameters, mapping information, lifetime policies, UPnP services, or runtime behavior.

To assist the QA engineer in interpreting these differences, the structured comparison can then be submitted to an AI-assisted analysis stage. The Large Language Model does not process the original debug archives; it receives only the structured information produced by the deterministic parser. Based on this information, the model summarizes the potential QA impact, identifies the affected protocol domains, and suggests additional validation actions when appropriate.

This design preserves a clear separation between deterministic extraction and AI-assisted interpretation: the Python parser remains responsible for identifying the technical differences, while the AI layer is used only to support their interpretation.

The complete comparison can finally be exported as a PDF report containing the structured differences and the associated analysis. This facilitates the documentation of investigations and the exchange of results between the QA and development teams.

A practical comparison between the MiB4 and LB77 Home Gateways using this workflow is presented in the Platform Validation section.

4.2.13 Monitoring and Traceability

The platform integrates monitoring and traceability mechanisms to support validation, troubleshooting, and collaborative use. Activity logs record PCP and UPnP validation events, audit logs preserve user and administrative actions, while LAN Agent logs provide information about distributed agent connections and execution status.

These mechanisms provide a centralized execution history and facilitate the analysis of validation sessions. Detailed monitoring and traceability interfaces are presented in Appendix C.6.

4.3 Platform Validation

The objective of this section is to validate the proposed platform by executing representative PCP and UPnP validation scenarios under realistic testing conditions. Rather than evaluating each module independently, the platform is assessed through complete validation workflows that reproduce the activities performed by QA engineers during Home Gateway verification. The following scenarios demonstrate the correct operation of the implemented platform and verify the interaction between its different modules throughout the validation process.

4.3.1 Representative Validation Scenarios and Results

The platform was evaluated through four representative scenarios covering local UPnP error handling, PCP server-side error propagation, distributed external-port conflict

detection, and debug comparison between two Home Gateway implementations. The experiments were performed using the MiB4 and LB77 Home Gateways together with the modified PCP server developed during this project.

4.3.1.1 Scenario 1: Local UPnP Mapping Conflict

This scenario verifies the ability of the MiB4 gateway to detect a mapping conflict locally. PC1 (`192.168.1.10`) and PC2 (`192.168.1.12`) were connected to the same MiB4 gateway. PC1 first created an `AddPortMapping` entry using external port `50009`. PC2 then requested the same external port from a different internal address.

The use of two workstations is important because a second request from the same client may be interpreted as a renewal or an update of the existing mapping. In this experiment, the MiB4 detected that port `50009` was already assigned to another LAN client and returned `ConflictInMappingEntry`, corresponding to IGD error code 718. The error was handled locally and did not require a PCP exchange with the server.

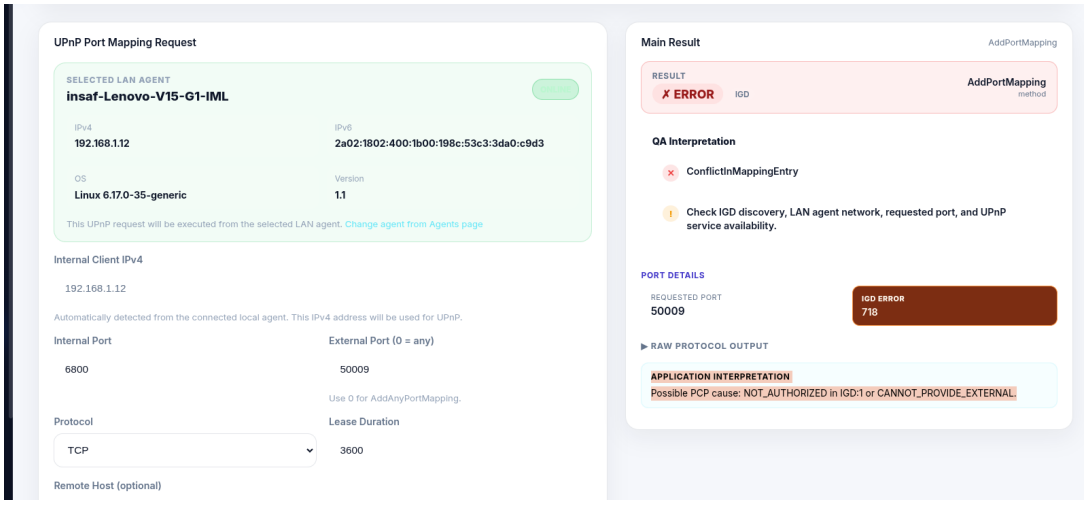


Figure 4.15: Local `ConflictInMappingEntry` error returned by the MiB4 gateway for a conflicting request from PC2.

This result confirms that the platform clearly identifies errors generated locally by the UPnP IGD implementation of the Home Gateway.

4.3.1.2 Scenario 2: PCP Resource Exhaustion Propagation

The second scenario validates the propagation of an error originating from the PCP server. The maximum number of active mappings accepted by the modified server was intentionally limited. After reaching this limit, an additional `AddPortMapping` request was sent from PC2 through the MiB4 gateway.

The PCP server rejected the new request with the `NO_RESOURCES` result code. The MiB4 received this response through the PCP interface and exposed the corresponding WAN-side error to the UPnP client. As shown in Figure 4.16, the platform identifies the PCP server as the source of the failure and displays both the original PCP result and its UPnP-side interpretation.

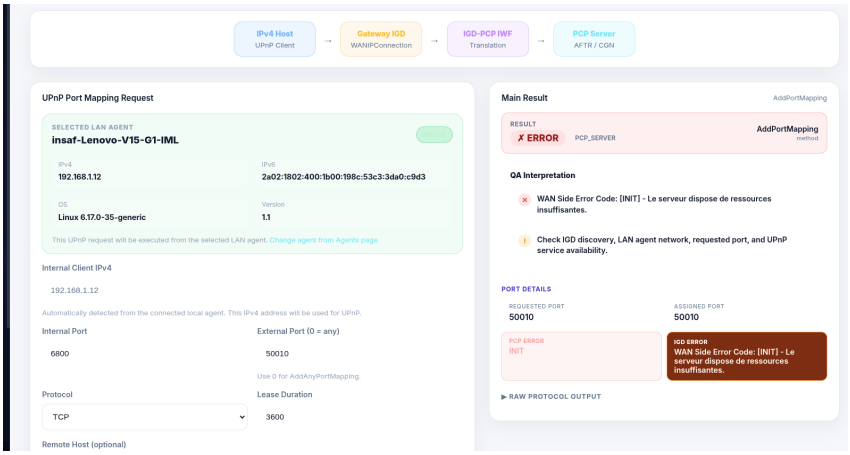


Figure 4.16: Propagation of the PCP `NO_RESOURCES` result through the MiB4 UPnP–PCP interworking function.

This result confirms that PCP errors are correctly forwarded by the gateway and presented with a clear origin and interpretation in the centralized platform.

4.3.1.3 Scenario 3: Distributed External-Port Conflict

The third scenario validates the distributed operation of two Home Gateways sharing the same PCP server. PC1 (`192.168.1.20`) was connected to the MiB4 gateway, while PC2 (`192.168.1.38`) was connected to the LB77 gateway. Although both LAN networks use the gateway address `192.168.1.1`, they represent two separate local network contexts.

PC1 first created a mapping through the MiB4 using a specific external port. PC2 then requested the same external port through the LB77. Since the port was already allocated in the centralized PCP mapping table, the server rejected the second request with `CANNOT_PROVIDE_EXTERNAL`. The LB77 forwarded the returned PCP error to its UPnP client, and the platform identified the server-side origin of the rejection.

This experiment demonstrates that the modified PCP server maintains a common mapping state across the MiB4 and LB77 gateways. It also confirms that the platform can validate distributed conflicts while preserving clear visibility of the requested port, the returned PCP result code, and the component responsible for the error.

4.3.1.4 Scenario 4: Debug Comparison Between MiB4 and LB77

The final validation scenario evaluates the ability of the Debug Analysis module to identify implementation differences between two Home Gateways. For this experiment, a debug file collected from the MiB4 gateway was compared with a debug file collected from the LB77 gateway. Both gateways were configured in the DS-Lite validation environment and connected to the same ISP-side infrastructure.

The MiB4 debug file, corresponding to software version `master_v13.4.12`, was selected as Debug A, while the LB77 debug file, corresponding to version `master_v13.4.16`, was selected as Debug B. Rather than comparing the complete raw files line by line, the platform first applied the deterministic Python parser described previously to extract the protocol information relevant to the validation process.

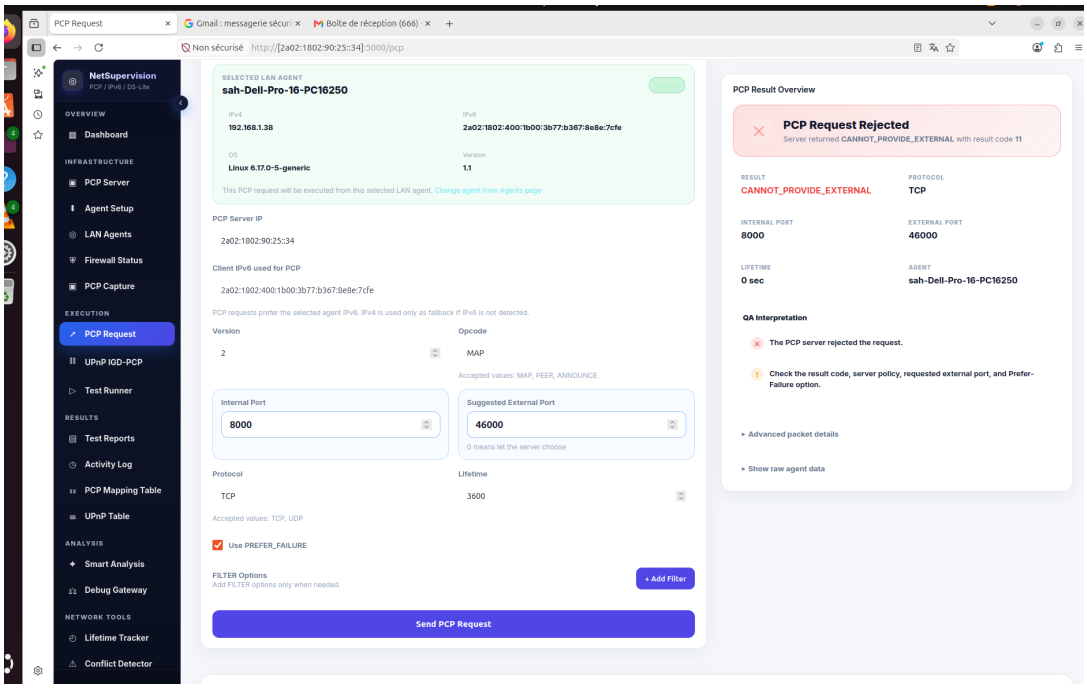


Figure 4.17: PCP server rejection of an unavailable external port during the distributed validation scenario.

The comparison identified 14 differences across several categories, including build information, PCP capabilities, lifetime configuration, UPnP services, and runtime information. Figure 4.18 shows part of the structured comparison generated by the platform.

The comparison revealed significant differences in the PCP implementation. On the MiB4, both `Device.PCP` and `Firewall.PCP` information were detected, with 24 PCP MAP entries and configured lifetime values between 43,200 and 86,400 seconds. On the LB77, `Firewall.PCP` information was present, while the `Device.PCP` client tree was not detected. No PCP MAP entries were found and the extracted minimum and maximum lifetime values were equal to zero.

Differences were also observed outside the PCP data model. The parser identified four UPnP services on the MiB4 compared with three on the LB77, while the runtime information contained four AppArmor denials on the LB77 and none on the MiB4.

After the deterministic comparison, the extracted differences were provided to the AI-assisted interpretation mechanism. The AI did not process the original debug archives; instead, it received the structured differences generated by the parser and evaluated their potential impact on subsequent validation activities.

Figure 4.19 presents the resulting QA-oriented interpretation.

The analysis highlights the absence of the PCP client tree and PCP MAP entries on the LB77 as the main differences affecting PCP validation. It also reports the change in lifetime information, the reduced number of discovered UPnP services, and the increase in AppArmor denials. These observations provide the QA engineer with a prioritized view of the differences that may require additional investigation or dedicated validation scenarios.

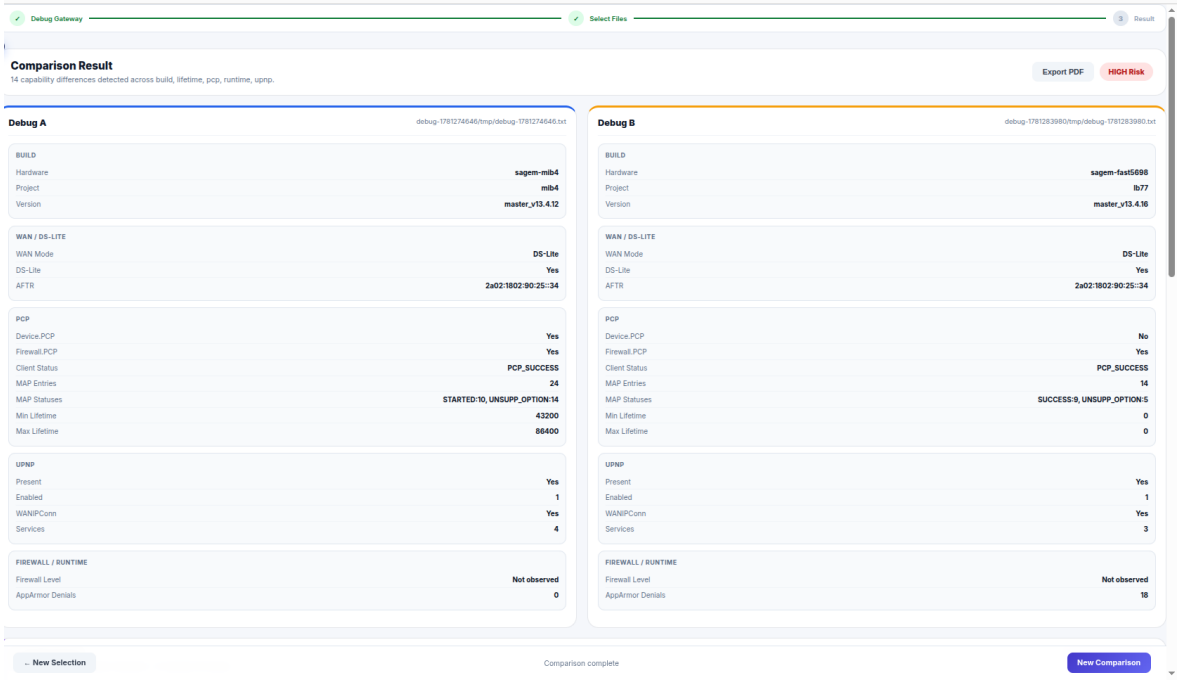


Figure 4.18: Deterministic debug comparison between the MiB4 and LB77 Home Gateways.

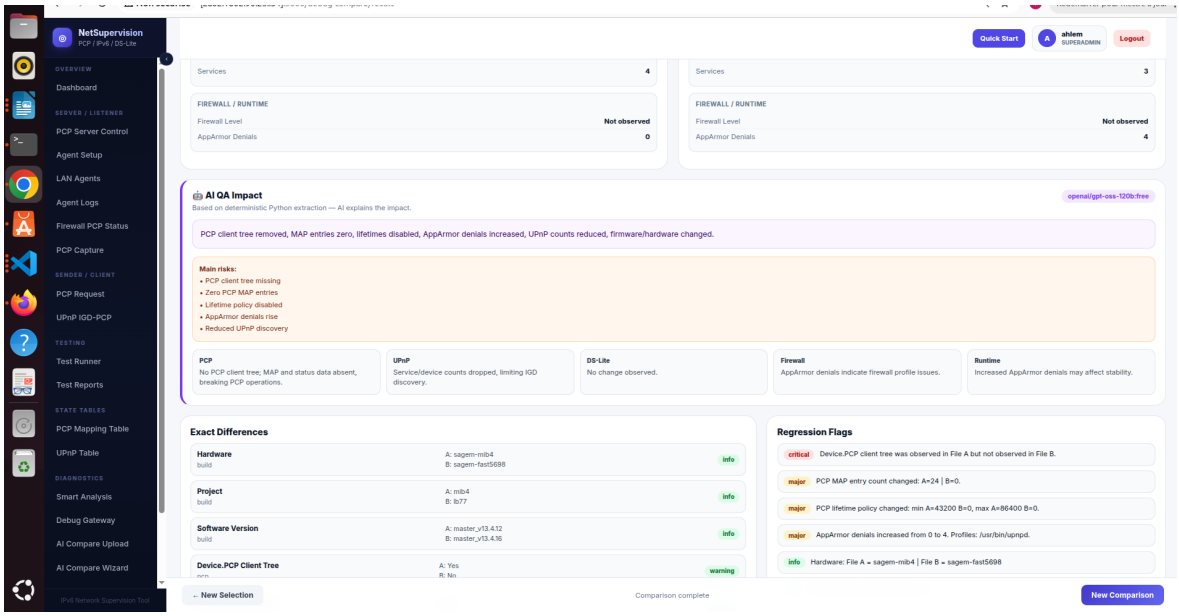


Figure 4.19: AI-assisted interpretation of the differences detected between the MiB4 and LB77 debug files.

This experiment confirms that the Debug Analysis module can transform two large Home Gateway debug files into a structured comparison and automatically highlight protocol-related differences. The deterministic parser preserves the technical evidence, while the AI-assisted layer provides an additional interpretation of their potential impact on the validation process.

4.4 Performance Evaluation

The previous sections demonstrated the implementation and functional validation of the proposed platform. This section evaluates its practical impact on the Home Gateway validation process. The evaluation focuses on workflow centralization, reduction of manual operations and configuration errors, execution-time improvement, and automation of repetitive QA activities.

4.4.1 Validation Process Improvement

Before the development of the platform, PCP and UPnP validation relied on several independent tools and manual operations. The QA engineer had to configure and start the PCP server, execute requests from dedicated clients, inspect packet exchanges using Wireshark or `tcpdump`, access the Home Gateway through Telnet, retrieve debugging files through TFTP, and manually organize the collected evidence.

The proposed platform centralizes these operations within a single environment. From the same interface, the engineer can select the appropriate LAN Agent, manage the PCP server, execute PCP and UPnP scenarios, inspect `Firewall.PCP`, capture packets, collect debug archives, run regression campaigns, request AI-assisted analysis, and retrieve generated reports. This reduces tool switching and provides clearer visibility of requests, responses, protocol errors, mappings, and execution history.

Table 4.3 summarizes the main improvements introduced by the platform.

4.4.2 Reduction of Configuration and Validation Errors

The platform also reduces errors caused by manual configuration and fragmented test environments. Three situations encountered during the project illustrate this contribution.

First, when the PCP server was not started before a manual test, the absence of a response did not clearly identify the source of the problem. The engineer could therefore spend time inspecting the client, packet capture, or Home Gateway before discovering that the server was not listening. The platform makes the PCP server state visible and allows it to be controlled directly from the interface.

Second, an `ADDRESS_MISMATCH` error was observed when PCP requests were initially executed from the company server while the PCP Client IP Address still referred to the validation workstation. Since the operating system selected the server address as the actual packet source, the addresses did not match. This experiment demonstrated that PCP and UPnP operations must be executed from the correct LAN context and motivated the distributed LAN Agent architecture.

Table 4.3: Comparison between the manual validation workflow and the proposed platform.

Activity	**Manual Workflow**	**Proposed Platform**
PCP server management	Manual remote access, configuration, and service execution	Centralized start, stop, status monitoring, and configurable error injection
PCP and UPnP execution	Separate command-line clients and manual parameter preparation	Unified interfaces with structured parameters and decoded results
Protocol observation	External Wireshark or `tcpdump` session	Integrated packet capture with PCAP export
Home Gateway verification	Manual Telnet access and data-model inspection	Automatic inspection of gateway and `Firewall.PCP` information
Debug collection	Manual Telnet, TFTP configuration, archive transfer, and extraction	Automated generation, transfer, integrity verification, and extraction
Failure analysis	Manual reading of protocol and debug logs	Structured evidence with AI-assisted interpretation
Traceability	Evidence distributed across terminals, files, and screenshots	Centralized activity logs, audit logs, histories, and reports

Finally, an incompatible MiniUPnPc client initially prevented correct discovery of the UPnP IGD service, producing misleading gateway-not-found results. The standardized agent installation and Docker fallback provide a controlled execution environment and reduce dependency inconsistencies between validation workstations.

These examples show that the platform improves not only execution speed but also the reliability and reproducibility of the validation process by standardizing configuration, execution, interpretation, and evidence collection.

4.4.3 Time Saving Evaluation

The execution time was evaluated for representative PCP, UPnP, regression, and debugging activities. The manual values correspond to the complete QA workflow, including environment preparation, protocol execution, gateway verification, packet inspection, and result interpretation. The platform values correspond to the centralized and automated workflow.

Table 4.4 summarizes the representative execution times observed during the validation activities.

The comparison shows a substantial reduction in the time required to perform representative validation activities. A standard PCP validation scenario is reduced from approximately five minutes to five seconds, while a UPnP validation scenario that previously required approximately eight to nine minutes can be completed in approximately eight to ten seconds.

Table 4.4: Execution-time comparison between manual validation and NetSupervision.

Validation Task	**Manual**	**NetSupervision**	**Reduction**
PCP validation (single test)	≈ 5 min	≈ 5 s	98.3%
UPnP validation (single test)	≈ 8–9 min	≈ 8–10 s	≈ 98%
13 PCP error scenarios	≈ 6 h 30 min	≈ 7 min	98.2%
Debug analysis and comparison	≈ 15 min	≈ 5 s	99.4%

The largest practical gain concerns PCP error validation. In the previous workflow, testing a single PCP result code could require approximately 30 minutes because the engineer had to block the operational PCP server, prepare or modify a dedicated script, restart the test environment, execute the request, and verify the resulting behavior. Repeating this procedure for 13 PCP result codes therefore required approximately six hours and thirty minutes. With the modified PCP server and the automated regression workflow, the same set of error scenarios can be executed in approximately seven minutes.

The percentage reduction is calculated using:

$$\text{Time Reduction} = \frac{T_{\text{manual}} - T_{\text{platform}}}{T_{\text{manual}}} \times 100 \tag{4.4.1}$$

where T_{manual} represents the duration of the previous workflow and T_{platform} represents the duration observed using NetSupervision.

Figure 4.20 provides a graphical comparison of these measurements. A logarithmic time scale is used because the evaluated durations range from a few seconds to several hours.

It is important to note that these reductions do not result from faster protocol transmission. PCP messages themselves are exchanged within a very short period in both approaches. The main improvement results from automating the operations surrounding the protocol exchange, including environment preparation, server management, request generation, gateway verification, result interpretation, and evidence collection.

4.4.4 Regression Execution Performance

In addition to the comparison with the previous manual workflow, the execution time recorded by the regression engine was examined for the lifetime validation category. This category contains three direct PCP scenarios and four UPnP-to-PCP scenarios.

For the three direct PCP lifetime tests, the recorded execution times were 20 ms, 23 ms, and 22 ms, corresponding to an average of approximately 21.7 ms per test. In comparison, the UPnP lifetime scenarios required approximately 11–12 s per test.

The difference is explained by the validation path followed by each scenario. A direct PCP test interacts directly with the PCP validation path, whereas an UPnP scenario requires UPnP IGD processing by the Home Gateway and the corresponding UPnP-to-PCP interworking operations before the final result can be verified.

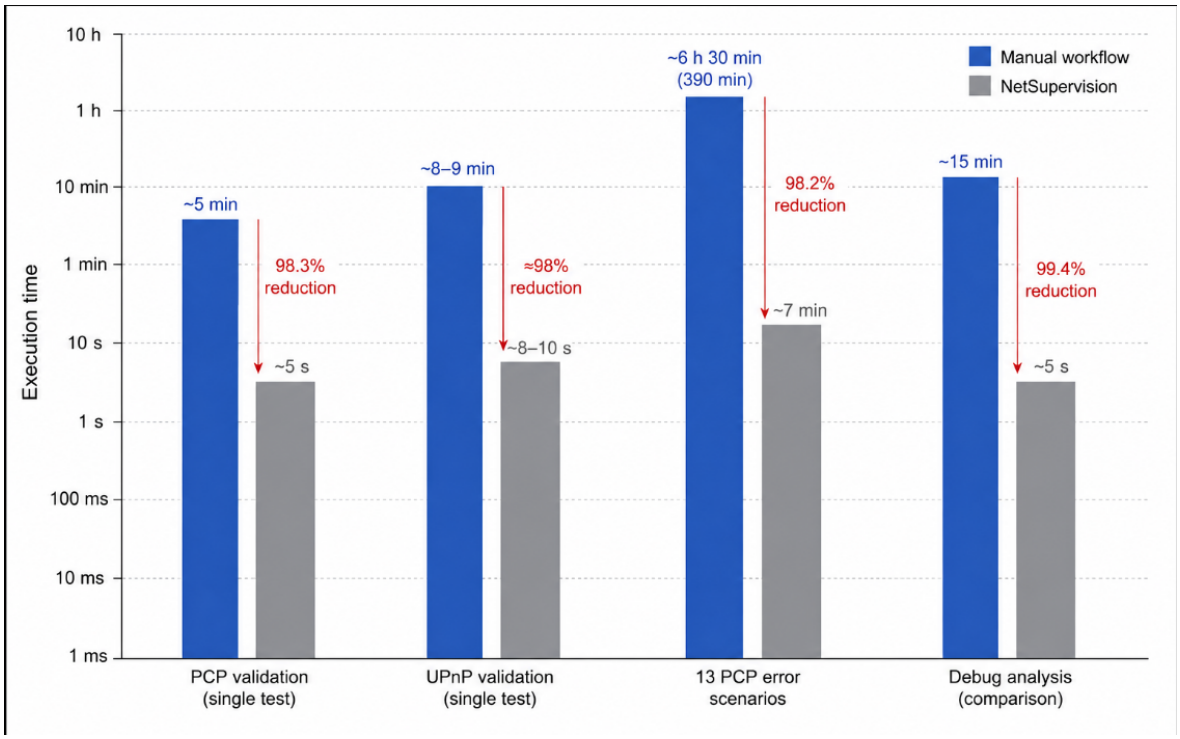


Figure 4.20: Execution-time reduction achieved by NetSupervision compared with the manual validation workflow.

Consequently, the complete lifetime category, composed of seven predefined test cases, can be executed automatically in approximately 45–50 seconds. These measurements demonstrate that the regression engine can execute heterogeneous protocol scenarios within the same campaign while preserving an individual verdict and execution trace for each test case.

Overall, the performance evaluation demonstrates that NetSupervision significantly improves the Home Gateway validation process by centralizing validation activities, automating repetitive tasks, and reducing execution time. These improvements increase the reproducibility and traceability of validation campaigns while providing QA engineers with a unified environment for PCP and UPnP testing.

Conclusion

In this chapter, we presented the implementation and validation of the NetSupervision platform, from the experimental environment to the main developed modules and their evaluation through representative PCP and UPnP validation scenarios. The obtained results demonstrated the ability of the proposed solution to centralize and automate the validation workflow while improving traceability and reducing repetitive manual operations. The following General Conclusion summarizes the main contributions of the project and presents possible future improvements.

General Conclusion

The transition toward IPv6 has considerably increased the complexity of Home Gateway validation. In IPv6 and DS-Lite environments, validating protocols such as PCP and UPnP requires engineers to analyze interactions between several network components while using multiple independent tools. This fragmented workflow makes validation, troubleshooting, and regression testing time-consuming and difficult to reproduce. The objective of this internship was therefore to design and develop a centralized supervision platform capable of simplifying these activities while improving the efficiency and traceability of the validation process.

To achieve this objective, NetSupervision was designed and implemented as a web-based platform dedicated to PCP and UPnP validation. The developed solution centralizes protocol request execution, Home Gateway inspection, packet capture, mapping management, debugging, activity monitoring, and automated regression testing. Distributed LAN Agents also allow validation tasks to be executed from the appropriate network environment while maintaining centralized control and traceability.

The platform was validated using representative PCP and UPnP scenarios on real Home Gateway equipment in an IPv6 and DS-Lite laboratory environment. The results confirmed the correct operation of the implemented functionalities and demonstrated the ability of the platform to automate complex validation procedures while preserving visibility of protocol behavior. Compared with the previous manual workflow, operations requiring several minutes can now be completed in a few seconds, while regression campaigns that previously required several hours can be executed within a few minutes. The platform also improves reproducibility, reduces manual errors, and allows QA engineers to focus on protocol analysis rather than repetitive tasks.

Several opportunities remain for extending the platform. The regression engine could include additional PCP and UPnP scenarios, customizable validation suites, scheduled campaigns, and automated firmware comparisons. The distributed architecture could also support larger infrastructures involving multiple Home Gateways and LAN Agents. More advanced AI-assisted debugging could correlate packet captures, gateway data models, execution logs, and debug files, while CI/CD integration could enable automatic regression campaigns for new firmware versions.

In conclusion, this project demonstrates that centralizing and automating PCP and UPnP validation can significantly improve the efficiency, reproducibility, and traceability of Home Gateway testing. NetSupervision provides a solid and extensible foundation for future validation platforms supporting increasingly complex broadband networks.

Appendix A

Home Gateway Technical Details

This appendix provides additional technical information about the Home Gateways used during the validation of the NetSupervision platform. These specifications are complementary to the validation environment presented in Section 4.1.2.

A.1 Sagemcom F@st 5670 (MiB4)

The first Home Gateway used during the validation process was the **Sagemcom F@st 5670 (MiB4)**. It supports xDSL and GPON access technologies, IPv4 and IPv6 connectivity, NAT, firewall services, and residential broadband functions.

The main technical characteristics are summarized in Table A.1 and are based on the official manufacturer documentation [26].

Table A.1: Technical specifications of the Sagemcom F@st 5670

Manufacturer	Sagemcom
Commercial model	F@st 5670
Internal platform	MiB4
Access technologies	VDSL2 and GPON Fiber
Wi-Fi standard	Wi-Fi 6 (IEEE 802.11ax)
Frequency bands	2.4 GHz and 5 GHz
Ethernet ports	4 x Gigabit Ethernet
USB ports	1 x USB 3.0
IPv6 support	Yes
Operating mode	Residential Home Gateway

Figure A.1 shows the device used during validation together with its rear interface panel.

(a) Front view with identification label

(b) Rear interface panel

Figure A.1: Sagemcom F@st 5670 Home Gateway used during validation.

A.2 LB77 (Sagemcom F@st 5698)

The second Home Gateway used during the validation process was the **LB77**, based on the **Sagemcom F@st 5698** platform. It represents a more recent gateway generation and was used to evaluate NetSupervision with a different hardware and firmware implementation.

The debug information collected from the device identifies the hardware platform as `sagem-fast5698` and the project identifier as `lb77`. Its main technical characteristics are summarized in Table A.2.

Table A.2: Technical specifications of the LB77 (Sagemcom F@st 5698)

Manufacturer	Sagemcom
Commercial name	Orange LB77
Hardware platform	Sagemcom F@st 5698
Access technologies	GPON Fiber
Wi-Fi standard	Wi-Fi 7 (IEEE 802.11be)
Ethernet ports	4 x Gigabit Ethernet + 1 x 10G Ethernet
Telephone ports	2 x RJ11
USB ports	1 x USB
Power supply	12V, 3.0A
IPv6 support	Yes
Operating mode	Residential Home Gateway

Figure A.2 shows the LB77 equipment used during validation.

(a) Full device view (b) Front status panel

Figure A.2: LB77 Home Gateway used during validation.

Although the two Home Gateways provide similar network functions, their hardware platforms and firmware implementations differ. Their use during the project therefore provided complementary validation environments for verifying the portability of NetSupervision and comparing PCP and UPnP behavior across different gateway generations.

Appendix B

Modified PCP Server Details

This appendix provides additional implementation details about the modified PCP server used during the validation of the NetSupervision platform. The server was extended from the original *PCP Test Server* in order to reproduce the stateful and error-handling behaviors required for Home Gateway validation.

B.1 Implemented Extensions

The original implementation receives PCP requests and returns configurable responses without maintaining persistent mapping state. During this project, several extensions were introduced to make the server suitable for realistic validation scenarios.

1. **Persistent mapping table.** An internal mapping table was implemented to store mappings created through PCP MAP requests. Each entry preserves the subscriber IPv6 address, transport protocol, internal and external ports, lifetime, and associated FILTER information. This allows mappings to be created, renewed, expired, and deleted while maintaining their state across successive requests.
2. **Resource management.** Resource-control mechanisms were introduced to simulate limitations that may occur in network-side NAT infrastructures. The server can limit the total number of active mappings and the number of mappings associated with a subscriber. When configured limits are reached, result codes such as `USER_EX_QUOTA` and `NO_RESOURCES` can be returned.
3. **External-port allocation.** The server verifies the availability of requested external ports and detects conflicts with existing mappings. When permitted, an alternative port can be assigned. When strict preservation is requested through `PREFER_FAILURE`, the server can instead return `CANNOT_PROVIDE_EXTERNAL`.
4. **PCP option processing.** Processing of PCP options was extended to validate supported options such as `THIRD_PARTY`, `FILTER`, and `PREFER_FAILURE`. Invalid lengths, duplicate options, unsupported combinations, and malformed requests can therefore be detected during validation.
5. **Request consistency checks.** Additional checks verify the consistency of received requests. In particular, the PCP Client IP Address contained in the request

can be compared with the actual source address of the packet, while mapping renewal conditions and other protocol constraints are also validated.

6. **Failure-injection mechanisms.** The modified server provides mechanisms for forcing specific PCP result codes and reproducing controlled network-side failures. This includes WAN unavailability, network failures, resource exhaustion, and unsupported option conditions. These mechanisms allow error scenarios to be reproduced without modifying the Home Gateway firmware.
7. **Extended logging.** The logging mechanism records mapping creation, renewal, deletion, resource allocation decisions, option processing, returned errors, and the internal mapping-table state. These logs provide additional technical evidence during debugging and firmware validation.

B.2 PCP Result Codes Used During Validation

Table B.1 summarizes the main PCP result codes used during the validation campaigns.

Table B.1: Main PCP result codes used during validation

Result code	**Validation condition**
`SUCCESS`	The request is successfully processed and the mapping is created, renewed, or deleted.
`UNSUPP_VERSION`	The requested PCP version is not supported by the server.
`MALFORMED_REQUEST`	The request contains invalid or inconsistent parameters.
`UNSUPP_OPTION`	The request contains an unsupported PCP option.
`NETWORK_FAILURE`	A network-side failure is simulated during the validation scenario.
`NO_RESOURCES`	The server cannot allocate the resources required for a new mapping.
`USER_EX_QUOTA`	The subscriber has reached the configured mapping limit.
`CANNOT_PROVIDE_EXTERNAL`	The requested external port is unavailable and port preservation is required.

Appendix C

Additional NetSupervision Interfaces and Execution Logs

Figure C.1 presents the user registration interface of the NetSupervision platform.

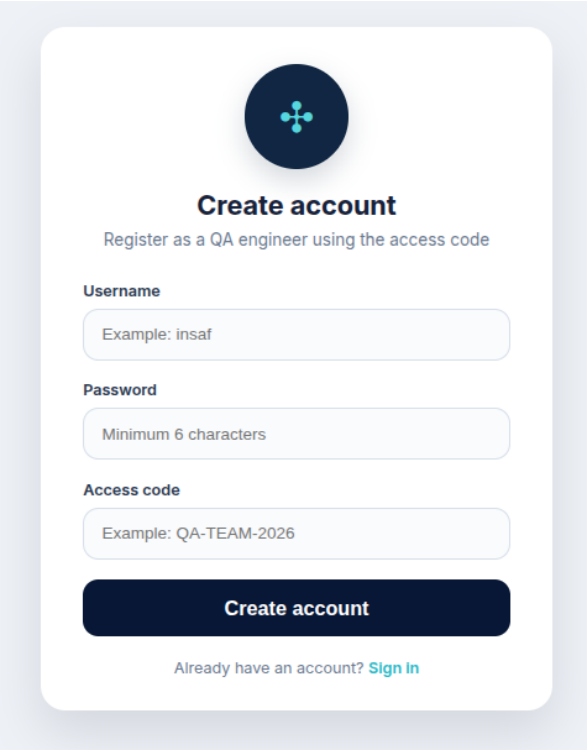


Figure C.1: User registration interface.

C.1 LAN Agent Deployment Interfaces

Figure C.2 presents the LAN Agent installation interface, while Figure C.3 illustrates the successful startup and registration of the agent with the NetSupervision platform.

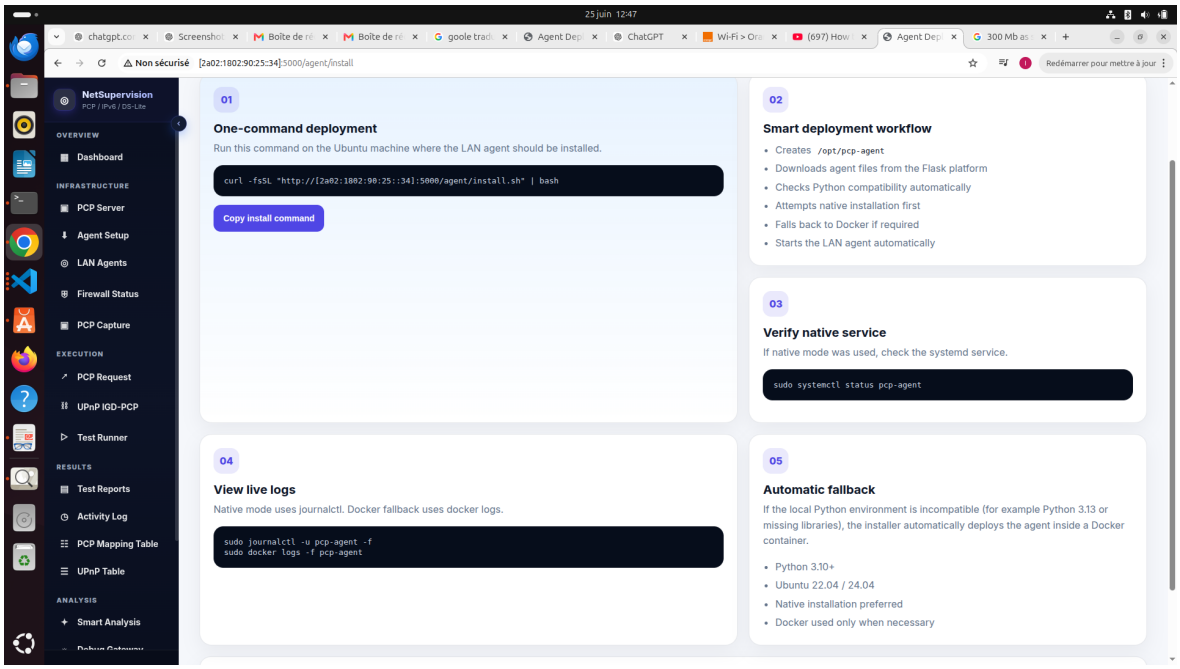


Figure C.2: LAN Agent installation interface.

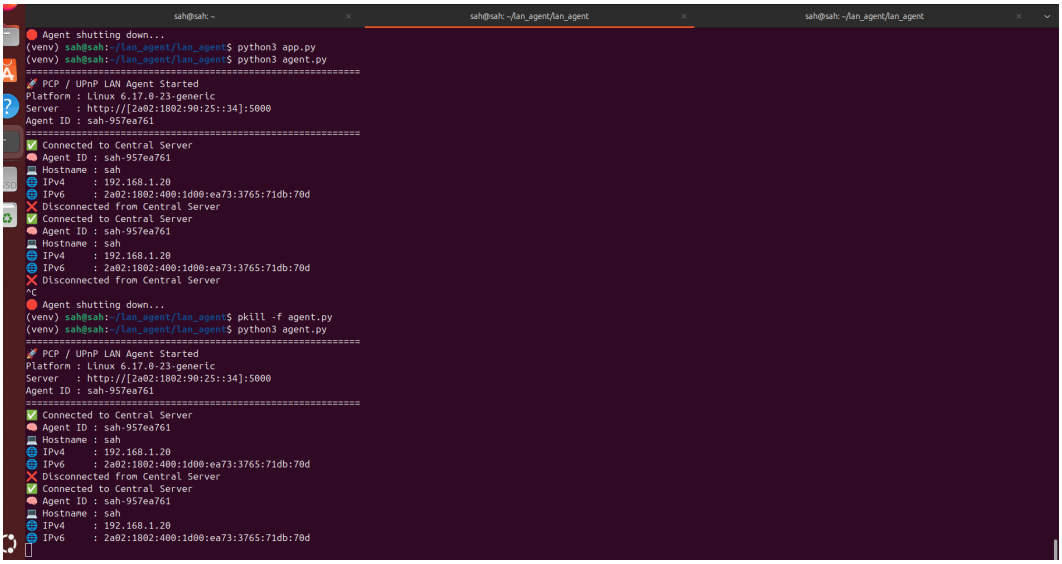


Figure C.3: LAN Agent startup and registration with the platform.

C.2 UPnP Execution Logs

Figure C.4 presents the detailed execution logs generated during UPnP validation.

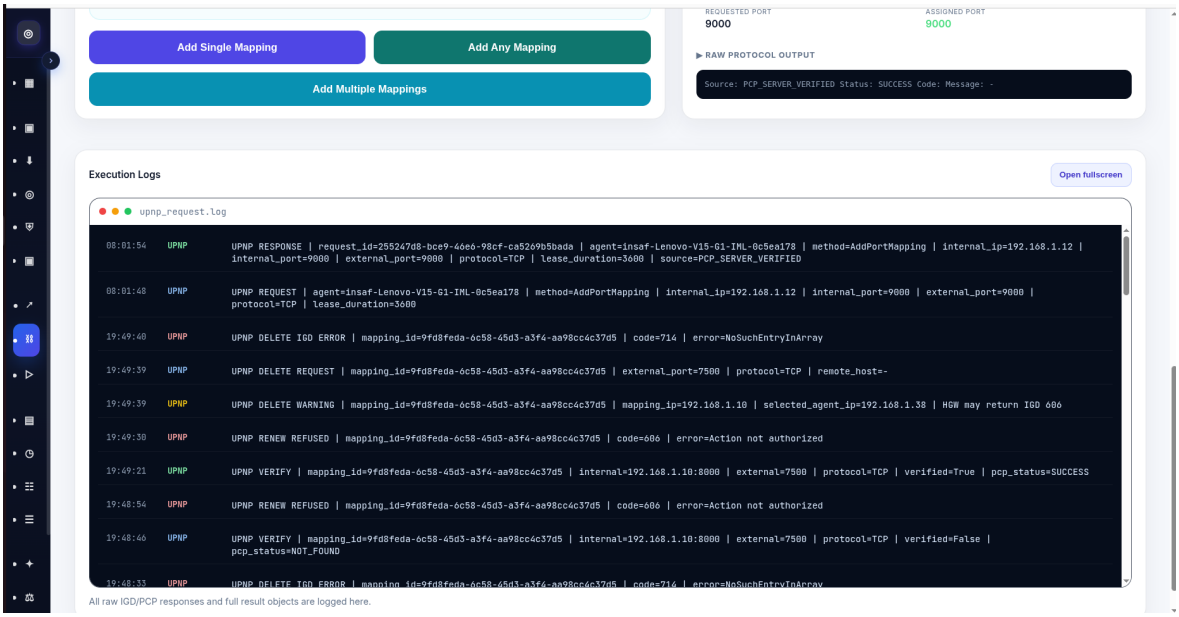


Figure C.4: Execution logs generated during UPnP validation.

C.3 UPnP Mapping Management Interface

Figure C.5 presents the interface used to manage UPnP mappings created through the IGD–PCP interworking function.

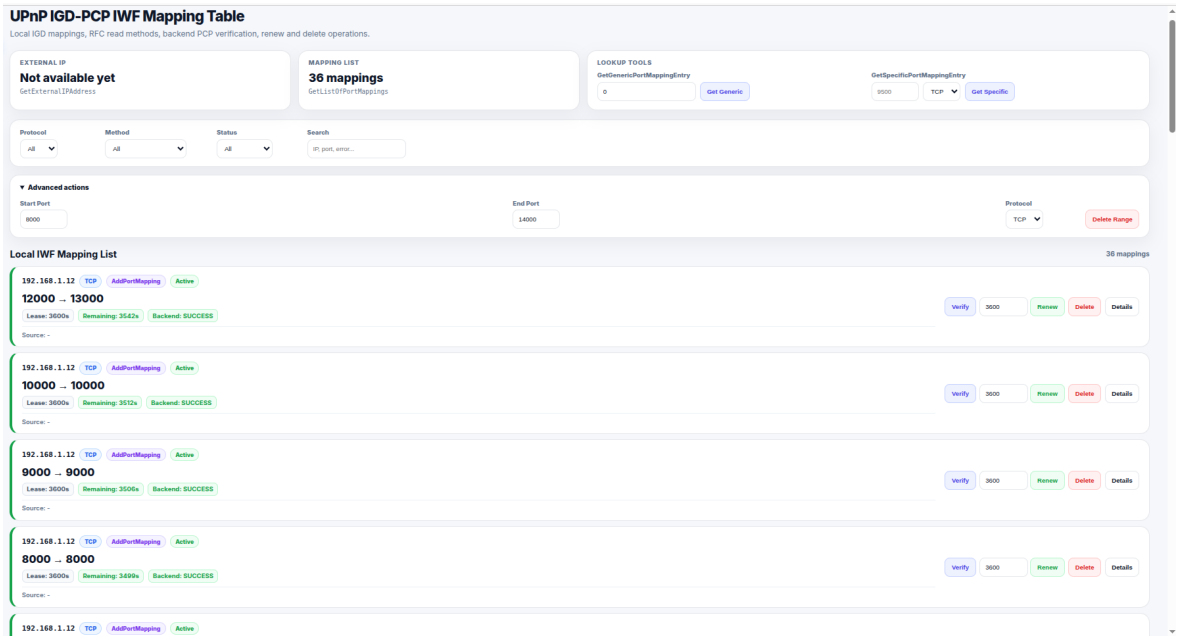


Figure C.5: UPnP IGD–PCP mapping management interface.

C.4 Regression Execution Logs

Figure C.6 presents the detailed execution evidence generated for an individual regression test.

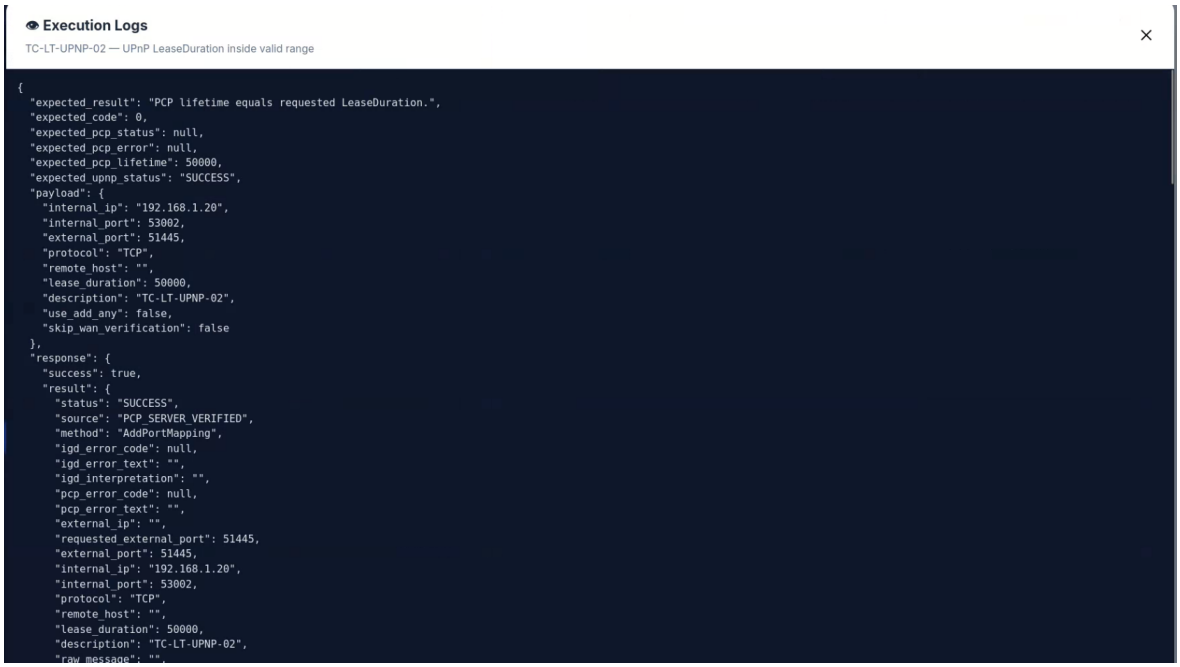
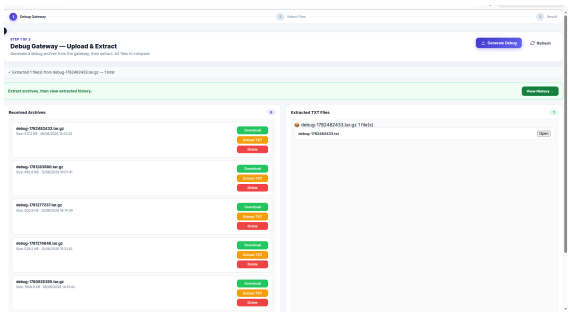


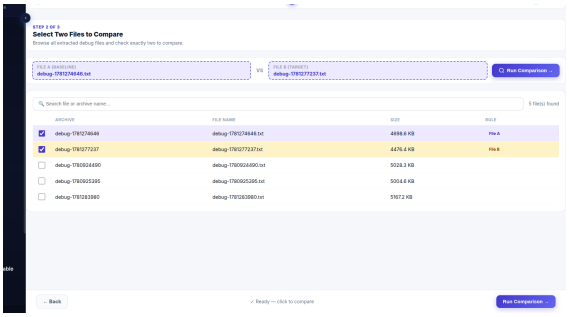
Figure C.6: Detailed execution logs generated by the regression module.

C.5 Debug Collection Interfaces

The following interfaces provide additional details about the automatic extraction and management of Home Gateway debug files.



(a) Automatic extraction of debug files.



(b) Repository of extracted debug files.

Figure C.7: Debug extraction and file management interfaces.

C.6 Monitoring and Traceability Interfaces

This appendix presents the complementary monitoring interfaces implemented in NetSupervision. These interfaces provide detailed visibility into protocol activities, user actions, and LAN Agent execution events. They complement the monitoring and traceability mechanisms introduced in Chapter 4.

C.6.1 Activity Logs

The Activity Logs interface provides a chronological view of PCP and UPnP validation events. It records requests, responses, mapping operations, verification results, warnings, and protocol errors. Filtering and search features allow QA engineers to locate specific events, while the recorded data can also be exported in CSV format.

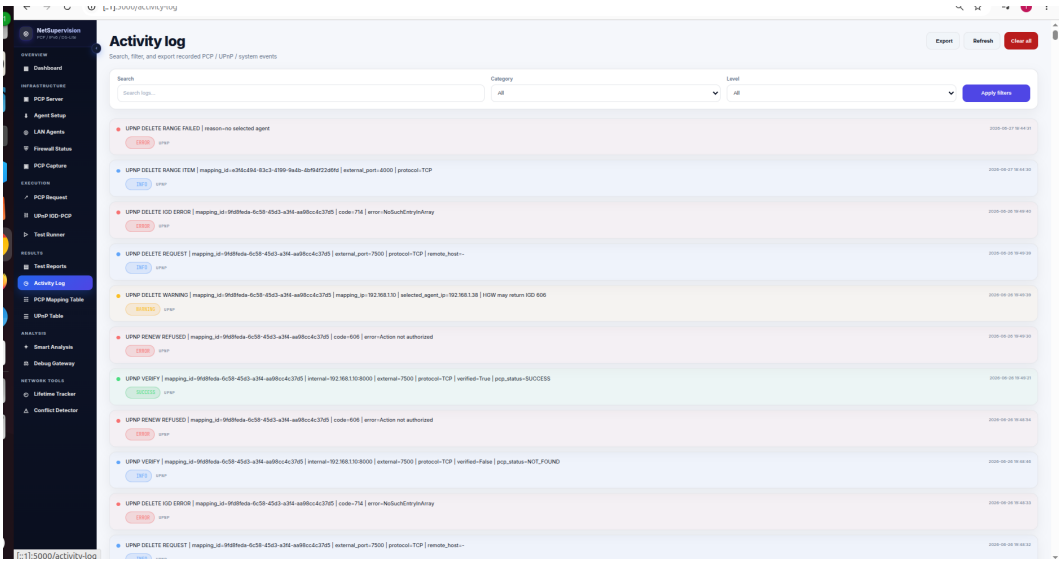


Figure C.8: Centralized activity log for PCP and UPnP validation events.

C.6.2 Audit and User Traceability

The Audit Log interface records administrative and user-related actions performed through the platform. Each entry identifies the authenticated user, the executed action, the affected module, and the execution time. This information ensures traceability during collaborative validation activities.

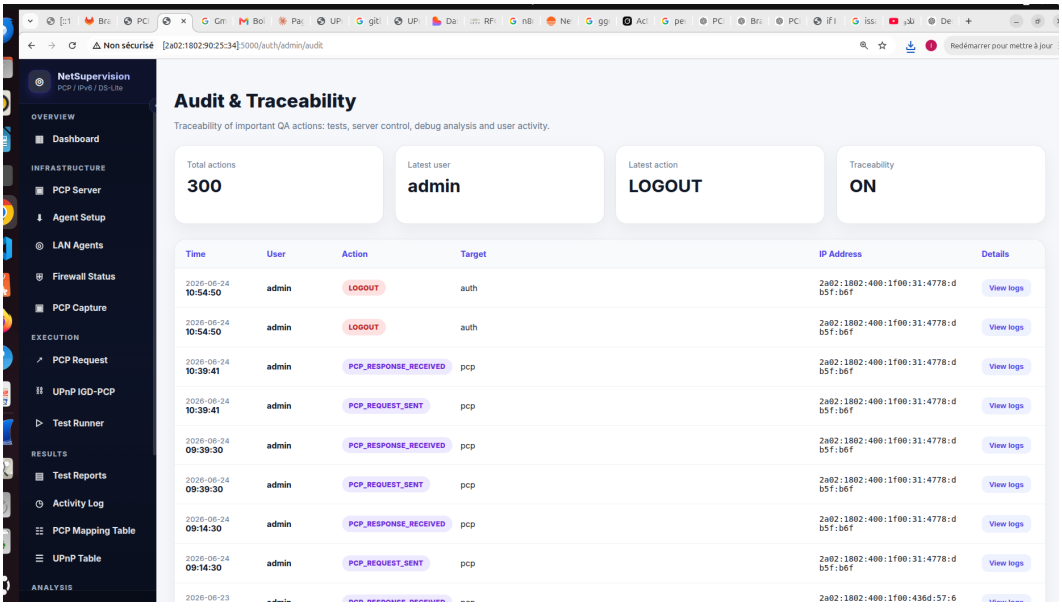


Figure C.9: Audit log recording administrative actions performed on the platform.

C.6.3 LAN Agent Logs

The LAN Agent Logs interface provides information about the operation of distributed validation workstations. It records agent connections, disconnections, selection events, communication warnings, and execution status, helping engineers diagnose communication or execution problems.

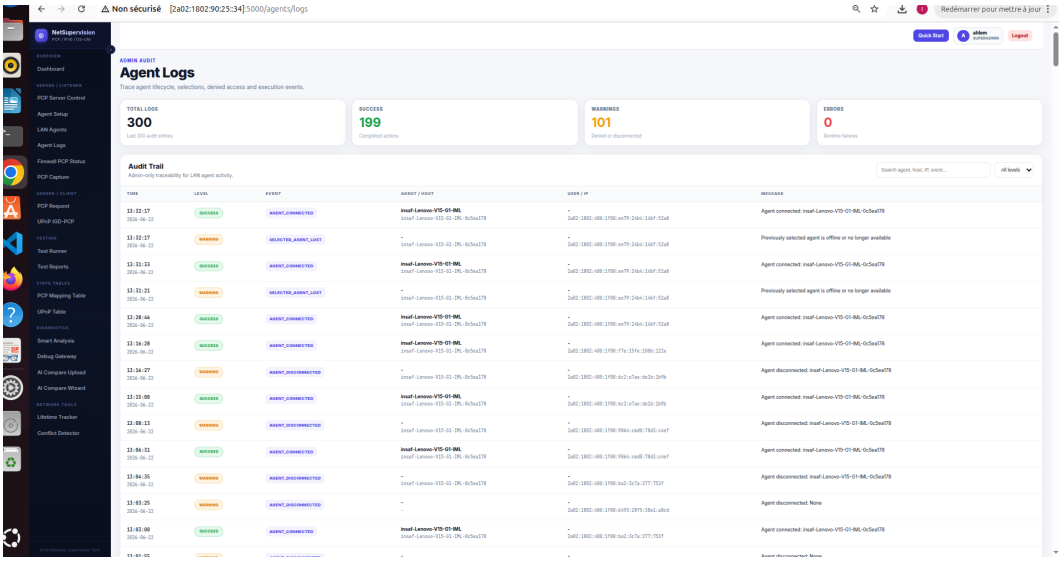


Figure C.10: Monitoring of distributed LAN Agent activity.

Bibliography

[1] TELNET Group, "Business domains," https://groupe-telnet.com/en/business-domain/, 2026, accessed: February 4, 2026.

[2] A. Durand, R. Droms, J. Woodyatt, and Y. Lee, "Dual-stack lite broadband deployments following IPv4 exhaustion," RFC Editor, RFC 6333, Aug. 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6333

[3] D. Wing, S. Cheshire, M. Boucadair, R. Penno, and P. Selkirk, "Port control protocol (PCP)," RFC Editor, RFC 6887, Apr. 2013. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6887

[4] M. Boucadair, R. Penno, and D. Wing, "Universal plug and play (UPnP) internet gateway device (IGD)–port control protocol (PCP) interworking function," RFC Editor, RFC 6970, Jul. 2013. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6970

[5] TELNET Holding, "About us," https://groupe-telnet.com/fr/about-us/, 2025, accessed: February 3, 2026.

[6] J. Postel, "Internet protocol," RFC Editor, RFC 791, Sep. 1981. [Online]. Available: https://www.rfc-editor.org/rfc/rfc791

[7] S. Perreault, I. Yamagata, S. Miyakawa, A. Nakagawa, and H. Ashida, "Common requirements for carrier-grade NATs (CGNs)," RFC Editor, RFC 6888, Apr. 2013. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6888

[8] E. Nordmark and R. Gilligan, "Basic transition mechanisms for IPv6 hosts and routers," RFC Editor, RFC 4213, Oct. 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4213

[9] M. Bagnulo, P. Matthews, and I. van Beijnum, "Stateful NAT64: Network address and protocol translation from IPv6 clients to IPv4 servers," RFC Editor, RFC 6146, Apr. 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6146

[10] M. Bagnulo, A. Sullivan, P. Matthews, and I. van Beijnum, "DNS64: DNS extensions for network address translation from IPv6 clients to IPv4 servers," RFC Editor, RFC 6147, Apr. 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6147

[11] Python Software Foundation, "Python 3 documentation," https://docs.python.org/3/, accessed: April 13, 2026.

[12] Pallets Projects, “Modular applications with blueprints,” https://flask.palletsprojects.com/en/stable/blueprints/, accessed: May 4, 2026.

[13] ——, “Jinja documentation,” https://jinja.palletsprojects.com/en/stable/, accessed: May 7, 2026.

[14] WHATWG, “HTML living standard,” https://html.spec.whatwg.org/, accessed: May 11, 2026.

[15] World Wide Web Consortium, “CSS snapshot 2026,” https://www.w3.org/TR/css-2026/, 2026, accessed: May 11, 2026.

[16] Ecma International, “ECMAScript language specification,” https://tc39.es/ecma262/, accessed: May 13, 2026.

[17] M. Grinberg, “Python Socket.IO server documentation,” https://python-socketio.readthedocs.io/en/stable/server.html, accessed: May 18, 2026.

[18] T. Bernard, “MiniUPnPc project,” https://miniupnp.tuxfamily.org/, accessed: April 6, 2026.

[19] J. Postel and J. Reynolds, “Telnet protocol specification,” RFC Editor, RFC 854, May 1983. [Online]. Available: https://www.rfc-editor.org/rfc/rfc854

[20] Paramiko Development Team, “Paramiko documentation,” https://docs.paramiko.org/en/stable/, accessed: June 8, 2026.

[21] SQLite Development Team, “About SQLite,” https://www.sqlite.org/about.html, accessed: May 25, 2026.

[22] SQLAlchemy Authors, “SQLAlchemy object relational mapper documentation,” https://docs.sqlalchemy.org/en/20/orm/, accessed: May 27, 2026.

[23] Pallets Projects, “Werkzeug security helpers,” https://werkzeug.palletsprojects.com/en/stable/utils/#module-werkzeug.security, accessed: June 1, 2026.

[24] ——, “Flask sessions,” https://flask.palletsprojects.com/en/stable/quickstart/#sessions, accessed: June 2, 2026.

[25] libpcpnatpmp Project, “libpcpnatpmp: NAT-PMP and PCP client/server library,” https://github.com/libpcpnatpmp/libpcpnatpmp, 2025, gitHub repository, accessed June 15, 2026.

[26] Orange, “Functional specifications for livebox 4 g4r0,” Orange, Internal Technical Specification, 2020, document version S0F17.